



slington college
(इस्लिङ्टन कलेज)

CS5054NI Advanced Programming & Technologies

50% Group Coursework 1

Sprint Semester 2025

15 Credit Semester Long Module

Group Information

London Met ID	Student Name
23050262	Sujal Parajuli
23050342	Yathartha Shrestha
23049338	Sakshyam Kafle
23047617	Niran Bhatta
22067721	Waseem Malik

Project Title: Ecommerce Website

Assignment Due Date: Thursday, May 22, 2025

Assignment Submission Date: Thursday, May 22, 2025

Submitted to: Mr. Prithivi Maharjan

Word Count: 14606

GitHub Link:	https://github.com/sthaizen/AuraTimes.git
---------------------	---

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Abstract:

Aura times are an e commerce website selling high quality watches. This project report provides an account on development of Aura times. This project's basic goal was to develop and implement an elegant, user friendly, and responsive web platform as an application for customers to browse, select and buy watches easily. With a modern UI/UX design, product categorization, detailed product page and a secure checkout, the website offers a lot of engaging 'About' section with animation, interactivity.

Front end and back-end integration was done using HTML, CSS, JavaScript, JSP, and MySQL database that was used for management of product and user data during a development process. Moreover, the project not only enabled the development of technical and design skills but also gave an insight into actual e-commerce solution and user centric design principles.

Acknowledgement:

I would also like to express my heartfelt thanks to Mrs. Rabina Lama, my valued module tutor for Advance Programming, for his constant guidance, healthy critiques, and insightful suggestions in the course of developing this e-commerce website project. Her advice was crucial in helping me remain on track, hurdle variances, and improve both the functionality of the site as well as its aesthetic appeal.

I am also sincerely indebted to my seniors and friends, who have constantly encouraged me and provided constructive recommendations at every step of the project. Their recommendations were instrumental in helping me overcome a number of technical and design-related issues and enhancing the features and overall user interface of the website. Their willingness to assist and provide their knowledge really helped in my learning process.

Finally, I would like to thank Islington College for providing me with the facilities, equipment, and supportive learning environment to achieve innovative thinking and technical research. The development of this project has not just been a fulfilling academic achievement but also an invaluable and inspirational learning experience that I can continue with my future goals.

Table of Contents

1) Introduction.....	1
1.1) Purpose:	1
1.2) Audience:	2
1.3) Aims and Objectives	2
1.4) List of Features:	3
1.4.1) User and Product page features:	3
1.4.2) Admin Features	3
2) Database Design and Development	4
2.1) Normalization:	4
2.1.1) Un-Normalized Form (UNF):	4
2.1.2) First Normal Form (1NF)	4
2.1.3) Second Normal Form (2NF) Conversion	5
2.1.4) Third Normal Form (3NF) Conversion	6
2.1.5) Final Table after 3NF:.....	8
2.2) Entity Relationship Diagram	9
2.3) Database Development:	10
3) Wireframe and Design.....	12
3.1) Home Page	12
3.1.1) Research Reference	12
3.1.2) Actual design	14
3.2) Product Page	15
3.2.1) Research Reference	15
3.2.2) Actual Design	17
3.3) Add to Cart Page	18
3.3.1) Research Reference	18
3.3.2) Actual Design:	20
3.4) Side Bar.....	21
3.4.1) Research Reference	21
3.4.2) Actual Design:	23
3.5) Login Page.....	24
3.5.1) Research Reference	24

3.5.2) Actual Design:	25
3.6) Registration Page:.....	26
3.6.1) Research Reference	26
3.6.2) Actual Design:	27
3.7) Contact page:	28
3.7.1) Research Reference:	28
3.7.2) Actual Design:	29
3.8) User profile page:	30
3.8.1) Research Reference:	30
3.8.2) Actual Design:	31
3.9) About us page	32
3.9.1) Research Reference	32
3.9.2) Actual Design	33
3.10) Admin Dashboard:	34
3.10.1) Research Reference	34
3.10.2) Actual Design No.1:	35
3.10.3) Actual Design No.2:	36
3.10.4) Actual Design No.3:	37
4) Java Classes Explanation	38
4.1) Overall, Class Diagram:	38
4.2) Individual Class Diagram	39
4.2.1) DBconfig.java.....	39
4.2.2) ProductModel.java	40
4.2.3) CollectionServices.java	42
4.2.4) CookieUtil.java:	43
4.2.5) ImageUtil.java	44
4.2.6) PasswordUtil.java.....	45
4.2.7) SessionUtil.java	46
4.2.8) ValidationUtil.java	47
4.2.9) AboutUsController.java	50
4.2.10) AddToCartController.java	51
4.2.11) CollectionController.java.....	52

4.2.12) ContactController.java	53
4.2.13) HomeController.java.....	55
4.2.14) LoginController.java	56
4.2.15) LogoutController.java	57
4.2.16) RegisterController.java.....	58
4.2.17) AboutUsController.java.....	60
4.2.18) AddToCartController.java	61
4.2.19) CollectionController.java.....	62
4.2.20) ContactController.java	64
4.2.21) HomeController.java.....	65
4.2.22) LoginController.java	67
4.2.23) LogoutController.java	68
4.2.24) RegisterController.java.....	69
4.2.25) Admin Dashboard Controller	70
4.2.26) Admin Profile Controller	71
4.2.27) Initial Controller	71
4.2.25) Dashboard Product Controller	72
4.2.26) Filter Product Controller.....	72
4.2.27) User Profile Controller	73
4.2.28) Store Location Controller	73
4.2.29) Search Product Controller.....	74
4.2.25) Ysindex Controller	74
5.1.1) Test 1 User Registration	75
5.1.2) Test 2 User Login	81
6) Development	94
6.1) UI/UX Development	94
6.2) Authorization and Authentication	94
6.2.1) Register Page	94
6.2.2) Login Page	95
6.2.3) Role-based page access Redirection	97
6.3.4) Session and Cookie.....	98
7) Coursework Development with Analysis.	103

7.1) Team Contribution:.....	103
7.2) Critical Analysis.	111
Challenge 1 Yathartha Shrestha:.....	111
Challenge 2 Sujal Parajuli:.....	112
Challenge 3 Sakshyam Kafle:	113
Challenge 4 Niran Bhatta:.....	114
8) Tools Used:	115
8.1) XMAPP.....	115
Tools Used:.....	115
9) Dynamic user profile:	124
10) Meeting Logs	126
11) Conclusion:	132

Table of Figure:

Figure 1 - ER Diagram of WATCH Resale	9
Figure 2 - System Database.....	10
Figure 3- Screenshot of the referenced UI (Rolex, 2009).....	12
Figure 4 - Wireframe for home page	13
Figure 5 - Home Page Actual Design	14
Figure 6 - Reference page for product page (nextgen, 2025)	15
Figure 7 - Wireframe for product page	16
Figure 8 - Product Page UI	17
Figure 9 - Reference page for add to cart page (NIKE, 2025).....	18
Figure 10 - Wireframe for add to cart page	19
Figure 11 - Add to Cart Page UI	20
Figure 12 - Reference for side bar (amazon, 2025)	21
Figure 13 - Wireframe for side bar.....	22
Figure 14 - Side bar UI	23
Figure 15 - Reference for login page	24
Figure 16 - Actual design for login page.....	25
Figure 17 Reference for registration Page (Webflow, 2025)	26
Figure 18 - Actual design for registration page.....	27
Figure 19 Contact Page Reference	28
Figure 20 - Actual design for contact us page.....	29
Figure 21 - Reference for user profile.....	30
Figure 22 - Actual design for user page	31
Figure 23 - Research reference for about us page	32
Figure 24 - Actual design for about us page	33
Figure 25 Admin Dashboard Reference	34
Figure 26 Actual Design No.1	35
Figure 27 Actual Design No.2	36
Figure 28 Actual Design No.3	37
Figure 29 Final Class Diagram	38
Figure 30 - Class diagram for DBconfig	39
Figure 31 - Class diagram for ProductModel.....	40
Figure 32 - Class diagram for CollectionServices	42
Figure 33 - Class diagram for CookieUtil	43
Figure 34 - Class diagram for ImageUtil.....	44
Figure 35 - Class diagram for PasswordUtil	45
Figure 36 - Class diagram for SessionUtil.....	46
Figure 37 - ValidationUtil.....	47
Figure 38 - Class diagram for AboutUsController	50
Figure 39 - Class diagram for AddToCartController	51
Figure 40 - Class diagram for CollectionController.....	52
Figure 41 - Class diagram for ContactController	53

Figure 42 - Class diagram for Home Controller	55
Figure 43 - Class diagram for LoginController	56
Figure 44 - Class diagram for LogOutController	57
Figure 45 - Class diagram for RegisterController	58
Figure 39 - Class diagram for AboutUsController	60
Figure 40 - Class diagram for AddToCartController	61
Figure 41 - Class diagram for CollectionController	62
Figure 42 - Class diagram for ContactController	64
Figure 43 - Class diagram for Home Controller	65
Figure 44 - Class diagram for LoginController	67
Figure 45 - Class diagram for LogOutController	68
Figure 46 - Class diagram for RegisterController	69
Figure 47 Class diagram for Admin Dashboard Controller	70
Figure 48 Class diagram for Admin Profile Controller	71
Figure 49 Class diagram for Initial Controller	71
Figure 50 Class diagram for Dashboard Product Controller	72
Figure 51 Class diagram for Filter Product Controller	72
Figure 52 Class diagram for User Profile Controller	73
Figure 53 Class diagram for Store Location Controller	73
Figure 54 Class diagram for Search Product Controller	74
Figure 55 Class diagram for Ysindex Controller	74
Figure 46 - Evidence of test 1 (Invalid Username)	75
Figure 47 - Evidence of test 1 (password strength)	76
Figure 48 - Evidence of test 1 (invalid gender)	77
Figure 49 - Evidence of test 1 (Nepali number only)	78
Figure 50 - Evidence of test 1 (Unique username)	79
Figure 51 - User details stored in database test 6	80
Figure 52 - Showing wrong credentials	81
Figure 53 Handling In browser	82
Figure 54 redirect to the admin page	83
Figure 55 Cookie Validation	84
Figure 56 Cookie Validation Evidence	84
Figure 57 User table	85
Figure 58 Login success	86
Figure 59 test	86
Figure 60 - Before adding products	87
Figure 61 - Product added successfully	88
Figure 62 - Product update in collection page	89
Figure 63 - Product updated in product detail page	90
Figure 64- add to cart page before adding products	91
Figure 65 - Adding product on add to cart page	92
Figure 54 – Code for Get Request Handling	96
Figure 55 - Code of add cookie util which defines utility	99

Figure 56 – Code of get cookie.....	99
Figure 57 - Code for delete cookie	100
Figure 58 - code for set Attribute	101
Figure 59 - code for get Attribute	101
Figure 60 - code for remove Attribute	102
Figure 61 - code for invalidate session Attribute.....	102
Figure 69 Team Contribution.....	103
Figure 70 Evidence 1 Login Form	104
Figure 71 Final Implementation of Login page.....	105
Figure 72 - Final implementation of home page	106
Figure 73 - Showing evidence of About us page.....	108
Figure 74 - Final implementation of about us page	109
Figure 75 Error for Challenges 1	111
Figure 76 Error for Challenges 2	112
Figure 77 Fixing the Challenges 3	113
Figure 78 Error for Challenges 4	114
Figure 79 Tools used 1	115
Figure 80 - SS of XAMPP control panel.....	116
Figure 81 Tools used 2	117
Figure 82 Tools used 3	119
Figure 83 Tools used 4	121
Figure 84 Tools used 5	122
Figure 85 - Code showing servlet mapping	124
Figure 86 - Code showing servlet uses GET requests	124
Figure 87 - Code showing servlet uses POST requests	125

Title: E-Commerce Website for Watch Sales – Aura Time**1) Introduction**

Watches are more than just timekeepers; watches also represent style, creativity and who we are. The focus of Aura Times is to link those who appreciate watches with premium used watches that are easy on the budget. If you want to buy a luxury timepiece for the first time or sell a vintage watch, Aura Times is there to help. Because the platform is built with Java, it guarantees proper performance, safety of transactions and an easy interface for all users. Because more people are becoming watch fans, Aura Times wishes to provide a safe online marketplace where style meets ease of shopping. Take initiative to explore and own your schedule at our unique platform Aura Times.

Aura Times is a new platform designed for reselling luxury watches, providing customers an easy and reliable way to purchase and sell them. Powered by Java technologies, the website is dependable, easy to navigate and comfortable for all users. If you dream about watches or want to add to your collection, Aura Times brings together an elegant, easy-to-use and affordable online marketplace.

1.1) Purpose:

Aura Times is designed to make it easy, fast and secure for buyers and sellers to resell top-notch and luxury watches on the Internet. The idea is to link people who want to buy or sell quality timepieces through a safe and reliable digital marketplace. With Java-based technologies, the website ensures everything a function smoothly, gets real-time updates and the user finds everything easy to use. Aura Times helps preserve the environment by restoring old, high-end watches for many buyers, making luxury watches more affordable to a larger group. Therefore, the platform is a helpful place for watch lovers to exchange, discover and admire everlasting elegance.

1.2) Audience:

The audience for this website is:

- Watch collectors looking for high-quality and unique timepieces.
- Companies and brands who want to sell and promote their watches online.
- Organizations in search of an efficient platform for inventory and sales management.
- Style Conscious Buyers who view watches as a fashion accessory.
- Users who want to own a high end watch at a lower cost.

1.3) Aims and Objectives

The main aims and objectives of the project are:

- To create and design a fully functional Java-based e-commerce website using Eclipse IDE.
- To create an interactive and user-friendly interface for an enjoyable shopping experience.
- To include secure payment gateways for secure transactions.
- To implement a robust inventory management system for tracking product availability.
- To enhance user engagement with features like product search, filtering, and suggestions.

1.4) List of Features:

1.4.1) User and Product page features:

- **User Authentication:** Secure sign-up and sign-in for administrators and buyers.
- **Search Bar:** The platform allows searches through its bar so users can filter products according to specified names or exact keywords.
- **Filters:** Sort by price (low-high, high-low).
- **Order Status:** The order status update function enables administrators to select the shipped status for customer orders.
- **Product catalogue:** Display of watches with images, descriptions, specifications, and prices.
- **Search and Filter:** Advanced search and filtering feature by brand, style, price, and features.
- **Shopping Cart:** Add, remove, and modify items before payment.
- **Order Management:** Users can view their orders and have management over their buying history.
- **Payment Integration:** Secure payment gateway for safe and seamless transactions.

1.4.2) Admin Features

- **Admin Login** – Separate login for admins (simple password check).
- **Add/Edit Products** – Form to insert or update product details (name, price, stock).
- **View Orders** – List all customer orders with status updates ("Pending," "Shipped").
- **Manage Users** – View/delete users (CRUD).

2) Database Design and Development

2.1) Normalization:

It is a data organizing processing a database in such a manner that duplication reduces, and accuracy increases. The normalizing of databases makes them perform optimally by reducing lots of problems during adding, deleting, or updating data. In normalization, data is organized in tables, and the relationship between different tables is established with the help of certain rules called normal forms. These rules keep the database organized, efficient, and error-free

2.1.1) Un-Normalized Form (UNF):

- Here Data is stored in the form it is collected, and no effort is made to eliminate redundancy or guarantee atomicity.
- It allows repeating groups and non-atomic attributes (attributes with multiple values or sets).

UNF:

User: (User_ID, UserName, Password, Email, Phone_Number, Address, {Product_ID, Product_Name, Description, Product_Price}, {Cart_Item_ID, Cart_Item_Quantity, Order_Item, {Order_ID, Total_Amt, Delivery_Address, Payment_ID, Payment_Method, Payment_Status, Payment_Created_At}}).

2.1.2) First Normal Form (1NF)

Rules Applied:

- No repeating groups (flatten nested structures into separate tables).
- All attributes are atomic (single values, no composite fields).

1NF Table Derived from UNF:**I. User Table:**

- (User_ID, UserName, Password, Email, Phone_Number, Address)

II. User, Product Table:

- (Product_ID, Product_Name, Description, Product_Price, User_ID*)

III. User, Cart Table:

- (Cart_Item_ID, Cart_Item_Quantity, Order_Item, User_ID*)

IV. User, Cart and Order Table:

- (Order_ID, Total_Amt, Delivery_Address, Payment_ID, Payment_Method, Payment_Status, Payment_Created_At, User_ID*, Cart_ID*)

2.1.3) Second Normal Form (2NF) Conversion**Rules Applied:**

- Removed partial dependencies (non-key attributes must depend on the entire primary key).
- Separated composite relationships into distinct entities.

2NF Table Derived from 1NF:**I. User Table (Already in 2NF):**

- (User_ID, UserName, Password, Email, Phone_Number, Address)

II. Product Table (Created for 2NF):

- (Product_ID, Product_Name, Description, Product_Price)

III. User, Product Table (2NF):

- (Product_ID*, User_ID*)

IV. Cart Table (Already in 2NF):

- (Cart_Item_ID, Cart_Item_Quantity, Order_Item)

V. User, Cart Table (2NF):

- (Cart_Item_ID*, User_ID*)

VI. User, Cart and Order Table (2NF):

- (Order_ID*, User_ID*, Cart_ID*)

VII. Order Table (Modified for 2NF):

- (Order_ID, Total_Amt, Delivery_Address, Payment_ID, Payment_Method, Payment_Status, Payment_Created_At)

Reasoning for the Changes:

- The separation of shared product and cart data from the user through changes to the User, Product and User, Cart tables eliminated partial dependencies.
- A product table exists independently from other entities to prevent duplicate storage of product records for individual users.
- The Order and Payment-related attributes were grouped where they aligned logically.

2.1.4) Third Normal Form (3NF) Conversion**Rule Applied:**

- Removed transitive dependencies (non-key attributes depending on non-primary attributes).

3NF derived from 2NF:**I. User Table:**

- (User_ID, UserName, Password, Email, Phone_Number, Address)

II. Product Table:

- (Product_ID, Product_Name, Description, Product_Price)

III. User, Product Table:

- (Product_ID*, User_ID*)

IV. Cart Table:

- (Cart_Item_ID, Cart_Item_Quantity, Order_Item)

V. User, Cart Table:

- (Cart_Item_ID*, User_ID*)

VI. User, Cart and Order Table:

- (Order_ID*, User_ID*, Cart_ID*)

VII. Order Table:

- (Order_ID, Total_Amt, Delivery_Address)

VIII. Payment Table:

- (Payment_ID, Payment_Method, Payment_Status, Payment_Created_At)

IX. User, Order and Payment Table:

- (User_ID*, Order_ID*, Payment_ID*)

Reasoning:

- In the Order table, attributes like Payment_Method, Payment_Status, and Payment_Created_At are dependent on Payment_ID, not directly on Order_ID.
- To eliminate transitive dependency, a separate Payment Table was created.
- A new User, Order and Payment Table was created to link all three entities clearly via foreign keys.

2.1.5) Final Table after 3NF:

1. **User Table:** (User_ID, UserName, Password, Email, Phone_Number, Address)
2. **Product Table:** (Product_ID, Product_Name, Description, Product_Price)
3. **User, Product Table:** (Product_ID*, User_ID*)
4. **Cart Table:** (Cart_Item_ID, Cart_Item_Quantity, Order_Item)
5. **User, Cart Table:** (Cart_Item_ID*, User_ID*)
6. **User, Cart and Order Table:** (Order_ID*, User_ID*, Cart_ID*)
7. **Order Table:** (Order_ID, Total_Amt, Delivery_Address)
8. **Payment Table:** (Payment_ID, Payment_Method, Payment_Status, Payment_Created_At)
9. **User, Order and Payment Table:** (User_ID*, Order_ID*, Payment_ID*)

2.2) Entity Relationship Diagram

An Entity Relationship Diagram (ER Diagram) is a picture of a database set up. It aids in helping people design a new database or in seeing how an existing one works. It's also referred to as entity relationship model or graph sometimes (Lucidchart, 2022).

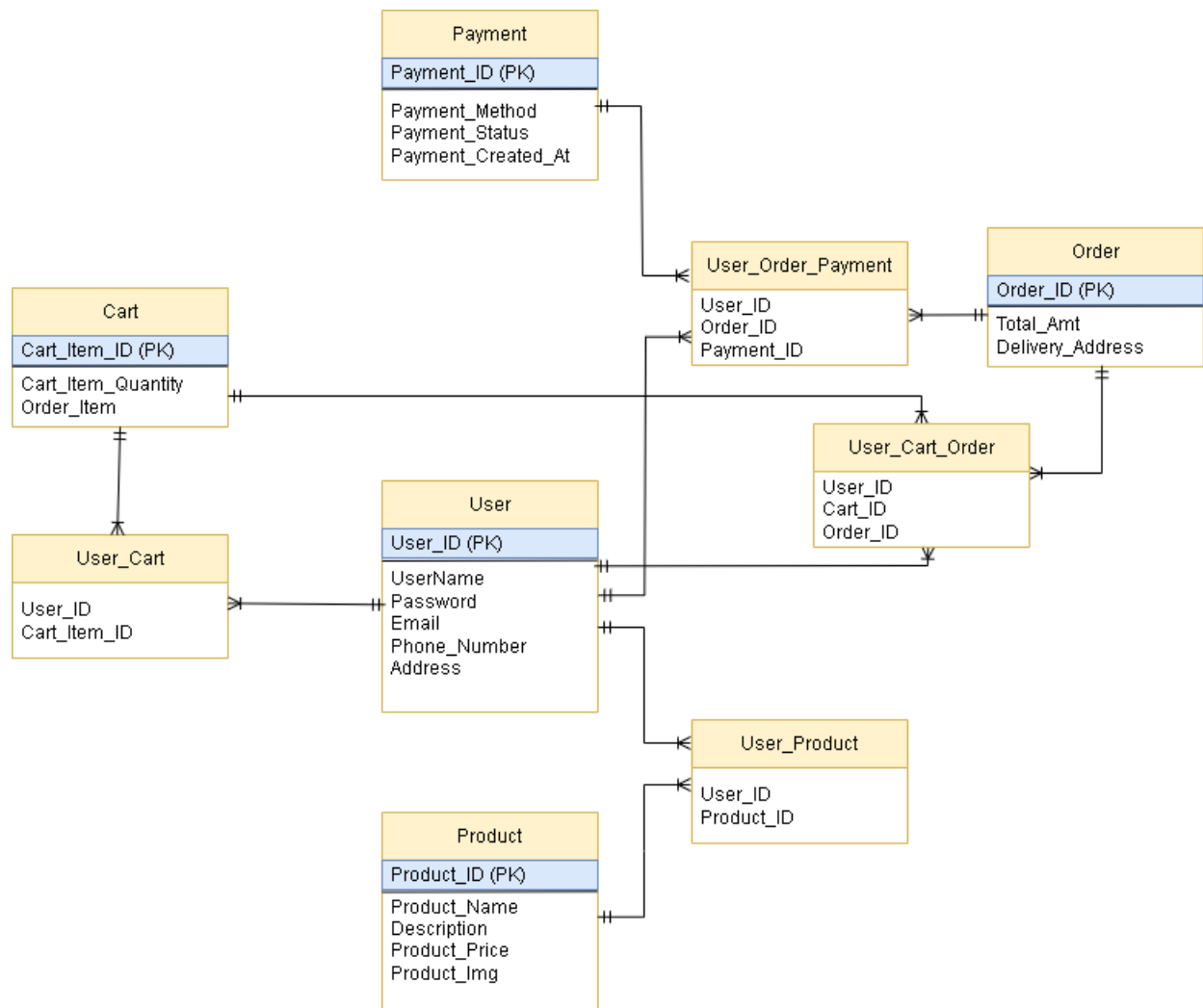


Figure 1 - ER Diagram of WATCH Resale

2.3) Database Development:

The watch website database operates as a system which manages user data and tracks system activity while giving administrators full control through consistent validation and integrity checks. User data consisting of credentials together with preferences browsing histories and administrative permissions shared a single UNF structure as the first design of the database. The normalization process of the database improved structure while eliminating redundancy and anomalies by using standard formats starting from 1NF up to 3NF. All repeating groups received specialized treatment in the First Normal Form (1NF) by converting data into atomic values that separated the tables for user details from watch preferences to browsing records through primary key enforcement. The Second Normal Form (2NF) served to break linkages between data components through the creation of distinct tables matched by foreign keys between User profiles and their attributes such as age, style preference and watch interests. Through the Third Normal Form implementation the administrators shifted their requirements into the Admi Profile table and enabled Watch Record table data management for independent watch viewing and saving records.

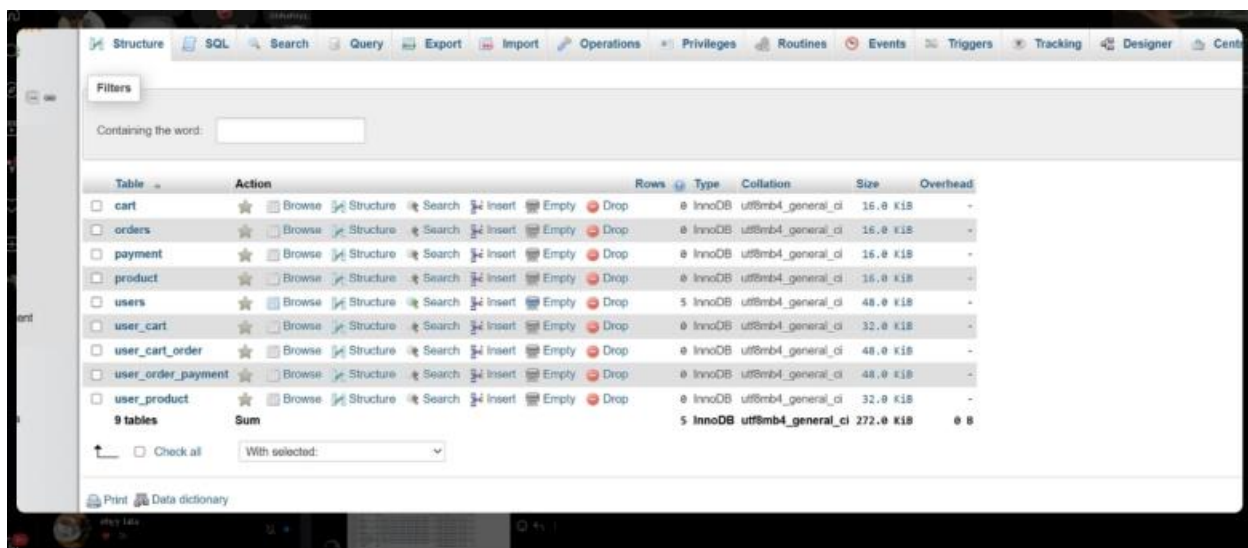


Table	Action	Rows	Type	Collation	Size	Overhead
cart	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 K1B	-
orders	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 K1B	-
payment	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 K1B	-
product	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 K1B	-
users	Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_general_ci	48.0 K1B	-
user_cart	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 K1B	-
user_cart_order	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 K1B	-
user_order_payment	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 K1B	-
user_product	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 K1B	-
9 tables	Sum	5	InnoDB	utf8mb4_general_ci	272.0 K1B	0 B

Figure 2 - System Database

This screen display shows how phpMyAdmin presents database structure for users to view. The central part features a list of tables that contains cart, orders, payment, product, and multiple user_ prefixed tables. Each table displays an available set of operations which permit users to examine data and explore structures or conduct searches to insert records and empty contents as well as drop tables from the database. Each table in this view presents essential information which includes row count data alongside storage engine details , character encoding uses utf8mb4_general_ci and table disk size data and potential storage overhead. A summary table beneath the listings reveals a total of eight database tables together with their accumulated size and overhead measurement. Database management functions can be accessed through the navigation bar located at the top while execution of SQL queries and data search capabilities as well as database import/export functions and operations management lie within these interfaces. A keyword entry filter at the left side reduces the tables shown in the interface. The image gives a compact summary of the database structure and offers all management tools for database administration.

3) Wireframe and Design

3.1) Home Page

3.1.1) Research Reference

The purpose of the research to gather inspiration in designing a premium and elegant user interface as the example would be the official web site of Rolex – a world-famous brand of luxury watches. It is rare in world of branding to find the Rolex website that has a proper layout, immersive visuals and user centric design. I feel that its interface fits the luxury and precision I want to give on my own project and reflects that very well.

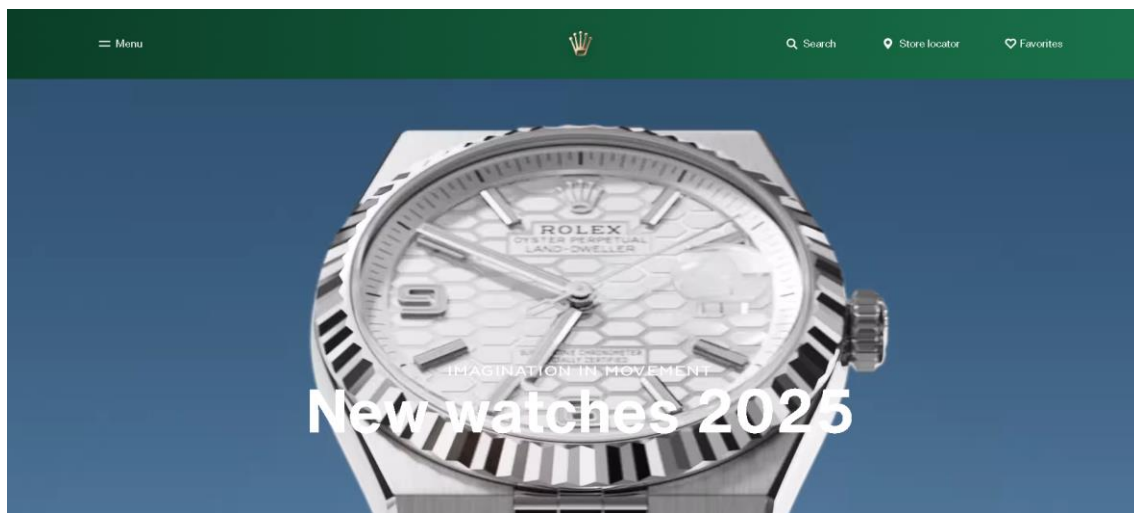


Figure 3- Screenshot of the referenced UI (Rolex, 2009)

Key Points That Impressed Me:

- **Premium Visual Design:** Professional, cinematic watch images that emphasize elegance and detail.
- **Smooth Transitions:** Smooth scrolling or hover transitions, adding an upscale feel to user interaction.
- **Organized Product Layout:** Convenient categorization and browsing capability for watches by type or collection.
- **Minimum and Clean Navigation:** A clear top menu bar that is simple but effective and clean, ensuring the users' attention remains focused on the product.

- **Storytelling Focus:** Storytelling descriptive text and history pages enhancing the emotional importance of the products.

Wireframe:

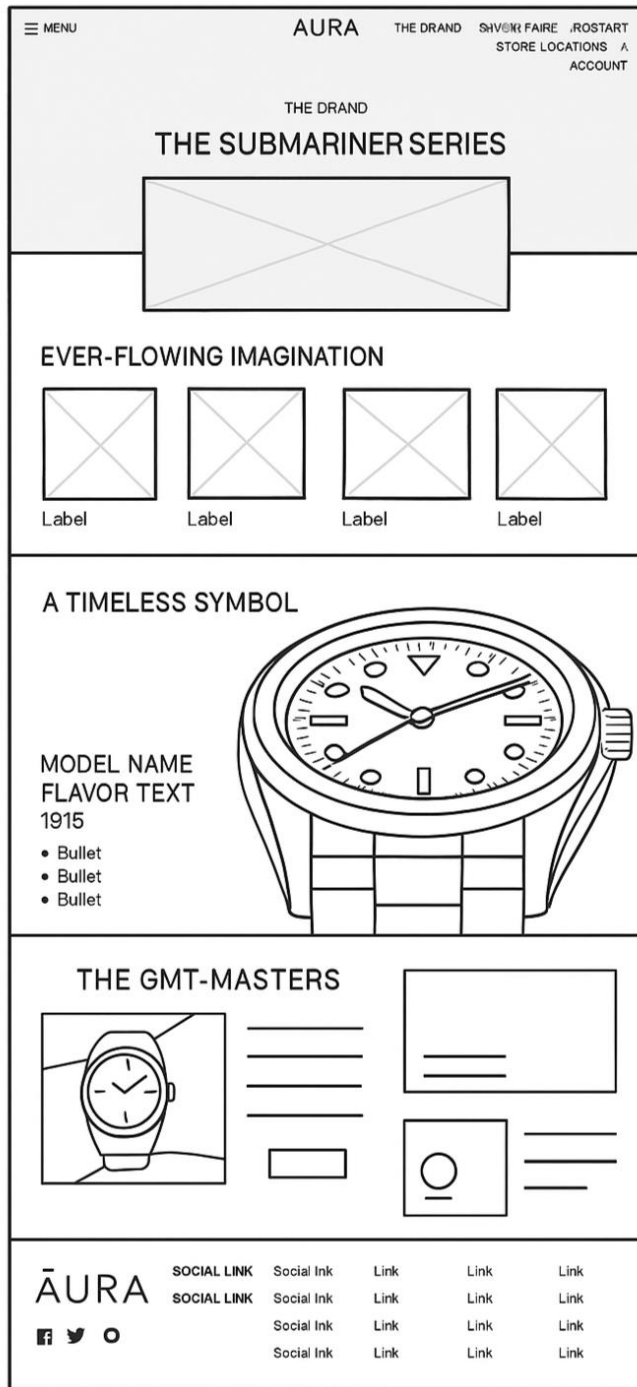


Figure 4 - Wireframe for home page

3.1.2) Actual design

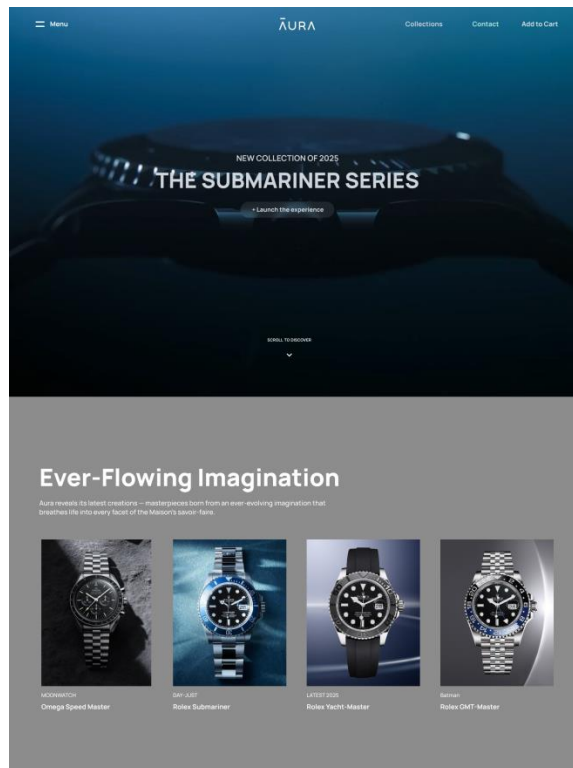


Figure 5 - Home Page Actual Design

3.2) Product Page

3.2.1) Research Reference

The product page was designed to highlight individual watch components with bold images, detailed descriptions, and interactive elements. The layout design took inspiration from Nextgen, where refinement is combined with accuracy to offer a high-end product discovery experience.

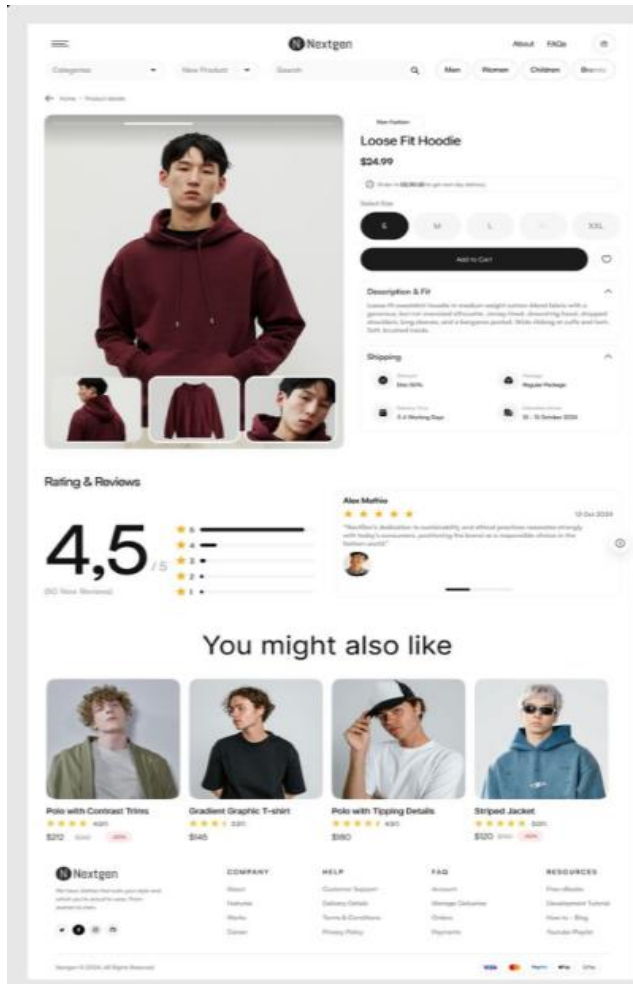


Figure 6 - Reference page for product page (nextgen, 2025)

Key Points that impressed me:

- Detailed Specifications: Clearly outlined features, materials, and movement types made it easier for users to observe the products and make decision while choosing products.
- Large Product Imagery: High-resolution images with zoom-in hover effects
- User Reviews & Ratings: A section with customer reviews provided transparency and promising behaviour
- Stock Availability Indicator: Real-time availability status helped in managing expectations.

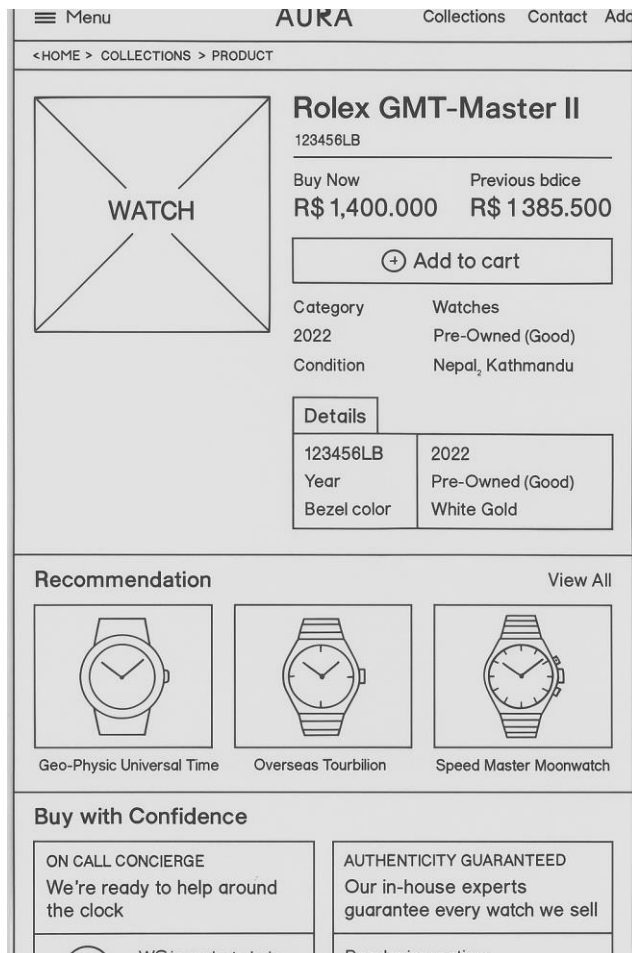
Wireframe:

Figure 7 - Wireframe for product page

3.2.2) Actual Design

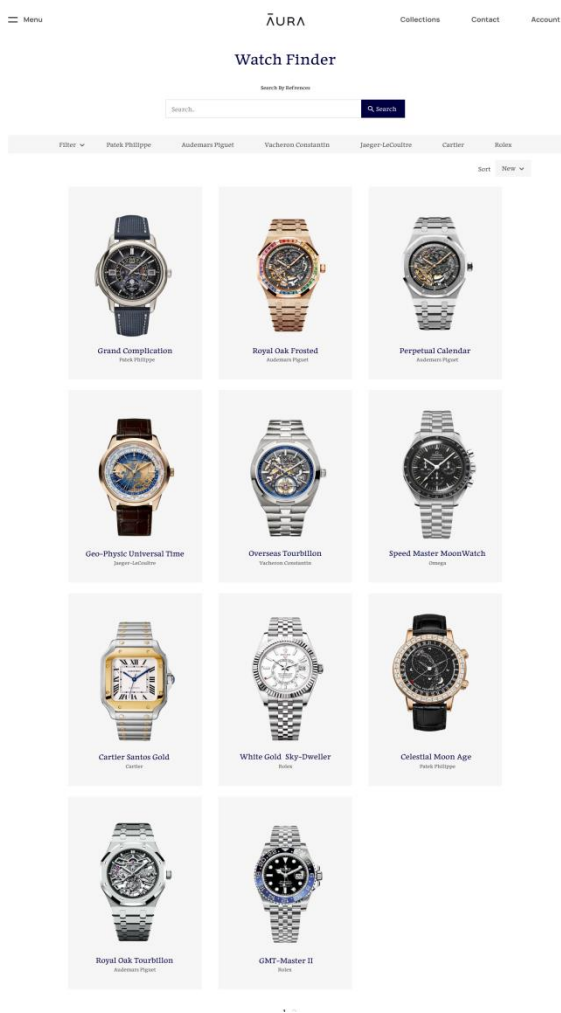


Figure 8 - Product Page UI

3.3) Add to Cart Page

3.3.1) Research Reference

The "Add to Cart" page displays a preview of selected items and operates checkout functionality. The design of this part inspiration came from Apple's and Nike's e-commerce carts, focusing on elegance and attractiveness with strong flow of UX.

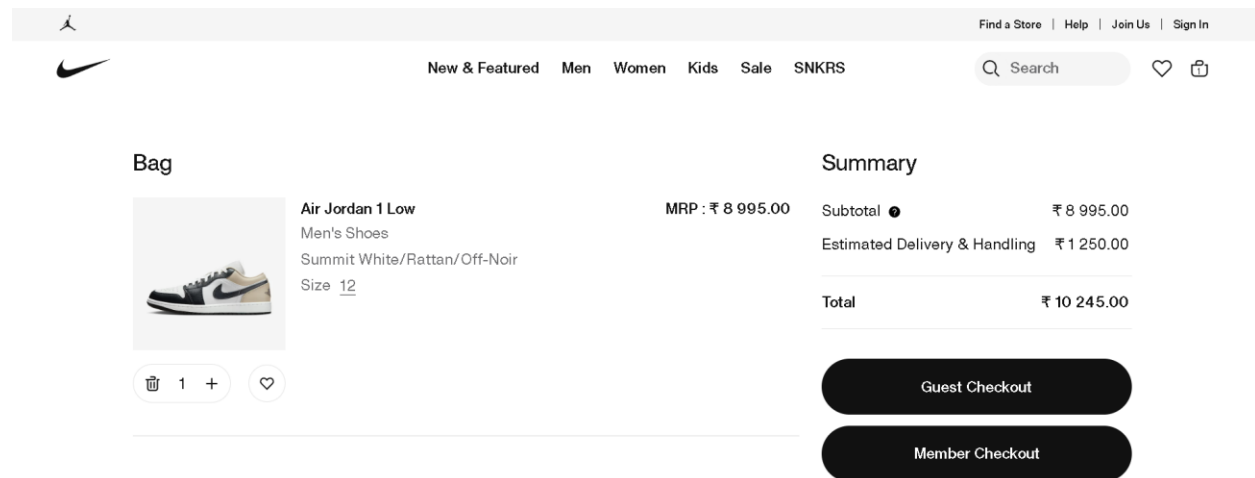


Figure 9 - Reference page for add to cart page (NIKE, 2025)

Key Points that impressed me:

- Cart Item: Helps users can visually recall their choices through thumbnails
- Quantity Adjustment Options: Users can increase or decrease item count according to their necessity
- Remove Item Feature: A simple and manageable way to manage cart items.
- Proceed to Checkout Flow: Clear button guiding the user for completing the purchase process.

Wireframe:

+ Menu		AURA	
Collections		Contact	Add to cart
YOUR CART Product Quantity Price		SUMMARY Shipping (10 to 15 business days) Tax XXXX Subtotal XXXX Total XXXX	
X  Rolex GMT-Master II 1 123455LB		Proceed To Checkout Have a Coupon? Enter your Code (Enter code)	
Free Shipping Free a Coupon? Enter your Co Need Help?			
 FREE DELIVERY WORLDWIDE**	14 14 DAY RETURNS	 FREE DEIVEORLWDW IDE	
AURA Purchase policy Returns & Refund Policy Payment Methods Shipping & Delivery    . .		Additional Information Set your watch Security & Safety Aura Times Aura Times Signup Visit Visit	
+ Menu		Colections	Contact

Figure 10 - Wireframe for add to cart page

3.3.2) Actual Design:

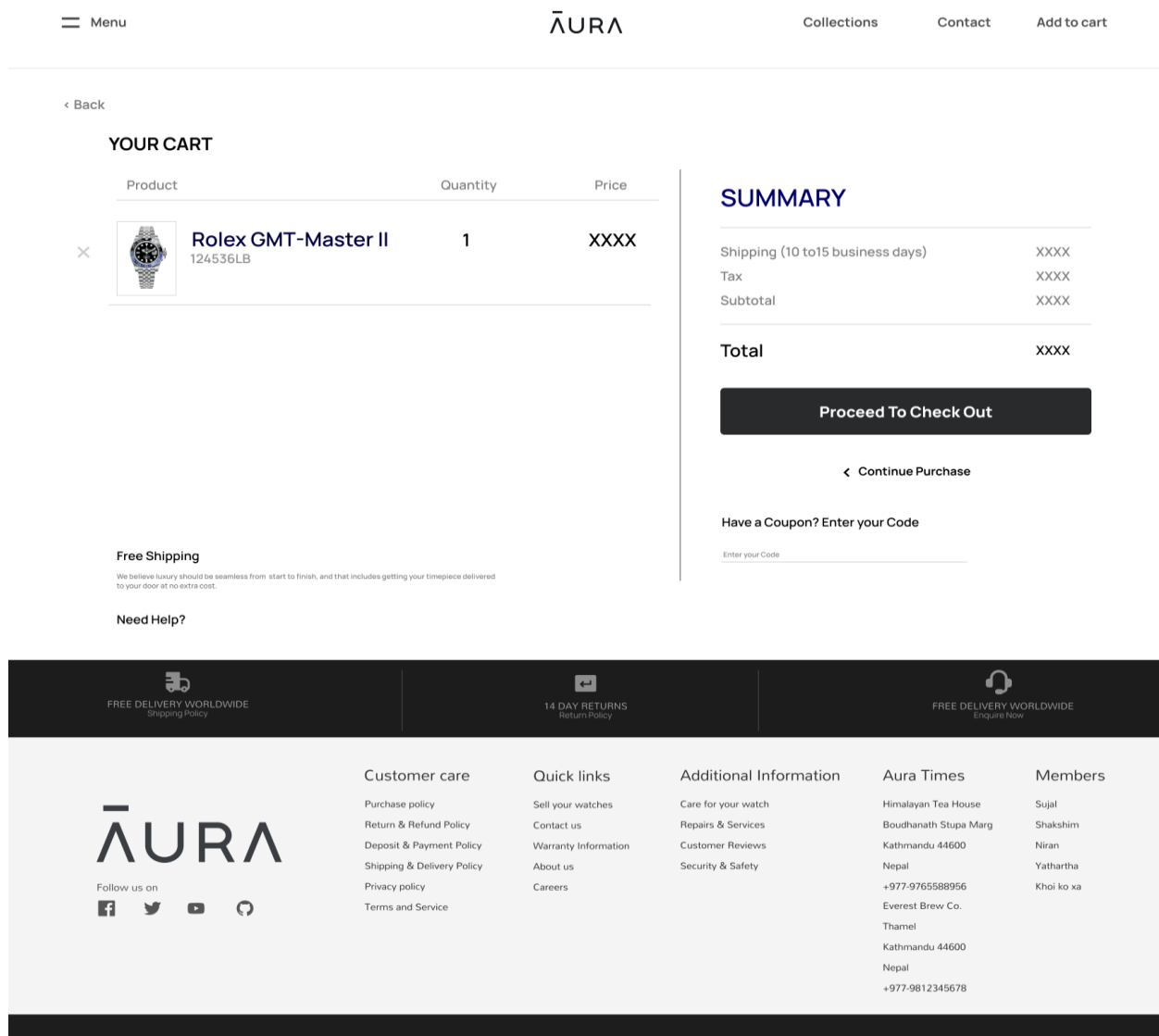


Figure 11 - Add to Cart Page UI

3.4) Side Bar

3.4.1) Research Reference

The side bar was developed to provide an easy way to filter and navigate the product categories. The concept of side bar was taken from Amazon and Flipkart's filter system, providing users a highly responsive filtering experience.

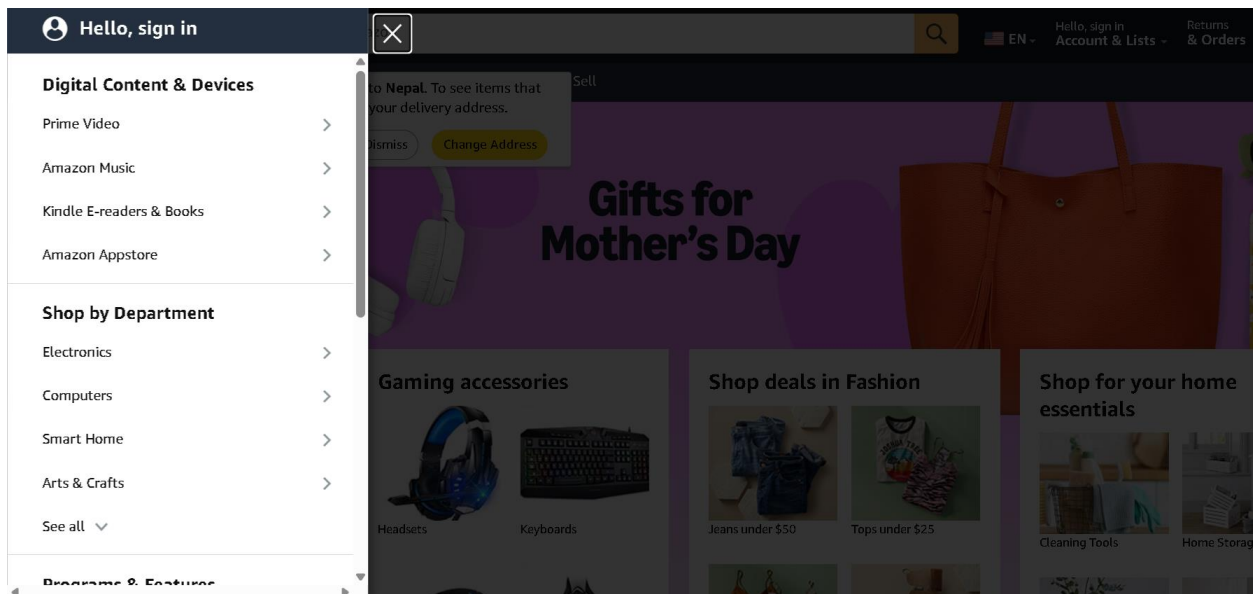


Figure 12 - Reference for side bar (amazon, 2025)

Key Points that impressed me:

- Filter by Product Brand, Price, and Style: Allowed narrowing down choices without reloading the entire page.
- Sticky Sidebar Design: Ensures constant visibility while scrolling, making it easy for users to use.
- Checkbox and Slider Input Types: User-friendly components for applying multiple filters.
- Dynamic Filter Application: Real-time filtering without page refresh and provides enhanced performance.

Wireframe:

Figure 13 - Wireframe for side bar

3.4.2) Actual Design:

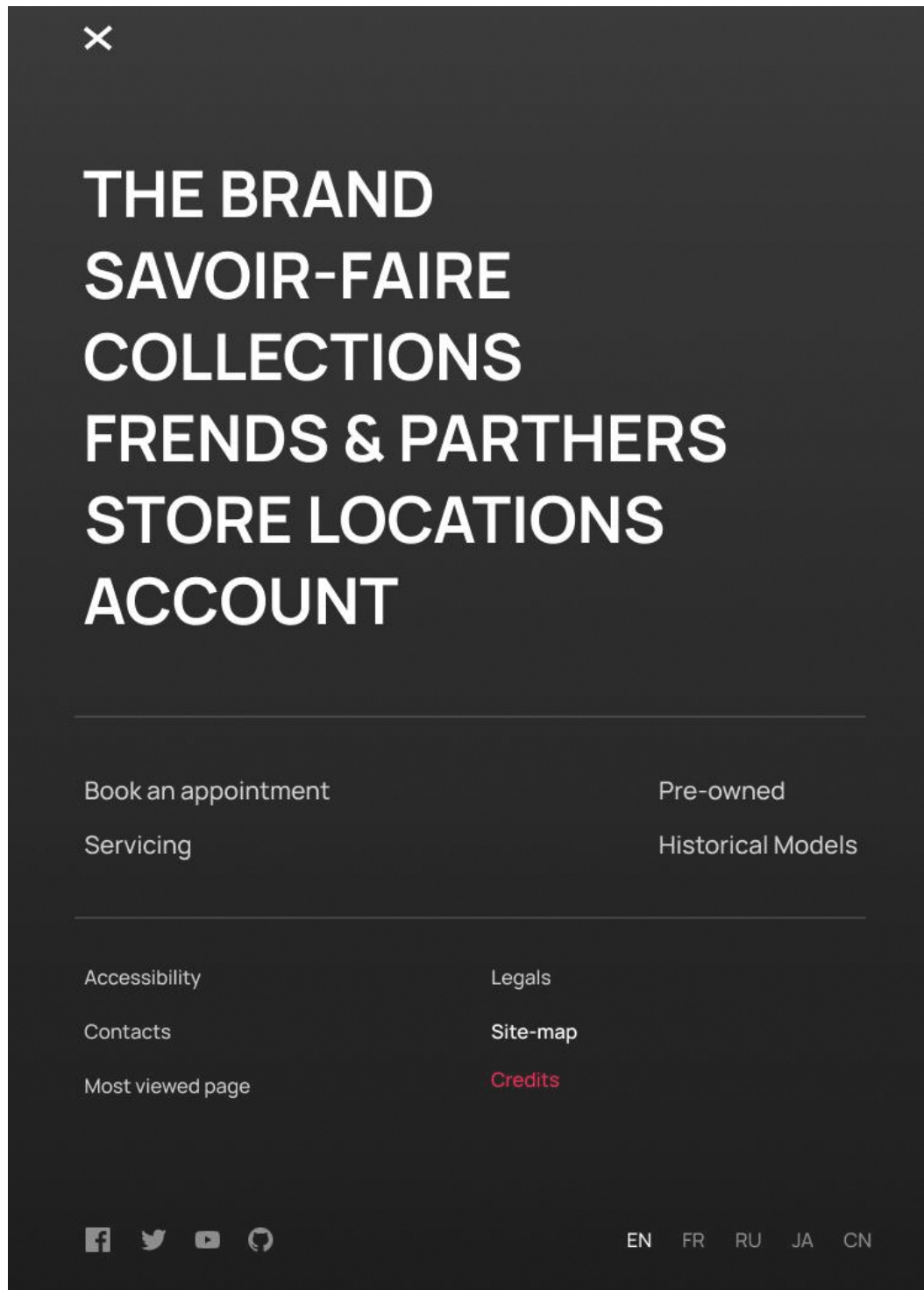


Figure 14 - Side bar UI

3.5) Login Page

3.5.1) Research Reference

Given reference was taken from Pinterest due to their simple and easy appearances which make it nice and tidy. Following is the Login page where user can login using their credentials which are later kept private.

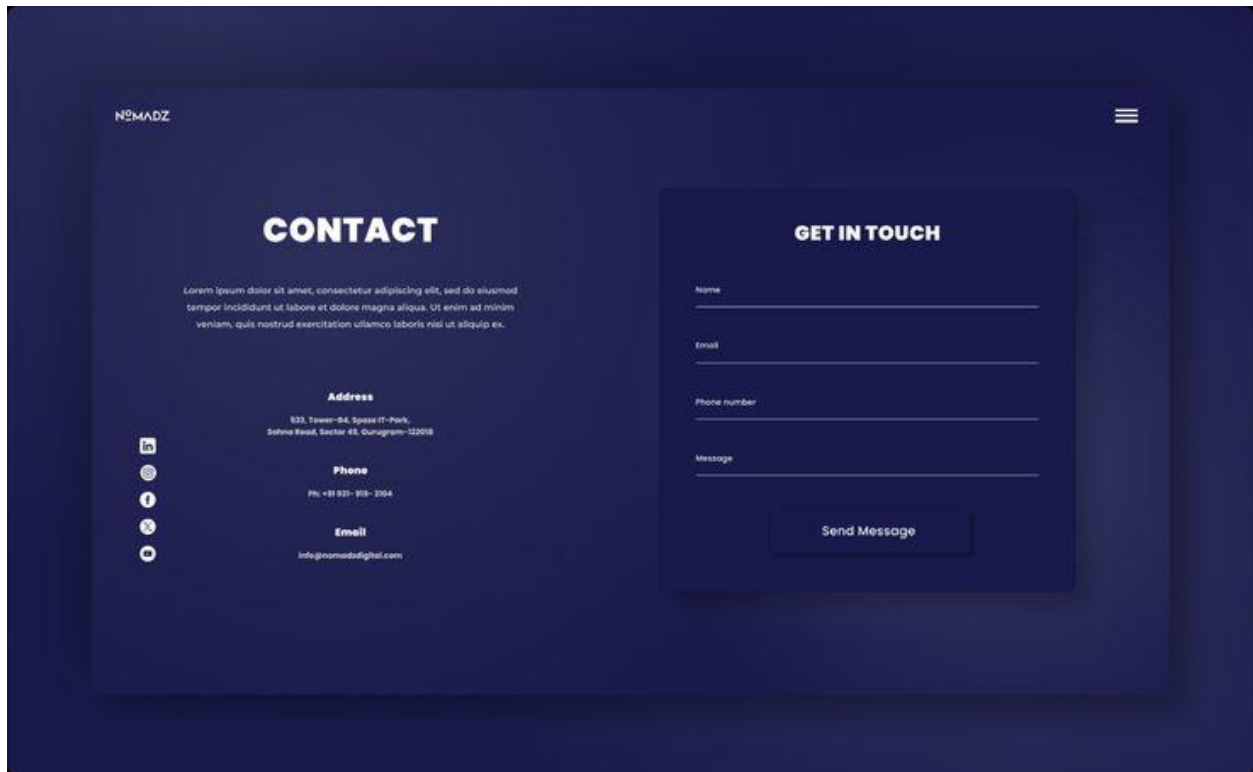


Figure 15 - Reference for login page

Key Points that impress me:

- User friendly forms
- Responsive layout
- Smooth Navigation

3.5.2) Actual Design:

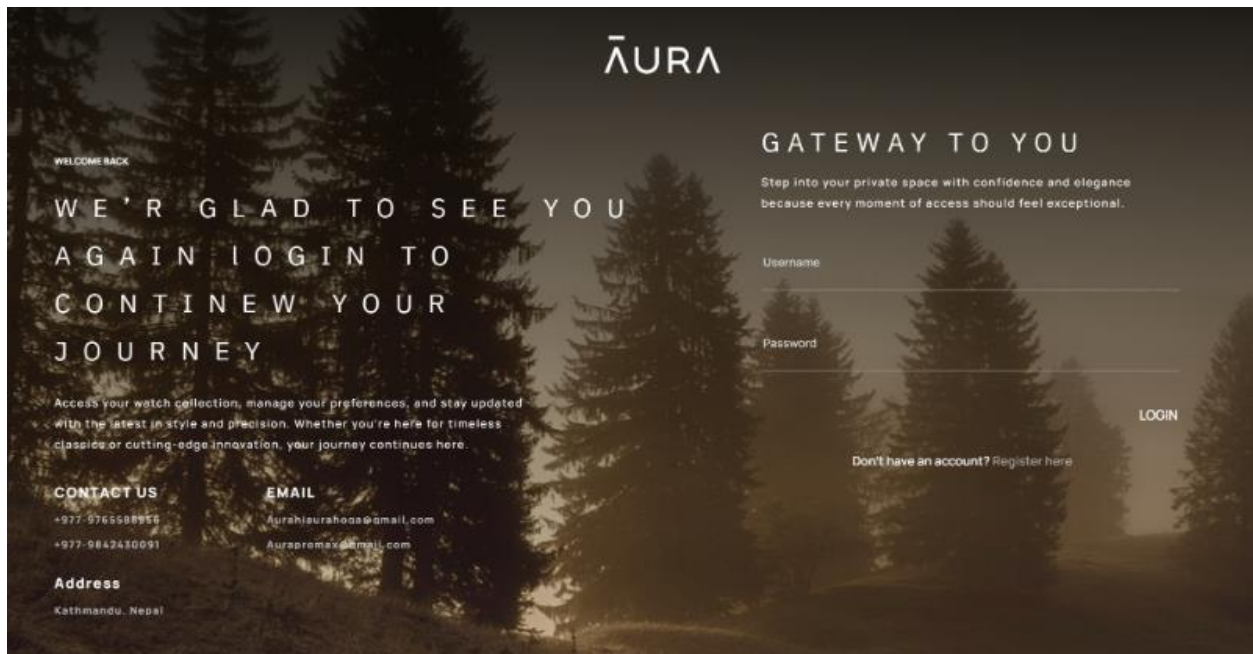


Figure 16 - Actual design for login page

3.6) Registration Page:

We took reference of registration from Web flow just as for login page due to its simple theme with perfect colour combination and design. Here user register using there credentials which later on they are login into the system.

3.6.1) Research Reference

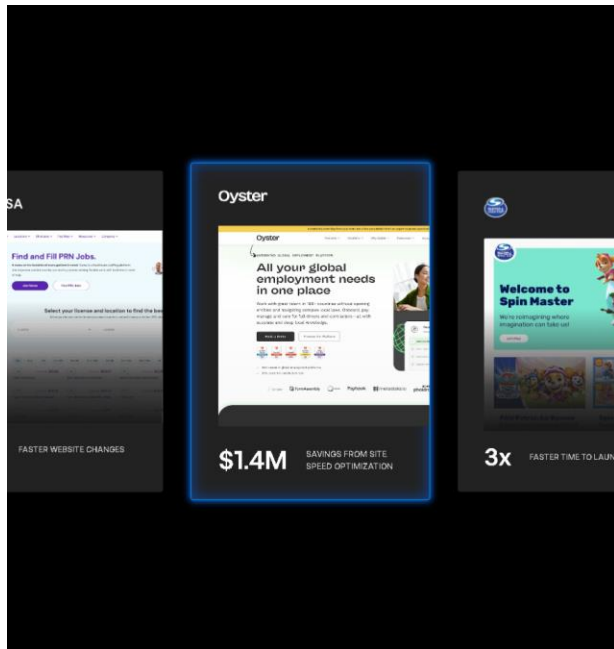
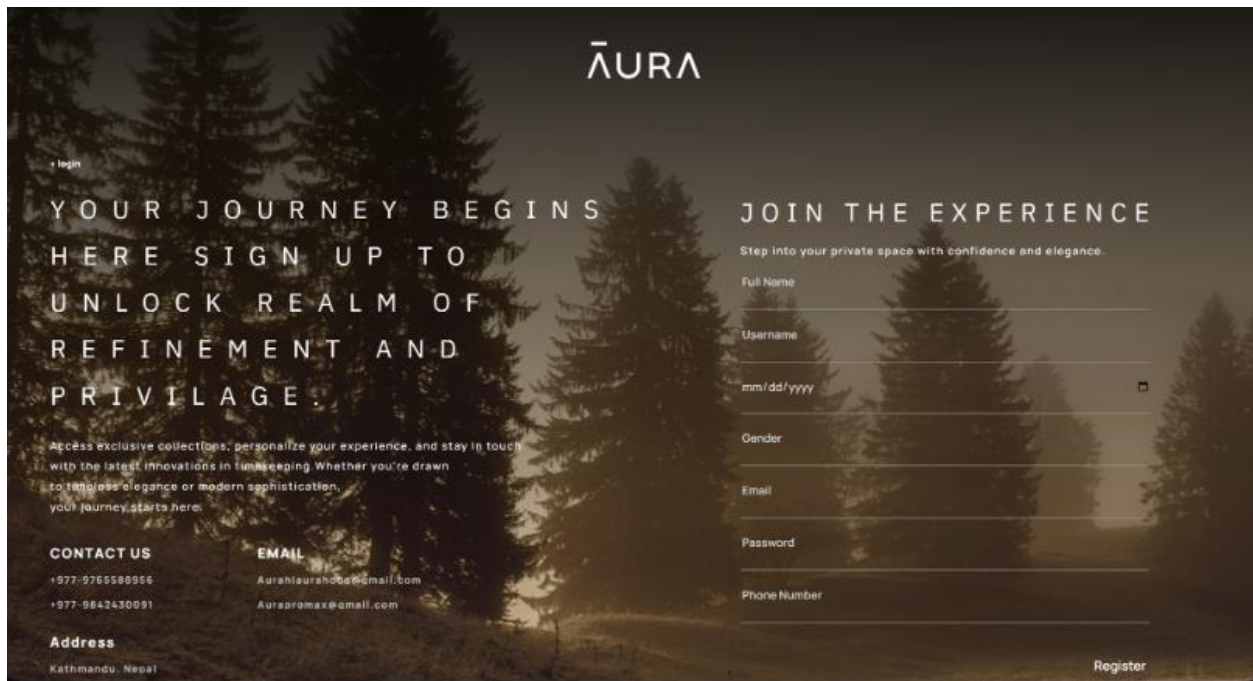


Figure 17 Reference for registration Page (Webflow, 2025)

Key points that impressed me:

- Simple and unique design
- Proper use of colour combination
- Effective use of animations

3.6.2) Actual Design:



The image shows a registration page for a brand named 'AURA'. The background is a dark, atmospheric photograph of a forest with tall evergreen trees. The page layout is clean and modern, with white text and form fields.

AURA

[+ login](#)

**YOUR JOURNEY BEGINS
HERE SIGN UP TO
UNLOCK REALM OF
REFINEMENT AND
PRIVILEGE.**

Access exclusive collections, personalize your experience, and stay in touch with the latest innovations in timekeeping. Whether you're drawn to timeless elegance or modern sophistication, your journey starts here.

CONTACT US
+977-9765588956
+977-9842430091

EMAIL
Aurahiaurahoc@gmail.com
Aurapromax@gmail.com

Address
Kathmandu, Nepal

JOIN THE EXPERIENCE
Step into your private space with confidence and elegance.

Full Name

Username

mm/dd/yyyy 

Gender

Email

Password

Phone Number

[Register](#)

Figure 18 - Actual design for registration page

3.7) Contact page:

3.7.1) Research Reference:

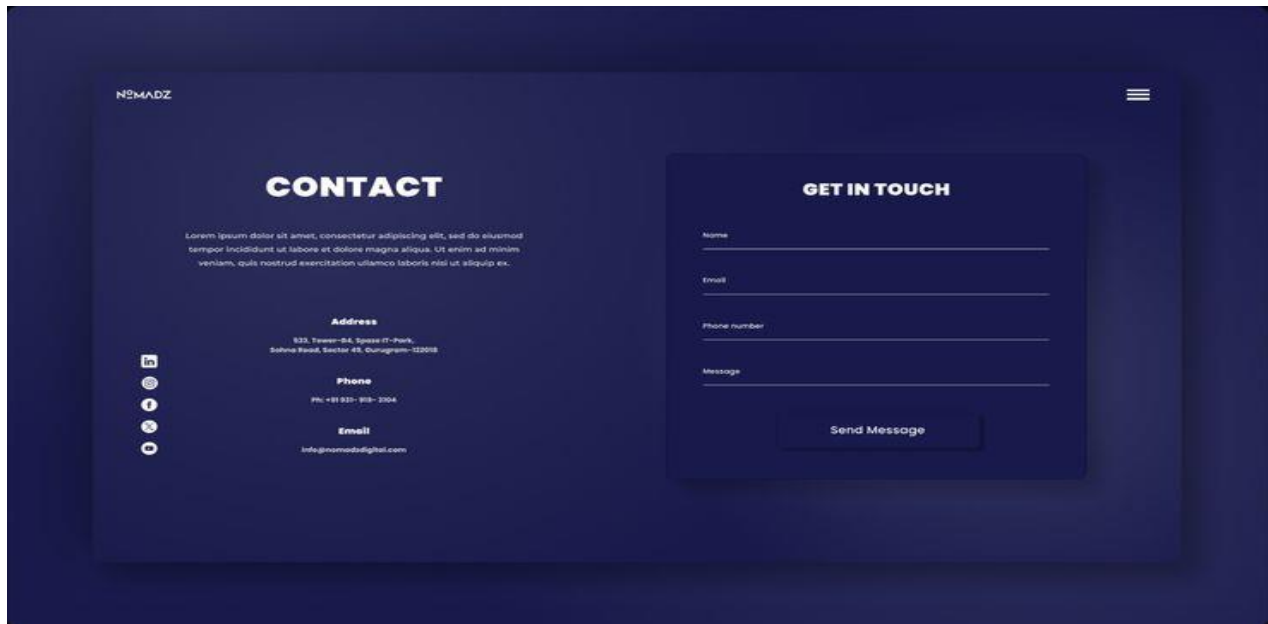
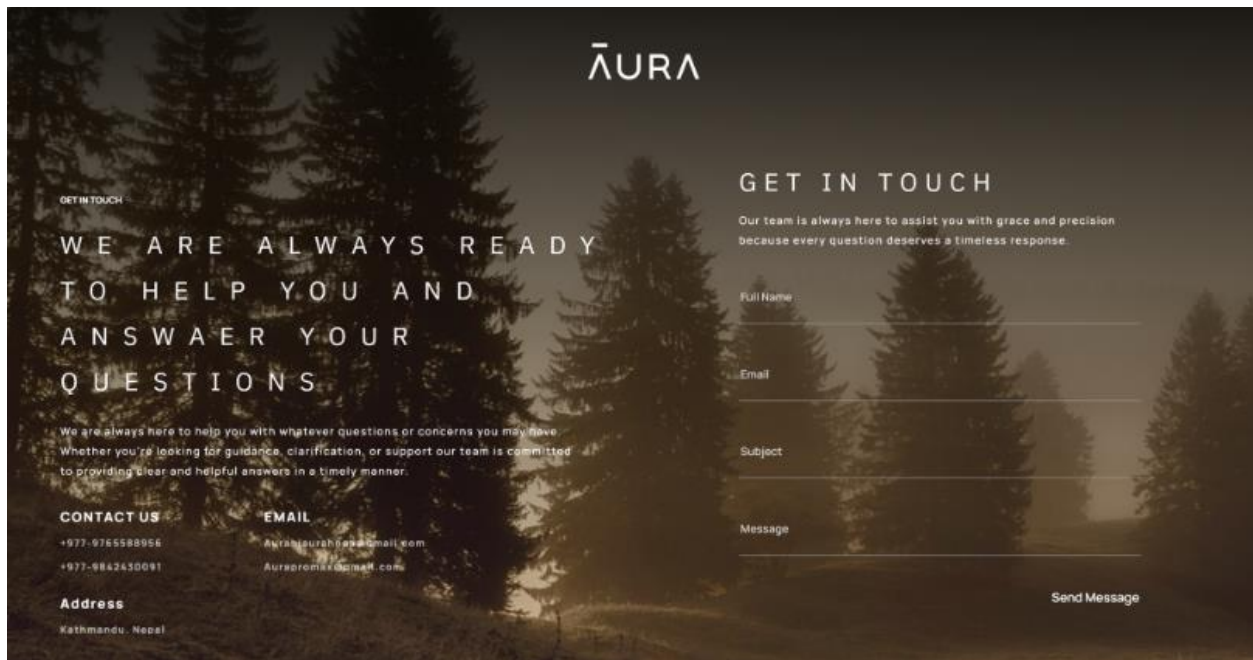


Figure 19 Contact Page Reference

Contact page is also made by taking reference from Pinterest same as Login page due to its simple design and features used.

3.7.2) Actual Design:

The image shows a contact page for 'AURA' with a background of a forest. The page is divided into two main sections: 'GET IN TOUCH' on the right and 'CONTACT US' on the left. The 'GET IN TOUCH' section includes a message about the team's commitment to timely responses and a form with fields for Full Name, Email, Subject, and Message, followed by a 'Send Message' button. The 'CONTACT US' section includes phone numbers, email addresses, and the physical address in Kathmandu, Nepal.

̄AURA

GET IN TOUCH

Our team is always here to assist you with grace and precision because every question deserves a timeless response.

Full Name

Email

Subject

Message

SEND MESSAGE

GET IN TOUCH

WE ARE ALWAYS READY TO HELP YOU AND ANSWER YOUR QUESTIONS

We are always here to help you with whatever questions or concerns you may have. Whether you're looking for guidance, clarification, or support our team is committed to providing clear and helpful answers in a timely manner.

CONTACT US

+977-9765588956
+977-9842630091

EMAIL

Auracontactus@gmail.com
Aurapromax@gmail.com

Address

Kathmandu, Nepal

Figure 20 - Actual design for contact us page

3.8) User profile page:

3.8.1) Research Reference:



Figure 21 - Reference for user profile

Key points that impressed me:

- Minimalistic and functional design
- Engaging user section
- Effective use of animations

3.8.2) Actual Design:

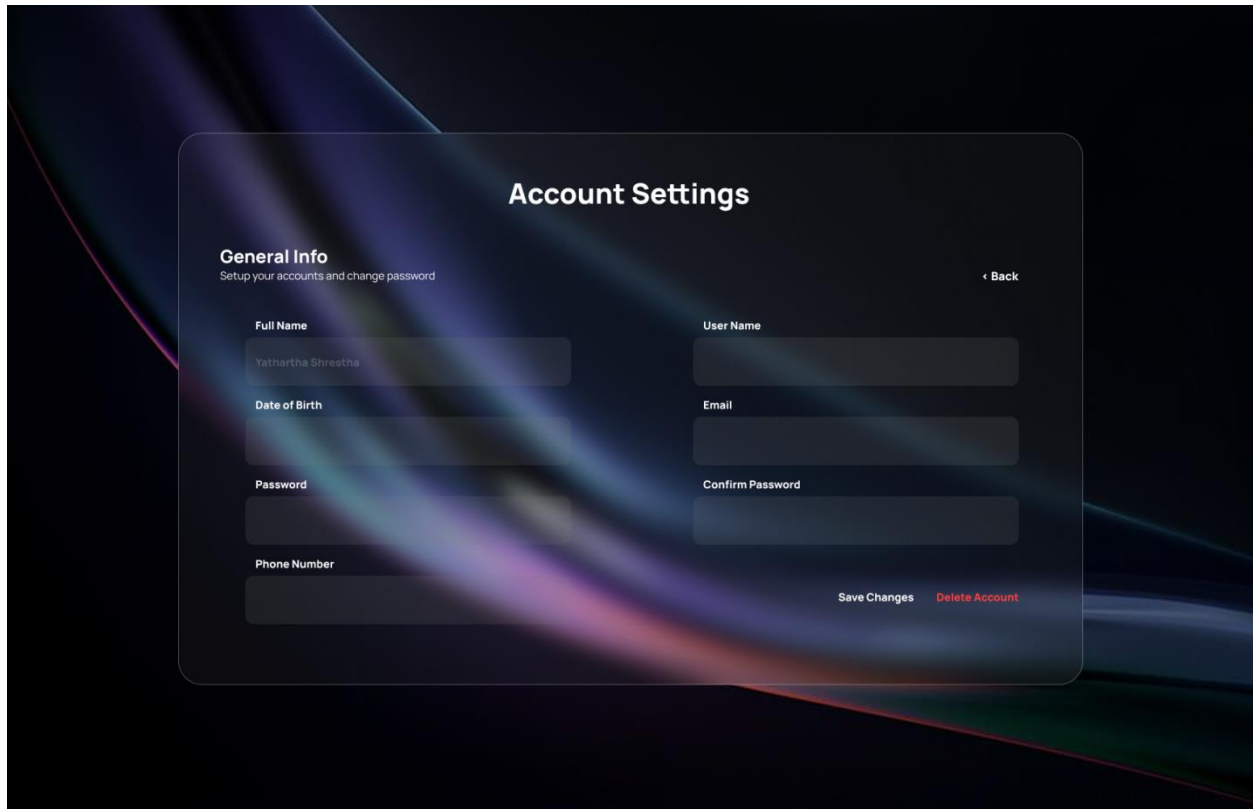
The image shows a dark-themed user interface for 'Account Settings'. The title 'Account Settings' is centered at the top in white. Below it, the section 'General Info' is displayed with the subtitle 'Setup your accounts and change password'. A 'Back' button with a left arrow is in the top right. The form contains several input fields: 'Full Name' (with 'Yathartha Shrestha' as a placeholder), 'User Name', 'Date of Birth', 'Email', 'Password', 'Confirm Password', and 'Phone Number'. At the bottom right, there are two buttons: 'Save Changes' and 'Delete Account' (in red text).

Figure 22 - Actual design for user page

Key points that impressed me:

- Minimalistic and functional design
- Engaging user section
- Effective use of animations

3.9) About us page

3.9.1) Research Reference

Following page was from Rolex due to its simple layout which uses perfect colour combination. Choosing this design helps guide the visual and functional design for about us page.

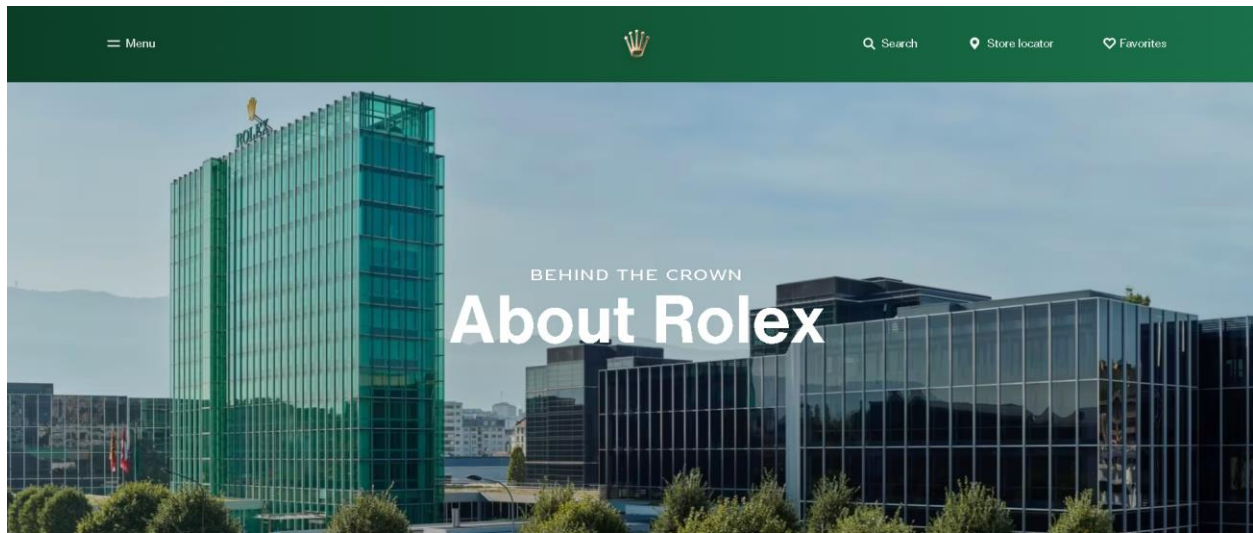


Figure 23 - Research reference for about us page

Key points that impressed me:

- Basic format with perfect colour combination
- Engaging user section
- Effective use of animations

3.9.2) Actual Design

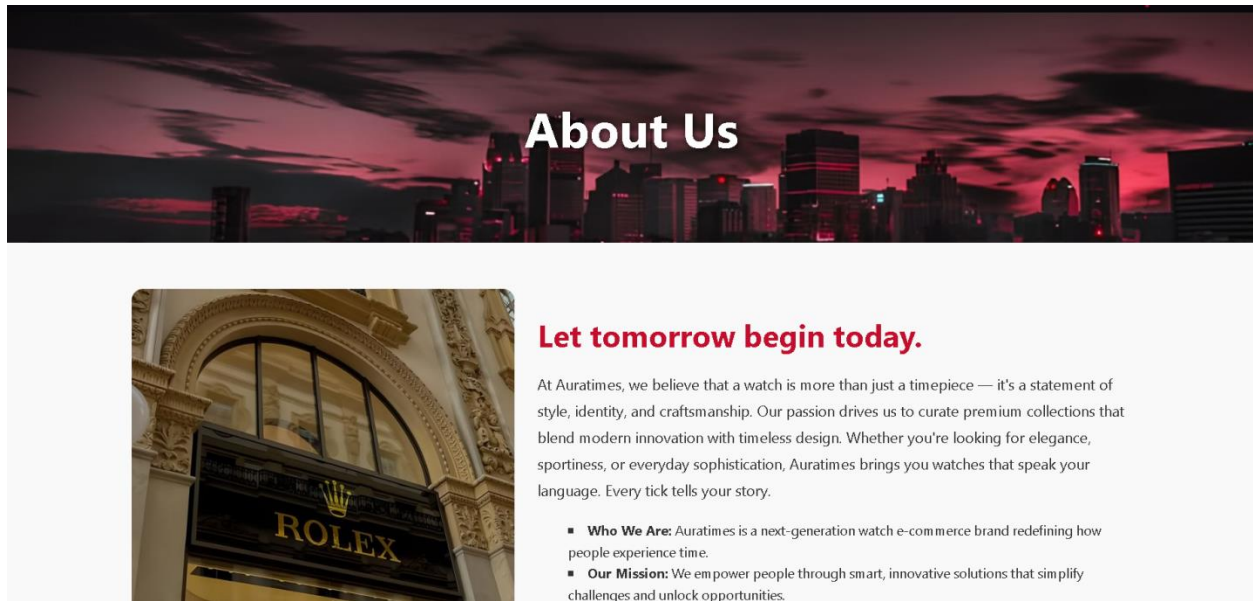


Figure 24 - Actual design for about us page

3.10) Admin Dashboard:

3.10.1) Research Reference

Reference of Admin Dashboard was taken from different pinterest UI of dashboard one of them is shown below:

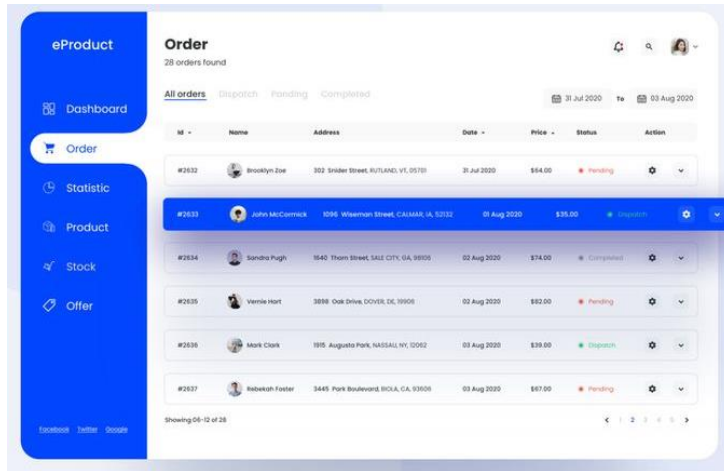


Figure 25 Admin Dashboard Reference

Key points that impressed me:

- Basic format with perfect colour combination
- Engaging admin section
- Effective use of animations inside bar

3.10.2) Actual Design No.1:

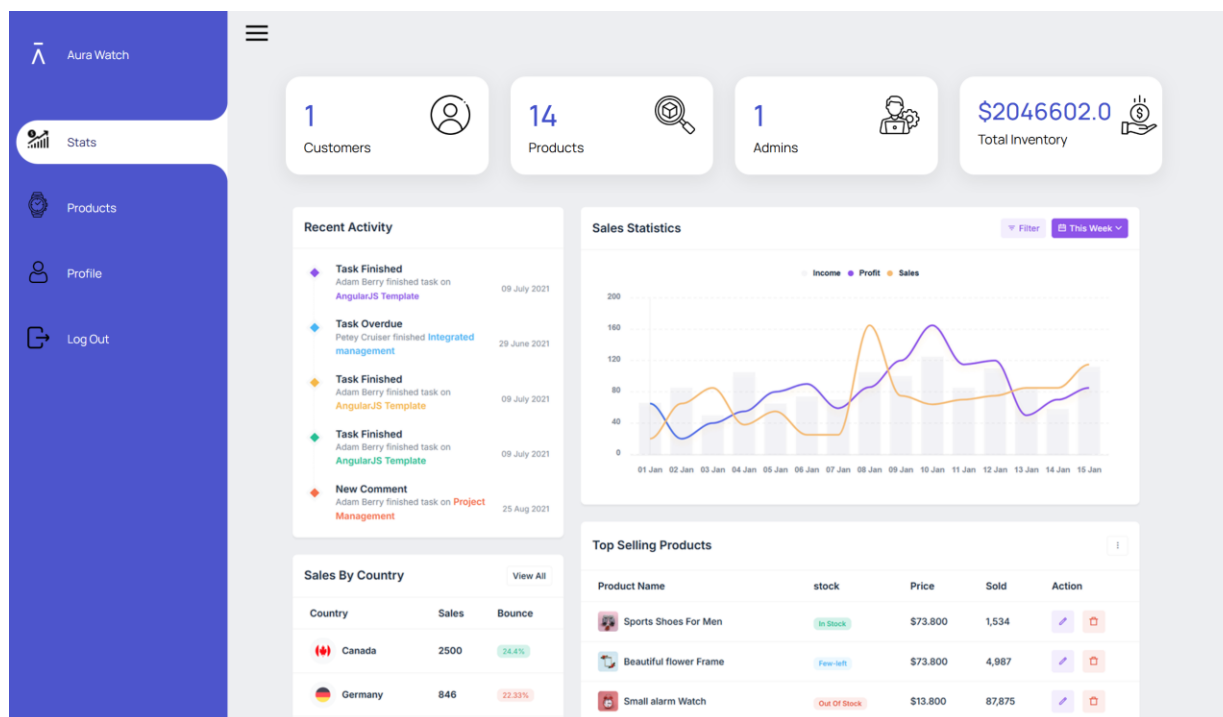


Figure 26 Actual Design No.1

3.10.3) Actual Design No.2:

- Aura Watch
- Stats
- Products
- Profile
- Log Out

Add Products

Product Name
Product Price
Watch Brand
Previous Bid

Description
Product Image No file chosen

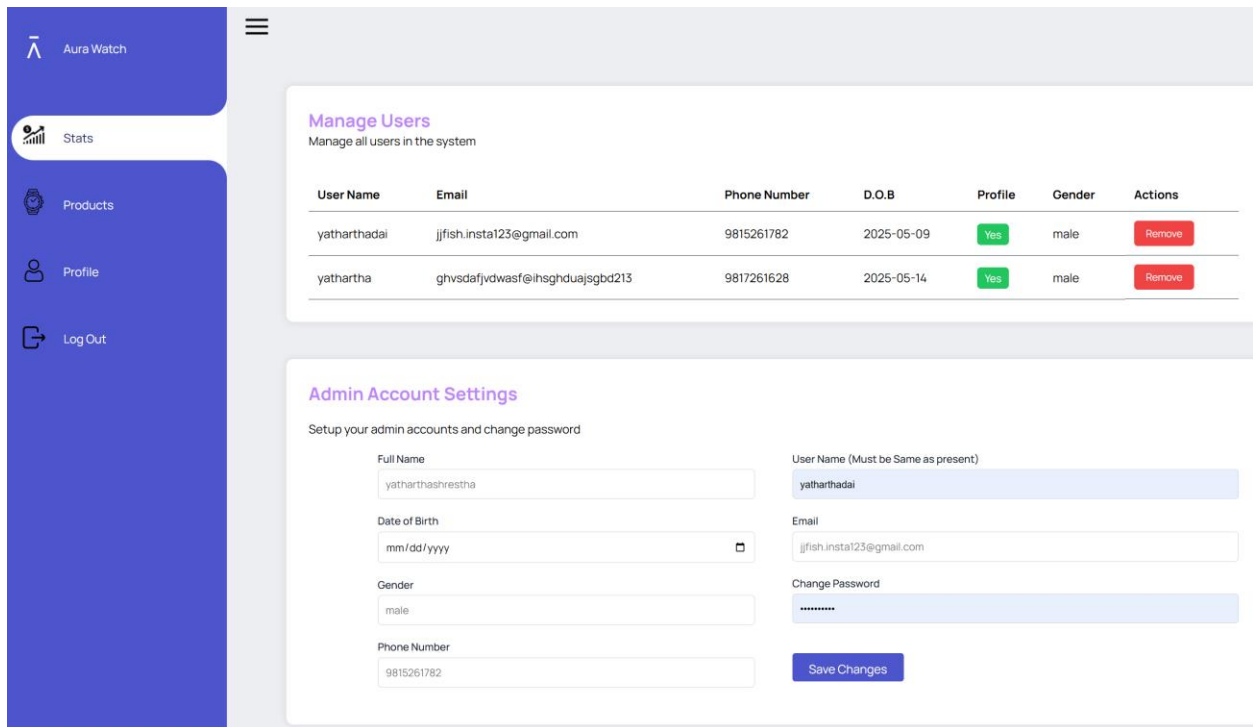
Manage Products

Manage all products in the system

Product Name	Description	Price	Watch Brand	Previous Bid	Actions
Omega Speedmaster	The Omega Speedmaster, often nicknamed the "Moonwatch", is a legendary chronograph watch first introduced by Omega in 1957. Originally designed for racing and motorsports, it gained worldwide fame for its association with NASA's space missions—most notably becoming the first watch worn on the Moon during the Apollo 11 mission in 1969.	\$7700.0	Omega	\$6500	Delete Edit
Rolex Yacht-Master	The Rolex Yacht-Master is a luxury nautical sports watch introduced in 1992. It blends Rolex's signature rugged performance with a more elegant, refined design, making it a favorite among sailors, skippers, and watch collectors who want both function and style.	\$17000.0	Rolex	\$18700	Delete Edit
Rolex Submariner	The Rolex Submariner is a luxury dive watch first introduced in 1953. It quickly became an icon for both professional divers and watch enthusiasts. Renowned for its robust build, underwater performance, and timeless design, the Submariner has become a symbol of prestige, reliability, and adventure.	\$12000.0	Rolex	\$12600	Delete Edit

Figure 27 Actual Design No.2

3.10.4) Actual Design No.3:



The image shows a web application interface for an admin dashboard. On the left is a blue sidebar with navigation links: 'Aura Watch' (with a home icon), 'Stats' (with a bar chart icon), 'Products' (with a watch icon), 'Profile' (with a person icon), and 'Log Out' (with a door icon). The main content area has a light gray header with a hamburger menu icon. Below the header, there are two main sections. The first section is titled 'Manage Users' with the subtitle 'Manage all users in the system'. It contains a table with two rows of user data. The second section is titled 'Admin Account Settings' with the subtitle 'Setup your admin accounts and change password'. It contains several form fields for updating the admin's profile and password, along with a 'Save Changes' button.

User Name	Email	Phone Number	D.O.B	Profile	Gender	Actions
yatharthadai	jjfish.insta123@gmail.com	9815261782	2025-05-09	Yes	male	<button>Remove</button>
yathartha	ghvsdafjvdwasf@ihsgghduajsgbd213	9817261628	2025-05-14	Yes	male	<button>Remove</button>

Admin Account Settings
Setup your admin accounts and change password

Full Name
yatharthashrestha

Date of Birth
mm/dd/yyyy

Gender
male

Phone Number
9815261782

User Name (Must be Same as present)
yatharthadai

Email
jjfish.insta123@gmail.com

Change Password
.....

Save Changes

Figure 28 Actual Design No.3

- We took different reference from internet for the UI of different section inside Admin Dashboard.

4) Java Classes Explanation

4.1) Overall, Class Diagram:

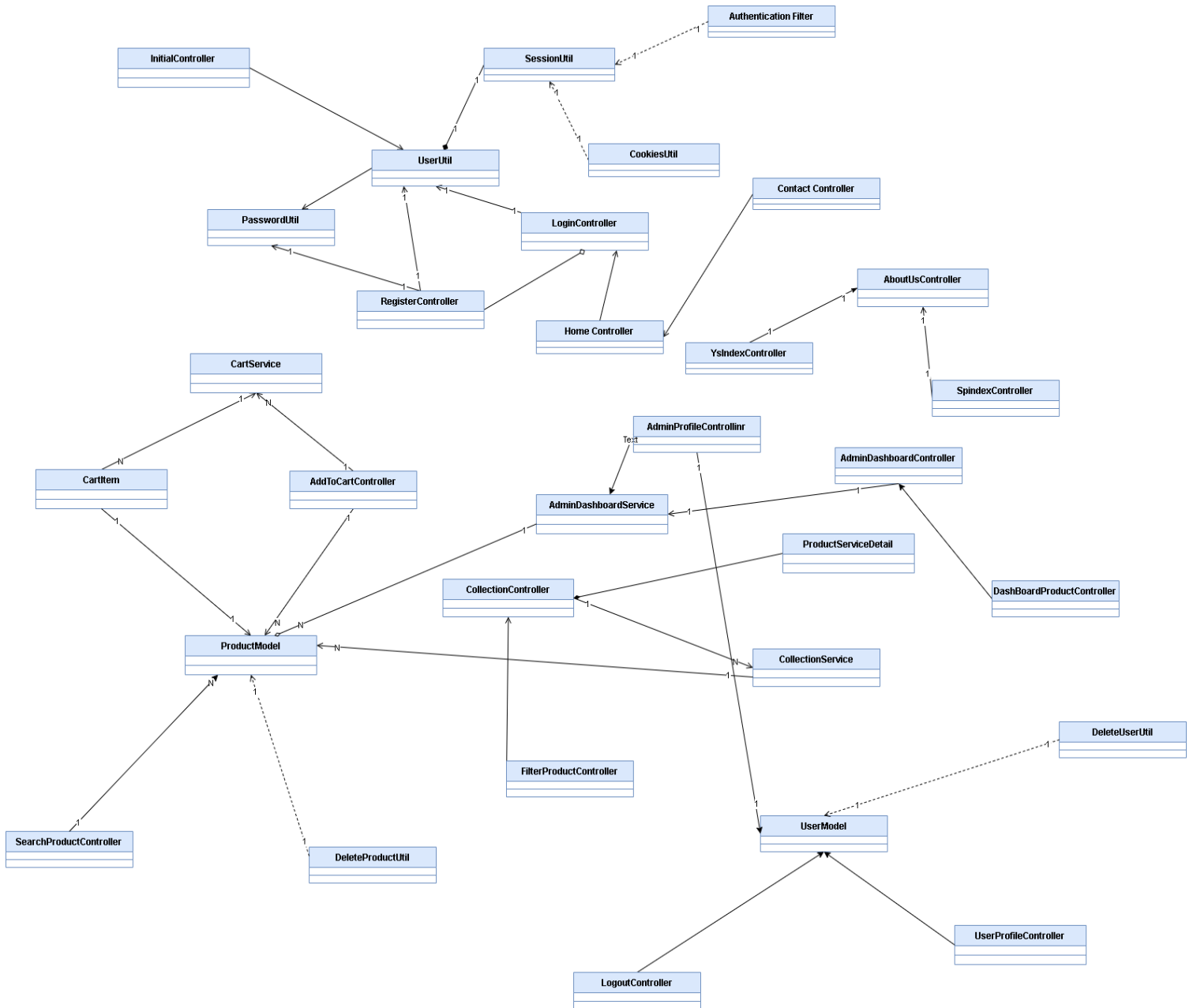


Figure 29 Final Class Diagram

4.2) Individual Class Diagram

4.2.1) DBconfig.java

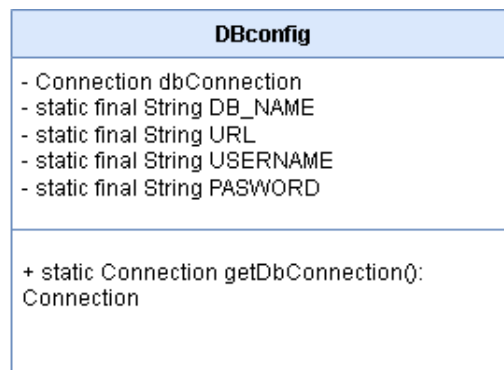


Figure 30 - Class diagram for DBconfig

DBconfig functions as the class that establishes and oversees database-application connection management. A watch resale e-commerce site such as any web application requires data persistence through fetching and saving user data. It has one method:

Method:

getConnection()

The method implements `DriverManager.getConnection ()` from Java to establish database connectivity by utilizing database URLs as well as usernames and passwords. The method returns an established connection of successful creation after completion.

Functionality: The approach enables database connection establishment to return the connection to requesting program sections.

Usage in the project: The application determines data operations through this method which controllers or model classes invoke to establish a database connection before performing operations.

4.2.2) ProductModel.java

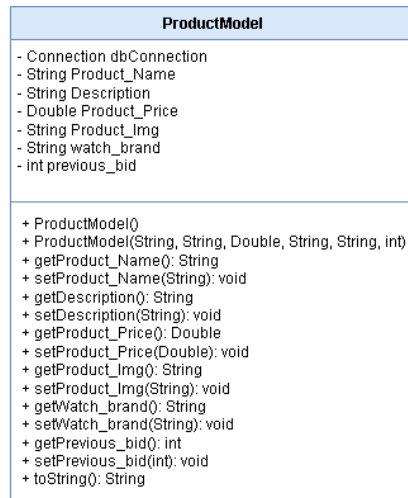


Figure 31 - Class diagram for ProductModel

The product-related data operations of the watch resale e-commerce site are handled by ProductModel which operates as the class. The product database receives requests from ProductModel for watch product retrieval or insertion and management tasks. The class establishes the base for every product listing process along with display operations. It contains multiple methods:

Methods:

getAllProducts()

- The procedure retrieves every record of products which exists in the database.
- **Functionality:** The method pulls back all watches and products that exist on the site website.
- **Usage in the project:** The system executes this procedure to exhibit every product from the list on both homepage pages and product collection pages for customer inspection.

getProductById(int productId)

- The method obtains product information from its distinctive product identification number.
- **Functionality:** The system allows database retrieval of complete watch or product information by utilizing its individual identifier.
- **Usage in the project:** This API helps customers to view detailed product information after selecting a product or to add products to their cart.

addProduct(Product product)

- The database receives a new product through this method for insertion.
- **Functionality:** The application allows users to insert new products by entering product name, price information, brand information and product image data.
- **Usage in the project:** The admin or seller can use this method to upload new watches for sale.

updateProduct(Product product)

- The method enables users to modify details of active products stored within the database.
- **Functionality:** The price along with product description and inventory availability can be updated through this method.
- **Usage in the project:** The seller/administrator uses this method to modify the details of an already uploaded product.

deleteProduct(int productId)

- The database will erase an existing product entry through its specific ID.
- **Functionality:** Removes the product permanently from the system.
- **Usage in the project:** The seller or administrator uses this function to remove product listings which should no longer be available for resale.

4.2.3) CollectionServices.java

Figure 32 - Class diagram for CollectionServices

CollectionServices manages the product display operations together with collection features within the watch resale e-commerce platform. The application uses collection services to determine the layout of products across different parts of its interface. The system provides methods dedicated to filtering view specifications and managing dynamic product set displays. It has the following method:

Method:**getCollection()**

- A method retrieves every product from the collection which was prepared or gathered to present as an exhibition display.
- **Functionality:** The system retrieves product data and watch data as lists from the database according to category-based or popularity-driven or promotional criterion.
- **Usage in the project:** The application uses this method to display product collections which include featured watches and category filters as well as homepage display.

4.2.4) CookieUtil.java:

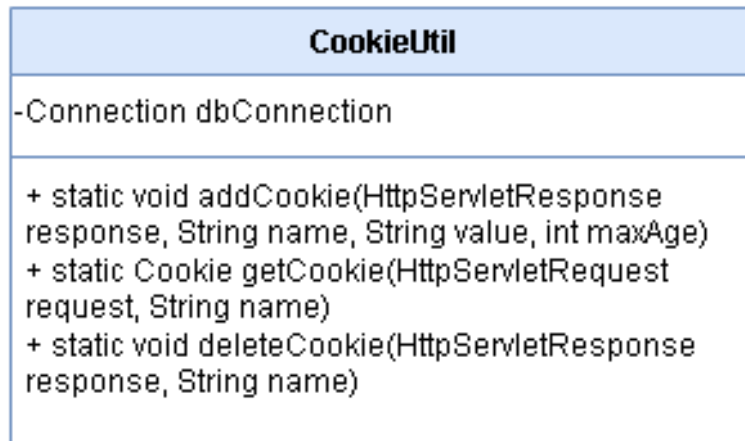


Figure 33 - Class diagram for CookieUtil

The watch resale e-commerce platform depends on CookieUtil to manage the browser cookies that operate within the system. The library offers functionality to generate cookies and provides access to existing cookies and enables cookie removal for storing temporary browser data like login sessions and cart information. It contains the following methods:

Methods:**create(String name, String value, int expiry)**

- The create method accepts parameters named name, value and expiry as strings and an integer. The method generates a cookie containing specified name and value parameters and a time value expressed in seconds.
- **Functionality:** It creates a cookie object containing specified details and determines its expiration time duration.
- **Usage in the project:** The service sets session user information inside the browser while also saving user preference data following login authentication.

find(HttpServletRequest request, String name)

- The find method undertakes a look for a cookie by name which arrives in the incoming HTTP request.
- **Functionality:** The method searches through all request cookies and returns the specified one with the name match.
- **Usage in the project:** This function obtains user login or session cookies from the browser which helps detect whether a user is already logged in or customize their experience.

clear(HttpServletResponse response, String name)

- The deletion process occurs when a cookie gets its maximum lifespan set to 0.
- **Functionality:** This function deletes a browser cookie through the creation of a new cookie with matching name and automatically defined immediate expiration.
- **Usage in the project:** The method functions during user logout and cookie deletion for privacy needs and reset operations.

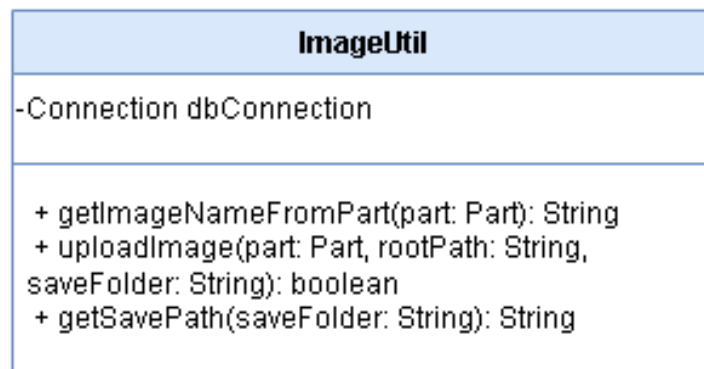
4.2.5) ImageUtil.java

Figure 34 - Class diagram for ImageUtil

The watch resale e-commerce platform depends on ImageUtil to process all image files. The application converts uploaded images to bytes arrays for storage purposes through built-in

functionality. This functionality proves helpful for image storage purposes both within databases and network transmission. It contains the following method:

Methods:

convertToBytes(Part filePart)

- The method transforms a submitted image file into an array of byte data.
- **Functionality:** During a form submission the process reads file contents which results in bytes through data stream processing.
- **Usage in the project:** The method serves to convert an admin or seller uploaded product image. Storing products in the database requires the conversion of images to bytes before their insertion in the record.

4.2.6) PasswordUtil.java

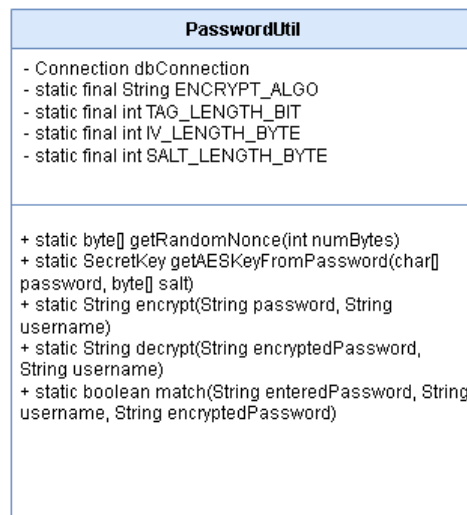


Figure 35 - Class diagram for PasswordUtil

The PasswordUtil utility serves as the security solution which strengthens password protection in the watch resale e-commerce platform. The software component implements encryption procedures on password data before database storage to prevent unauthorized access to user information. It contains the following method:

Method:**encrypt(String password)**

- Such a method transforms an original password into a secure hashed output.
- **Functionality:** The system converts passwords into an unreadable form that maintains uniform length after applying encryption by algorithms like SHA or MD5 to the provided strings. The output of encryption cannot be unencrypted to reveal the original password.
- **Usage in the project:** The cryptographic method gets invoked for user sign-up procedures as well as authentication processes. The registration system encrypts passwords before storing the data within the database. The login system first turns the provided input password into an encrypted form before matching it with the stored data encryption.

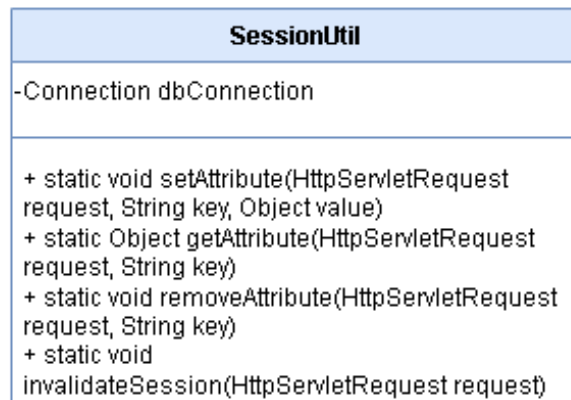
4.2.7) SessionUtil.java

Figure 36 - Class diagram for SessionUtil

The watch resale e-commerce platform depends on SessionUtil to handle all session operations. SessionUtil contains methods that allow storing and retrieving user data through sessions to monitor both user behavior and login status across website pages. It includes the following method:

Method:**getLoggedInUser(HttpSession session)**

- A request to getLoggedInUser(HttpSession session) can retrieve the user object stored in the session.
- **Functionality:** The method examines a session attribute containing the user data from login to return its contents.
- **Usage in the project:** The session provides current user data to retrieve user authentication details on personal pages such as user dashboards and checkout.

4.2.8) ValidationUtil.java

ValidationUtil
- Connection dbConnection
+ static boolean isNullOrEmpty(String input) + static boolean isValidEmail(String email) + static boolean isValidPhoneNumber(String phoneNumber) + static boolean isAlphanumericStartingWithLetter(String username) + static boolean isValidPassword(String password) + static boolean isValidGender(String gender)

Figure 37 - ValidationUtil

ValidationUtil consists of multiple static methods which check user input within the watch resale platform. Input validation enforces data entry conformity for user-provided details including email addresses and phone numbers and passwords which must satisfy both security standards and system requirements. The utility stands as an important tool which protects against incorrect data entry or processing. It includes several methods:

Methods:**isEmpty(String input)**

- The method verifies if the user input consists of null or if it is empty.
- **Functionality:** The function verifies empty and undefined inputs for essential field completion.
- **Usage in the project:** The application deploys the method as a cross-functional tool that verifies necessary user entries in registration, login and form submissions.

isValidEmail(String email)

- The method verifies if the received email address maintains the proper syntax of an email.
- **Functionality:** The pattern ensures the string includes valid structure according to name@anyone.com standards.
- **Usage in the project:** The system uses this feature to authenticate email addresses supplied by users before system acceptance in registration and login processes.

isValidPhoneNumber(String phoneNumber)

- The method performs validation to determine if the phone number consists of numerical values.
- **Functionality:** It checks if there are only digits present together with the correct string length (10 digits are an example).
- **Usage in the project:** The method is employed when users submit contact information both during the account creation process and profile update procedures.

isAlphanumericStartingWithLetter(String username)

- The method verifies that username begins with a letter while also containing only characters that are letters and numbers.
- **Functionality:** The program ensures usernames exclude numerical and special characters at the beginning thus creating a friendly and uniform interface.
- **Usage in the project:** The system uses this check before users finalize their usernames or when updating their profiles.

isValidPassword(String password)

- This function ensures that the password satisfies the set complexity standards.
- **Functionality:** The method checks for strong passwords through two criteria which evaluate password length and incorporation of numbers, symbols, or letters.
- **Usage in the project:** The system implements this function to uphold secure password requirements on registration pages and password alteration screens.

isValidGender(String gender)

- Through this method the system verifies that user-entered gender matches one of the acceptable options ("Male", "Female", "Other") or similar choices.
- **Functionality:** A machine performs a comparison between entered values and authorized gender choices to maintain standardized gender data.
- **Usage in the project:** User selection of a valid gender option is required in both the registration form and profile setup feature.

4.2.9) AboutUsController.java

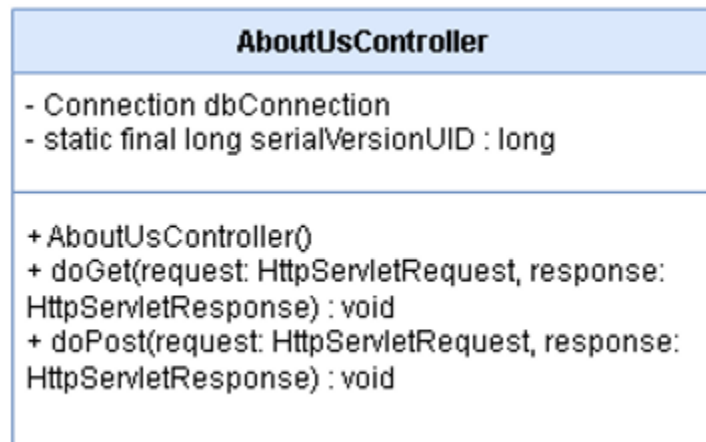


Figure 38 - Class diagram for AboutUsController

AboutUsController manages the functionality that allows website users to access the "About Us" section of the watch resale e-commerce website. Through this class the system displays pages to users and also handles their possible interactive actions (for instance feedback submission or inquiry processing in case such features have been implemented).

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

- **doGet:** The doGet method works with HTTP GET protocol requests. The method forwards HTTP Get requests to display the About Us page through a designated view (JSP page).
- **doPost:** The doPost processes HTTP POST requests that arrive to its server. This method handles form submissions which occur on the About Us page whether they are contact forms or feedback submissions.

Usage in the project:

- **doGet:** When users visit the "About Us" section this event is activated. Without data submission the page displays correctly to the user.
- **doGet:** This method will process user data which exists on About Us page forms such as Contact Us or Send Feedback.

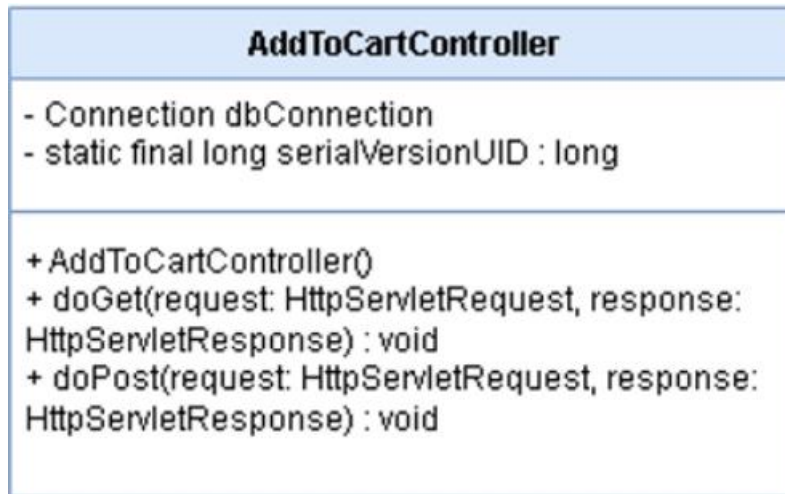
4.2.10) AddToCartController.java

Figure 39 - Class diagram for AddToCartController

The AddToCartController class operates as the solution to manage watch product addition to shopping carts in the watch resale e-commerce platform. The system receives user commands for saving items for future purchase.

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

- **doGet:** User requests using HTTP GET to include chosen watch items within their cart using this method. This system retrieves product information alongside the product ID from incoming requests to create a session-cart entry followed by redirection to the new view.
- **doPost:** This application method processes HTTP POST actions to introduce items into user shopping bags. The method functions like doGet for handling form submissions which differ from typical link clicks.

Usage in the project:

- Pressing the "Add to Cart" link by any logged-in user activates this method. This operation guarantees that the product gets included in the cart session so users can purchase it later.
- The method activates when any form submitted from the cart page. Such functionality enables the platform to perform secure detailed cart addition operation through POST.

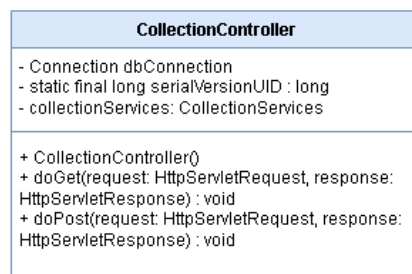
4.2.11) CollectionController.java

Figure 40 - Class diagram for CollectionController

The CollectionController class functions as the manager that presents all available watch products through the collection webpage. The class retrieves watch product information from the database to display it on the product collection section of the watch re-sale e-commerce platform.

It includes the following methods:**Method:**

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

- **doGet:** Http GET requests directed to this method display the entire watch collection available within the platform. The method fetches database watch products which it presents to the view layer for display.
- **doPost:** The method supports receiving and processing POST requests through HTTP. Users can apply search or filter criteria by using this method during their collection browsing.

Usage in the project:

- **doGet:** Users see all watches appear on the page following their click to either Collection or Shop links.
- **doPost:** This method becomes active when users provide their search filter selections or criteria such as price ranges and product brands or conditions. The application updates the displayed collection according to the selection made by the user.

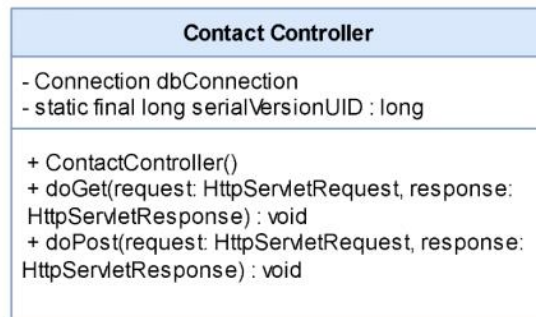
4.2.12) ContactController.java

Figure 41 - Class diagram for ContactController

The ContactController class operates as the central management system for the website's "Contact Us" feature in its watch resale e-commerce portal. Through this controller the website presents contact features and serves as a processor for user transmissions submitted through the contact form.

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

- **doGet:** An HTTP GET request processing method displays the contact interface to users. Users can access this page while filling a contact form to deliver their feedback.
- **doPost:** The method handles data from HTTP POST requests for processing purposes. The data submitted through the contact form is handled by this method which stores or sends the data through email.

Usage in the project:

- **doGet:** The Contact Us method displays the contact form after user clicks without performing any data operations.
- **doPost:** The method procures message details at user submission of the contact form before proceeding with administrative assessment or follow-up dialogue.

4.2.13) HomeController.java

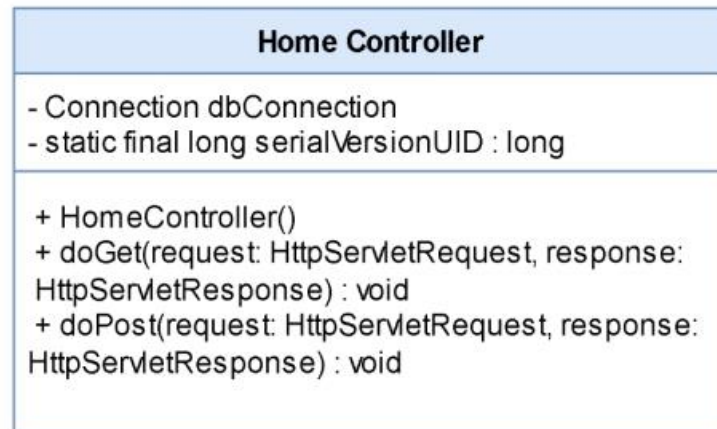


Figure 42 - Class diagram for Home Controller

The watch resale e-commerce website home page gets its management from the HomeController class. The main content for the page that users see upon visiting becomes available when this controller processes requests.

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

- **doGet:** The doGet method accepts HTTP GET requests in order to showcase the home page. After loading featured products together with latest additions and promotional banners it will redirect the user to the homepage view.
- **doPost:** The method supports HTTP POST requests that lead to the home page display. The method primarily handles post requests for special page functions including newsletter subscription and homepage search form submission since it does not process standard page views.

Usage in the project:

- **doGet:** This method shows the homepage content to users who visit the main URL or access the "Home" button to introduce the platform and display important products or offerings.
- **doPost:** This method receives homepage data when users submit forms that include quick searches and email subscriptions through the backend processing.

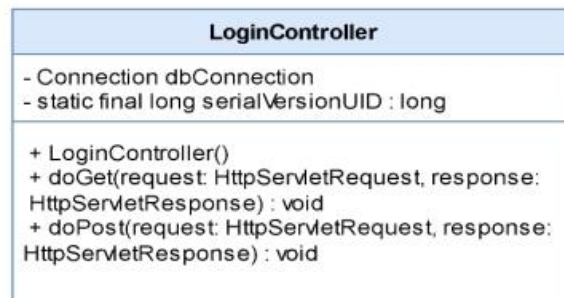
4.2.14) LoginController.java

Figure 43 - Class diagram for LoginController

The watch resale e-commerce website home page gets its management from the HomeController class. The main content for the page that users see upon visiting becomes available when this controller processes requests.

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

- **doGet:** The doGet method accepts HTTP GET requests in order to showcase the home page. After loading featured products together with latest additions and promotional banners it will redirect the user to the homepage view.
- **doPost:** The method supports HTTP POST requests that lead to the home page display. The method primarily handles post requests for special page functions including newsletter subscription and homepage search form submission since it does not process standard page views.

Usage in the project:

- **doGet:** This method shows the homepage content to users who visit the main URL or access the "Home" button to introduce the platform and display important products or offerings.
- **doPost:** This method receives homepage data when users submit forms that include quick searches and email subscriptions through the backend processing.

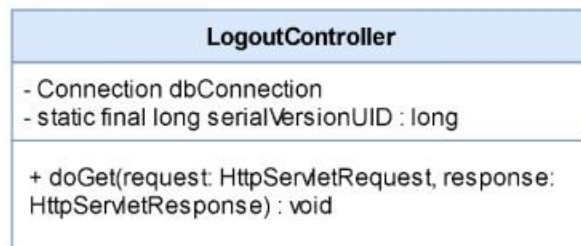
4.2.15) LogoutController.java

Figure 44 - Class diagram for LogOutController

The logout process for users accessing the watch resale e-commerce platform rests with the LogoutController class. A safe logout session termination occurs when users select to end their session because of this controller function.

It includes the following method:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

Functionality:

- The **doGet** method has the responsibility to handle HTTP GET requests which initiate the user logout process. The controller ends user session data then sends them to either the login page or the home page.

Usage in the project:

- Users can eliminate their session when they click the “Logout” button or link and this action simultaneously ensures secure logout access to protected pages.

4.2.16) RegisterController.java

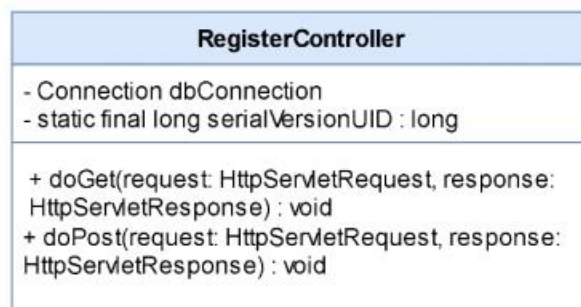


Figure 45 - Class diagram for RegisterController

New users on the watch resale e-commerce platform can register through the functionality of the RegisterController class. This controller controls both the presentation of the registration form and the storage process of new user account data inside the system.

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

- **doGet:** The method deals with getting HTTP GET requests to present the registration interface to users. The system directs the request to show the registration form page for users to provide their personal details.
- **doPost:** The method reacts to HTTP POST requests that users send from their registration form submission. During this process the user data gets validated before the program stores account information into the database.

Usage in the project:

- **doGet:** This method enables users to view registration or sign-up forms by executing a click action.
- **doPost:** Users present their registration data through this operation. The method establishes a new account when all entry criteria are valid and potentially leads users to visit the login or welcome sections.

4.2.17) AboutUsController.java

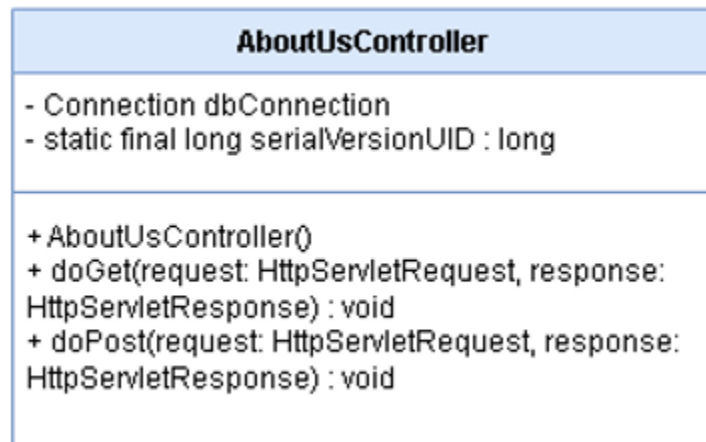


Figure 46 - Class diagram for AboutUsController

AboutUsController manages the functionality that allows website users to access the "About Us" section of the watch resale e-commerce website. Through this class the system displays pages to users and also handles their possible interactive actions (for instance feedback submission or inquiry processing in case such features have been implemented).

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

- **doGet:** The doGet method works with HTTP GET protocol requests. The method forwards HTTP Get requests to display the About Us page through a designated view (JSP page).
- **doPost:** The doPost processes HTTP POST requests that arrive to its server. This method handles form submissions which occur on the About Us page whether they are contact forms or feedback submissions.

Usage in the project:

- doGet: When users visit the "About Us" section this event is activated. Without data submission the page displays correctly to the user.
- doGet: This method will process user data which exists on About Us page forms such as Contact Us or Send Feedback.

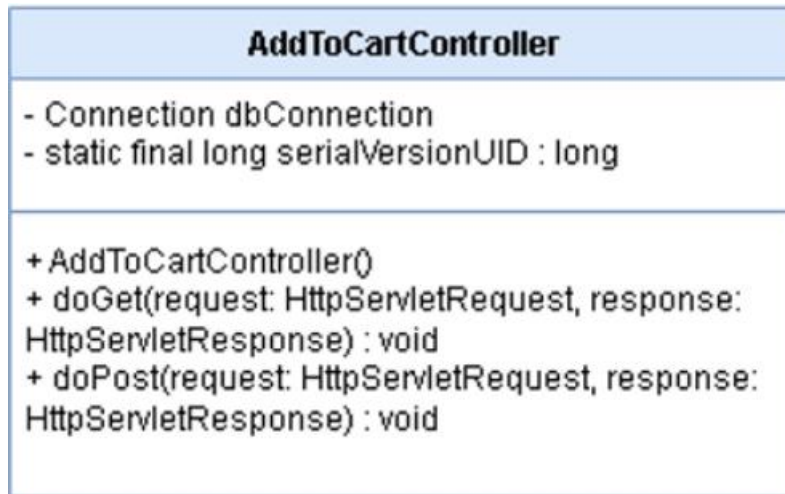
4.2.18) AddToCartController.java

Figure 47 - Class diagram for AddToCartController

The AddToCartController class operates as the solution to manage watch product addition to shopping carts in the watch resale e-commerce platform. The system receives user commands for saving items for future purchase.

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

doGet: User requests using HTTP GET to include chosen watch items within their cart using this method. This system retrieves product information alongside the product ID from incoming requests to create a session-cart entry followed by redirection to the new view.

doPost: This application method processes HTTP POST actions to introduce items into user shopping bags. The method functions like doGet for handling form submissions which differ from typical link clicks.

Usage in the project:

Pressing the "Add to Cart" link by any logged-in user activates this method. This operation guarantees that the product gets included in the cart session so users can purchase it later.

The method activates when any form submitted from the cart page. Such functionality enables the platform to perform secure detailed cart addition operation through POST.

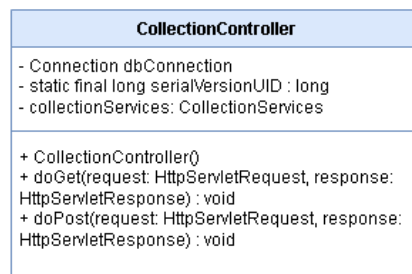
4.2.19) CollectionController.java

Figure 48 - Class diagram for CollectionController

The CollectionController class functions as the manager that presents all available watch products through the collection webpage. The class retrieves watch product information from the database to display it on the product collection section of the watch re-sale e-commerce platform

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

doGet: Http GET requests directed to this method display the entire watch collection available within the platform. The method fetches database watch products which it presents to the view layer for display.

doPost: The method supports receiving and processing POST requests through HTTP. Users can apply search or filter criteria by using this method during their collection browsing.

Usage in the project:

doGet: Users see all watches appear on the page following their click to either Collection or Shop links.

doPost: This method becomes active when users provide their search filter selections or criteria such as price ranges and product brands or conditions. The application updates the displayed collection according to the selection made by the user.

4.2.20) ContactController.java

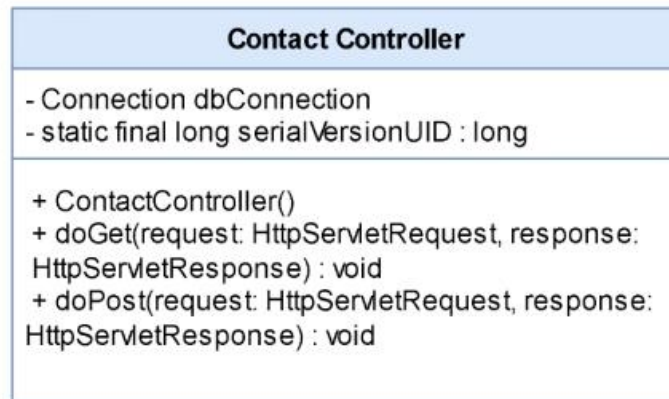


Figure 49 - Class diagram for ContactController

The ContactController class operates as the central management system for the website's "Contact Us" feature in its watch resale e-commerce portal. Through this controller the website presents contact features and serves as a processor for user transmissions submitted through the contact form.

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

doGet: An HTTP GET request processing method displays the contact interface to users. Users can access this page while filling a contact form to deliver their feedback.

doPost: The method handles data from HTTP POST requests for processing purposes. The data submitted through the contact form is handled by this method which stores or sends the data through email.

Usage in the project:

doGet: The Contact Us method displays the contact form after user clicks without performing any data operations.

doPost: The method procures message details at user submission of the contact form before proceeding with administrative assessment or follow-up dialogue.

4.2.21) HomeController.java

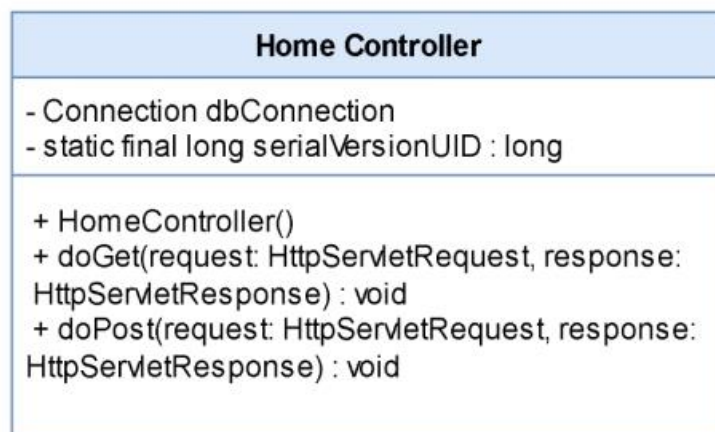


Figure 50 - Class diagram for Home Controller

The watch resale e-commerce website home page gets its management from the HomeController class. The main content for the page that users see upon visiting becomes available when this controller processes requests.

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

doGet: The doGet method accepts HTTP GET requests in order to showcase the home page. After loading featured products together with latest additions and promotional banners it will redirect the user to the homepage view.

doPost: The method supports HTTP POST requests that lead to the home page display. The method primarily handles post requests for special page functions including newsletter subscription and homepage search form submission since it does not process standard page views.

Usage in the project:

doGet: This method shows the homepage content to users who visit the main URL or access the "Home" button to introduce the platform and display important products or offerings.

doPost: This method receives homepage data when users submit forms that include quick searches and email subscriptions through the backend processing.

4.2.22) LoginController.java

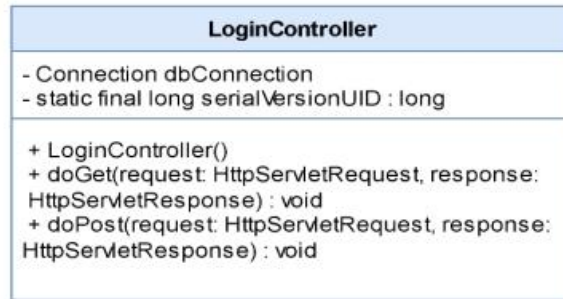


Figure 51 - Class diagram for LoginController

The watch resale e-commerce website home page gets its management from the HomeController class. The main content for the page that users see upon visiting becomes available when this controller processes requests.

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

doGet: The doGet method accepts HTTP GET requests in order to showcase the home page. After loading featured products together with latest additions and promotional banners it will redirect the user to the homepage view.

doPost: The method supports HTTP POST requests that lead to the home page display. The method primarily handles post requests for special page functions including newsletter subscription and homepage search form submission since it does not process standard page views.

Usage in the project:

doGet: This method shows the homepage content to users who visit the main URL or access the "Home" button to introduce the platform and display important products or offerings.

doPost: This method receives homepage data when users submit forms that include quick searches and email subscriptions through the backend processing.

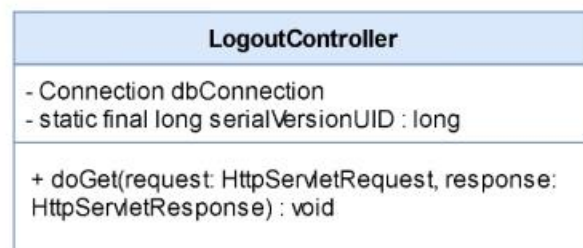
4.2.23) LogoutController.java

Figure 52 - Class diagram for LogOutController

The logout process for users accessing the watch resale e-commerce platform rests with the LogoutController class. A safe logout session termination occurs when users select to end their session because of this controller function.

It includes the following method:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

Functionality:

The doGet method has the responsibility to handle HTTP GET requests which initiate the user logout process. The controller ends user session data then sends them to either the login page or the home page.

Usage in the project:

Users can eliminate their session when they click the “Logout” button or link and this action simultaneously ensures secure logout access to protected pages.

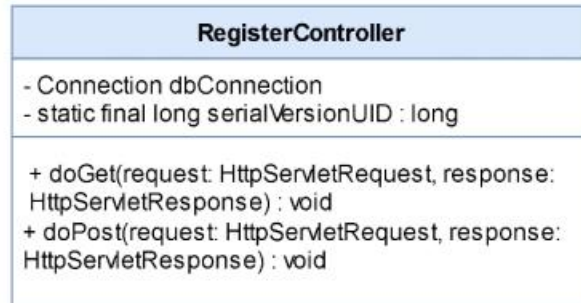
4.2.24) RegisterController.java

Figure 53 - Class diagram for RegisterController

New users on the watch resale e-commerce platform can register through the functionality of the RegisterController class. This controller controls both the presentation of the registration form and the storage process of new user account data inside the system.

It includes the following methods:

Method:

doGet(HttpServletRequest request, HttpServletResponse response)

doPost(HttpServletRequest request, HttpServletResponse response)

Functionality:

doGet: The method deals with getting HTTP GET requests to present the registration interface to users. The system directs the request to show the registration form page for users to provide their personal details.

doPost: The method reacts to HTTP POST requests that users send from their registration form submission. During this process the user data gets validated before the program stores account information into the database.

Usage in the project:

doGet: This method enables users to view registration or sign-up forms by executing a click action.

doPost: Users present their registration data through this operation. The method establishes a new account when all entry criteria are valid and potentially leads users to visit the login or welcome sections.

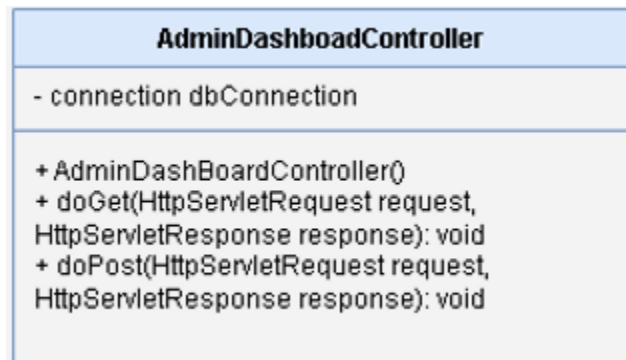
4.2.25) Admin Dashboard Controller

Figure 54 Class diagram for Admin Dashboard Controller

4.2.26) Admin Profile Controller

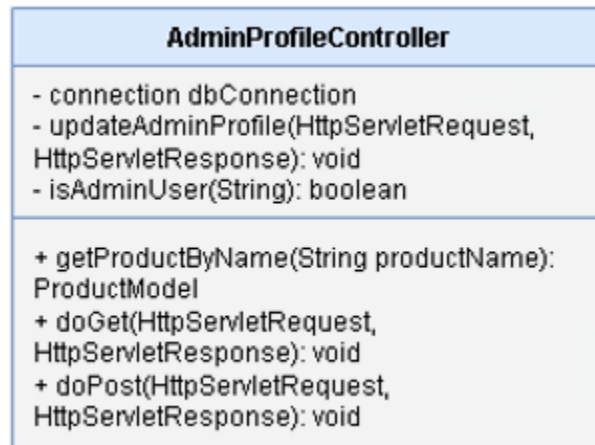


Figure 55 Class diagram for Admin Profile Controller

4.2.27) Initial Controller

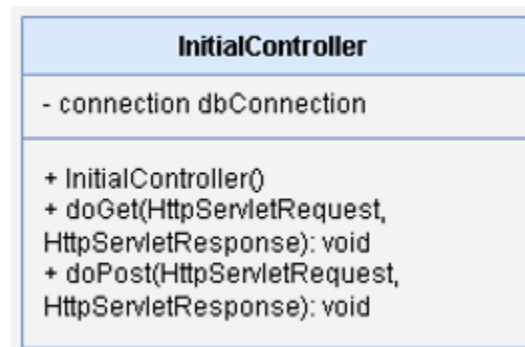


Figure 56 Class diagram for Initial Controller

4.2.25) Dashboard Product Controller

Figure 57 Class diagram for Dashboard Product Controller

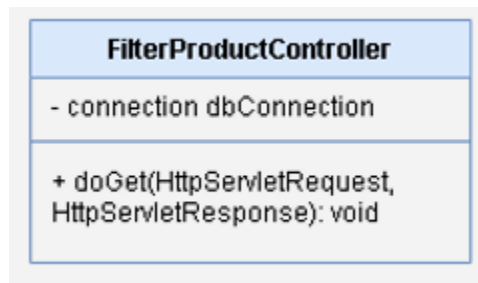
4.2.26) Filter Product Controller

Figure 58 Class diagram for Filter Product Controller

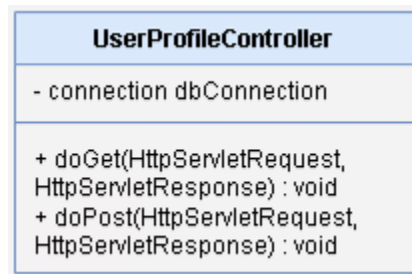
4.2.27) User Profile Controller

Figure 59 Class diagram for User Profile Controller

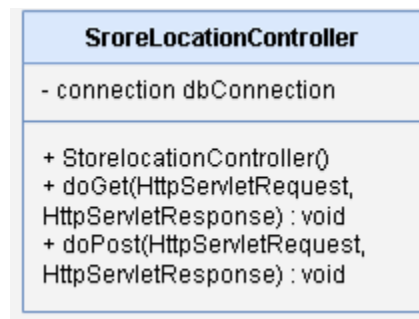
4.2.28) Store Location Controller

Figure 60 Class diagram for Store Location Controller

4.2.29) Search Product Controller

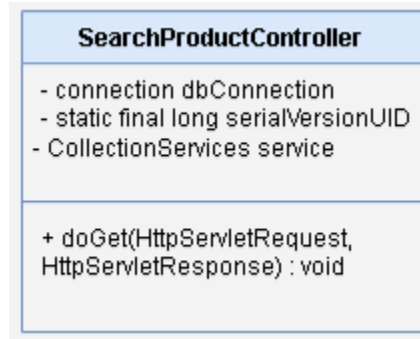


Figure 61 Class diagram for Search Product Controller

4.2.25) Ysindex Controller

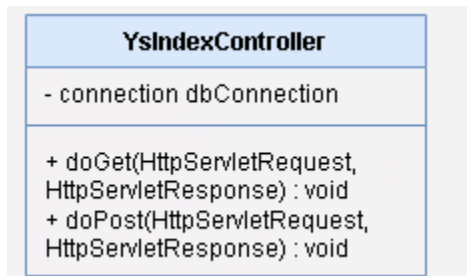


Figure 62 Class diagram for Ysindex Controller

5.1.1) Test 1 User Registration

Invalid Username

Test No	Input	Expected Output	Actual Output	Status
1	Enter incorrect credentials	Show invalid message	Username must start with a letter and be alphanumeric	Pass

For this test, users enter an incorrectly formatted username to check if the system blocks a login with wrong credentials. Your username cannot be used because the first letter is not a letter. The validation message the system provides for the issue says, “The username should be a letter followed by alphanumeric characters.” As a result, input constraints are applied, helping to ensure a good experience and protect data. The expected behaviour was observed when the test was performed.

Evidence:

The screenshot shows the AURA website's registration page. The background features a misty forest scene. The page includes a login link, a sign-up prompt, and a registration form. The form fields are: Full Name, Username, mm/dd/yyyy (date), Gender, Email, Password, and Phone Number. A red error message is displayed above the Username field: "Username must start with a letter and be alphanumeric". The "Register" button is at the bottom right.

Figure 63 - Evidence of test 1 (Invalid Username)

Password Strength

Test No	Input	Expected Output	Actual Output	Status
2	Enter weak password	Show "Password too weak"	Password must be 8+ chars with a numeric symbol, and uppercase letter	Pass

This test ensures password strength verification by typing in a password that doesn't pass all the rules. You should see a message warning that your password is not strong enough. When you submit your password, a detailed note appears telling you to add a symbol, number and uppercase letter for stronger security. It ensures that the way we validate passwords is correct. Tests were passed when the program's outcome matched the specified outcomes.

Evidence:

The screenshot shows the AURA website's registration page. The background is a dark, misty forest. The AURA logo is at the top center. On the left, there's a 'login' link and a sign-up prompt: 'YOUR JOURNEY BEGINS HERE SIGN UP TO UNLOCK REALM OF REFINEMENT AND PRIVILEGE'. Below this, it says 'Access exclusive collections, personalize your experience, and stay in touch with the latest innovations in timekeeping. Whether you're drawn to timeless elegance or modern sophistication, your journey starts here.' At the bottom left, there's a 'CONTACT US' section with phone numbers and an 'EMAIL' section with email addresses. On the right, there's a 'JOIN THE EXPERIENCE' section with a registration form. The form includes fields for Full Name, Username, Password (with a strength indicator), Gender, Email, and Phone Number. A 'Register' button is at the bottom right. A message above the form states: 'Step into your private space with confidence and elegance. Password must be 8+ chars, with a number, symbol, and uppercase letter'.

Figure 64 - Evidence of test 1 (password strength)

Invalid Gender

Test No	Input	Expected Output	Actual Output	Status
3	Enter invalid gender	Gender must be in male or female	Gender must be in “male” or “female”	Pass

The test does this by inserting a value that is neither male nor female. The system is made to inspect data errors closely and reflect with an error notification whenever it detects any. Result then indicates that only male or female are allowed for gender field. As a result, data is accurate and consistent in the database. I passed the test since I saw the expected validation message.

Evidence:

The screenshot shows the AURA website's registration page. The background is a dark, misty forest. The header 'AURA' is at the top center. On the left, there's a 'login' link and a section titled 'YOUR JOURNEY BEGINS HERE SIGN UP TO UNLOCK REALM OF REFINEMENT AND PRIVILEGE'. Below this, it says 'Access exclusive collections, personalize your experience, and stay in touch with the latest innovations in timekeeping. Whether you're drawn to timeless elegance or modern sophistication, your journey starts here.' There are two contact sections: 'CONTACT US' with phone numbers and 'EMAIL' with email addresses. At the bottom left, it says 'Address Kathmandu, Nepal'. On the right, there's a 'JOIN THE EXPERIENCE' section with a sub-header 'Step into your private space with confidence and elegance.' Below this, there's a validation error message: 'Gender must be 'male' or 'female''. The registration form includes fields for 'Full Name', 'Username', 'mm/dd/yyyy' (date), 'Gender', 'Email', 'Password', and 'Phone Number'. A 'Register' button is at the bottom right.

Figure 65 - Evidence of test 1 (invalid gender)

Allowed only Nepali number

Test No	Input	Expected Output	Actual Output	Status
4	Only input Nepali numbers which starts with 98 with 10 digits	Phone number must be a 10-digit number starting with 98	Phone number must be a 10-digit number starting with 98	Pass

To test the phone number validation, you enter a number that does not follow the way Nepali numbers are written. Only 980 numbers and numbers with a first digit of “9” will be accepted. When an invalid phone number was supplied, the application showed the expected message, “Phone number must be a 10-digit number prefaced with 98.” It proves the phone number is checked against the country’s standards for formatting. The test completed successfully because the output behaved as planned.

Evidence:

YOUR JOURNEY BEGINS
HERE SIGN UP TO
UNLOCK REALM OF
REFINEMENT AND
PRIVILEGE.

Access exclusive collections, personalize your experience, and stay in touch with the latest innovations in timekeeping. Whether you're drawn to timeless elegance or modern sophistication, your journey starts here.

CONTACT US
+977-9765588956
+977-9842430091

EMAIL
Aurah@aurahoclock@gmail.com
Aurapromax@gmail.com

Address
Kathmandu, Nepal

JOIN THE EXPERIENCE
Step into your private space with confidence and elegance.
Phone number must be a 10-digit Nepali number starting with 98

Full Name
Username
mm/dd/yyyy
Gender
Email
Password
Phone Number

Register

Figure 66 - Evidence of test 1 (Nepali number only)

Username already exists

Test No	Input	Expected Output	Actual Output	Status
5	Entering same username twice	Database error: duplicate entry found.	Database error: duplicate “username” for key Username	Pass

The system forces usernames to be unique during the registration process because of this test. Once users have chosen the same username twice, the application has to check for the repetition and not let a new record be saved. After testing, the system produced a message revealing that the unique constraint on username is active and working. This leads to fewer issues where users have the same information. I got the right response when I ran the test which means it passed.

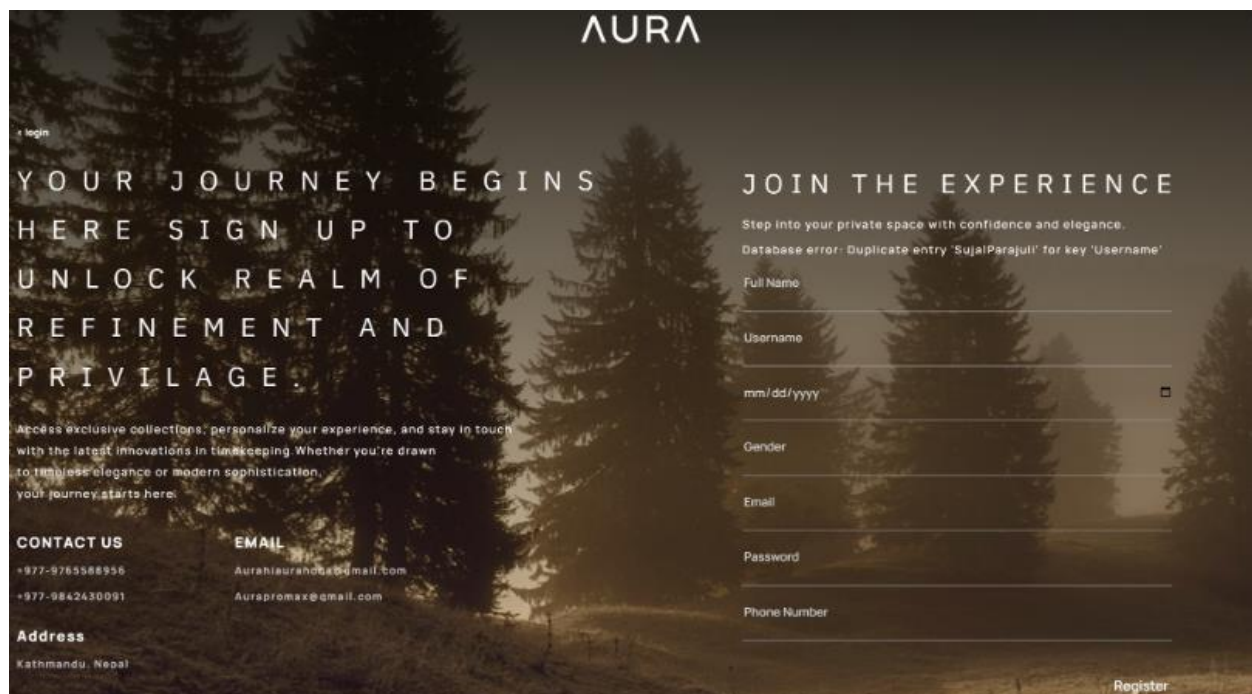


Figure 67 - Evidence of test 1 (Unique username)

User details stored in database:

Test No	Input	Expected Output	Actual Output	Status
6	Registering user details	Data stored in database	Database where you credentials are shown	Pass

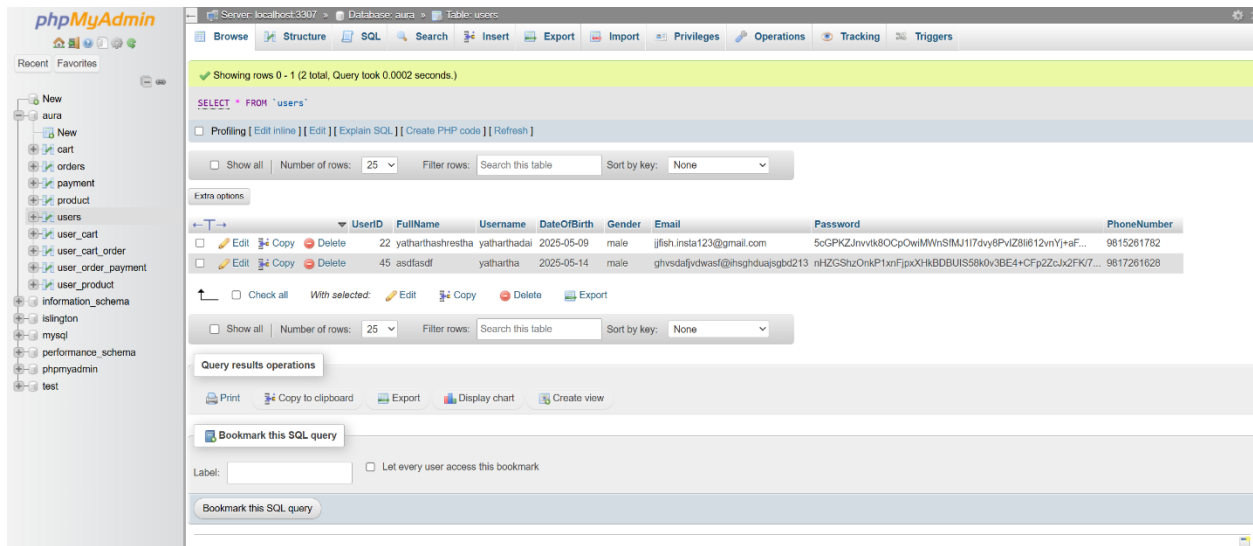


Figure 68 - User details stored in database test 6

5.1.2) Test 2 User Login

Wrong Credentials:

Test No	Input	Expected Output	Actual Output	Status
1	Enter incorrect credentials	Show invalid message	Invalid username or passwords	Pass

We can see that if an incorrect username is typed, the login page does not proceed. The username doesn't start with a letter, or it includes non-alphanumeric characters. Once the user tries typing, a validation message appears saying that the username should begin with a letter and include only alphanumeric characters. So, unneeded work and delay are avoided, and the user interface is improved. There are no session or cookies generated since validation ends in failure.

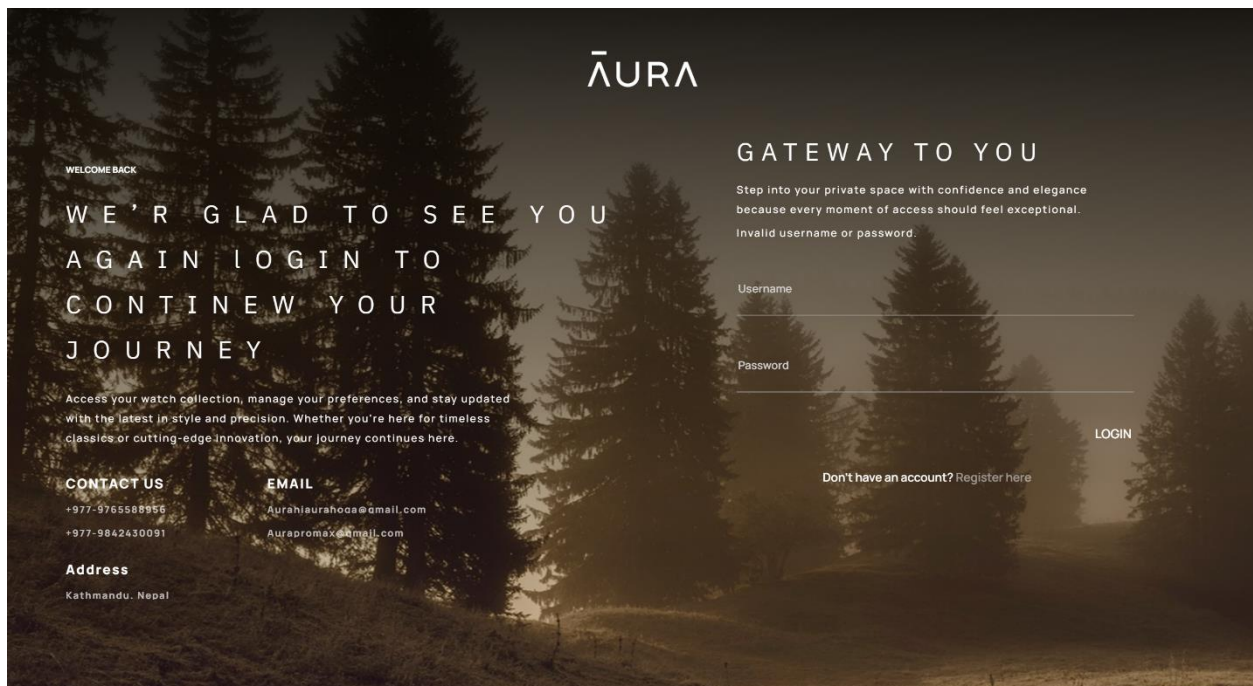


Figure 69 - Showing wrong credentials

Session Handling in Browser

Test No	Input	Expected Output	Actual Output	Status
2	Successful login	Session stores user data	Session created with correct attributes	Pass

It confirms if the browser receives a session after successful login. When you introduce the proper data, the server is designed to place the session attributes for username and role. Upon running the testing, it was shown that the browser stayed signed in, combined with correct session values, allowing for personalized use on several web pages. The test succeeded because session data creation and management went as scheduled.

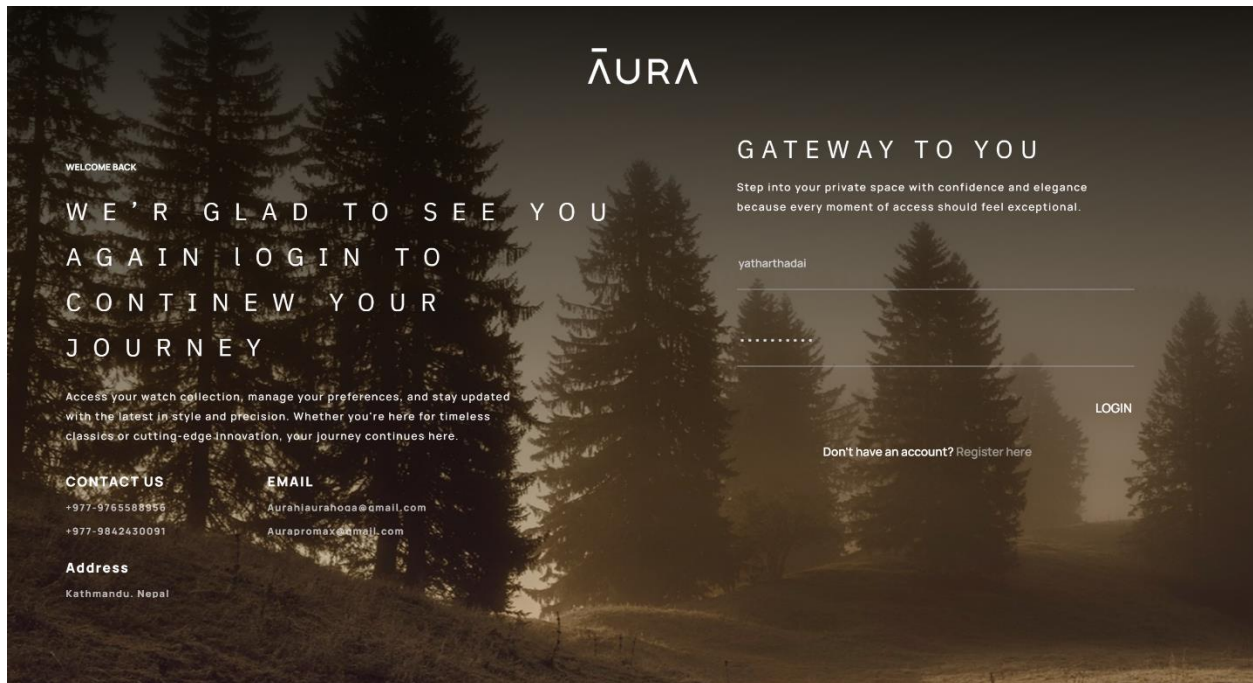


Figure 70 Handling In browser

After entering the details of admin, it will directly redirect to the admin page as shown in the figure below.

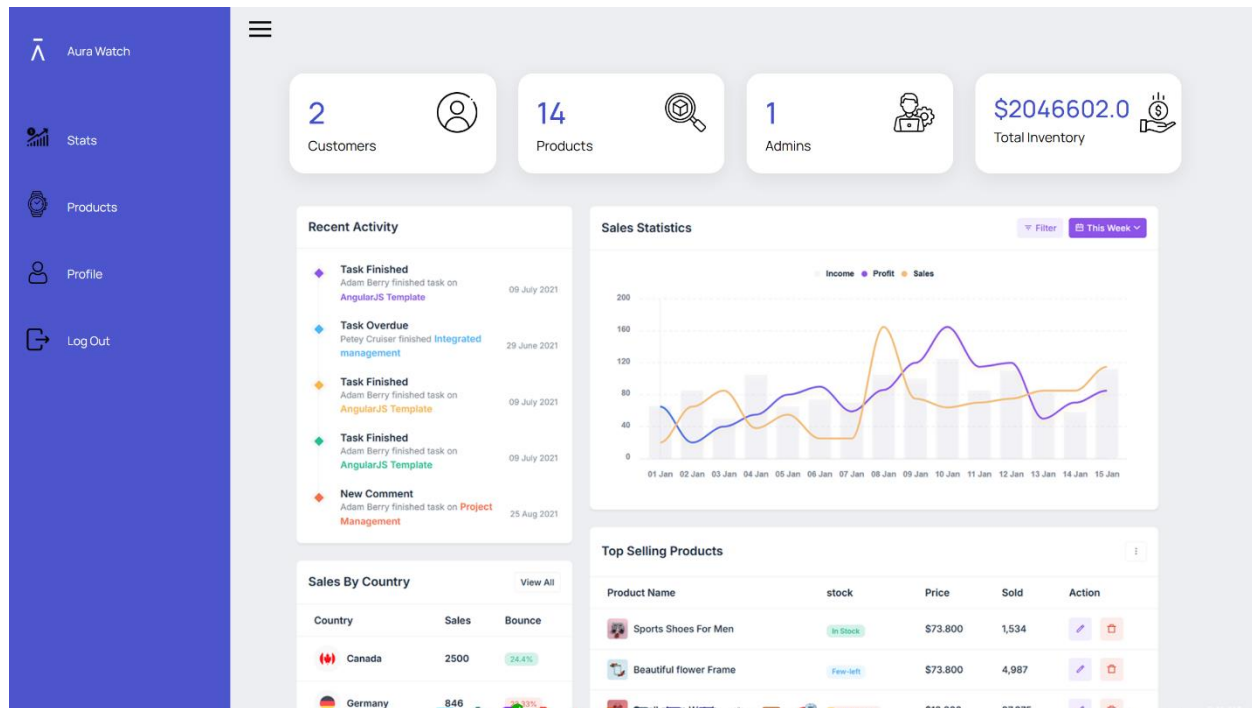


Figure 71 redirect to the admin page

Cookie Creation and Validation

Test No	Input	Expected Output	Actual Output	Status
3	Successful login	Cookies created for username and role	Cookies username and role stored in browser	Pass

It checks if cookies are correctly generated when a login process completes successfully. If the user enters the correct username and password, cookies should hold their username and role, supporting their return to the application whenever they want to log in again. All the cookies found in the browser contained both the correct information. Because of this, both the user journey and retaining session data are less troublesome. There were no problems with the test.

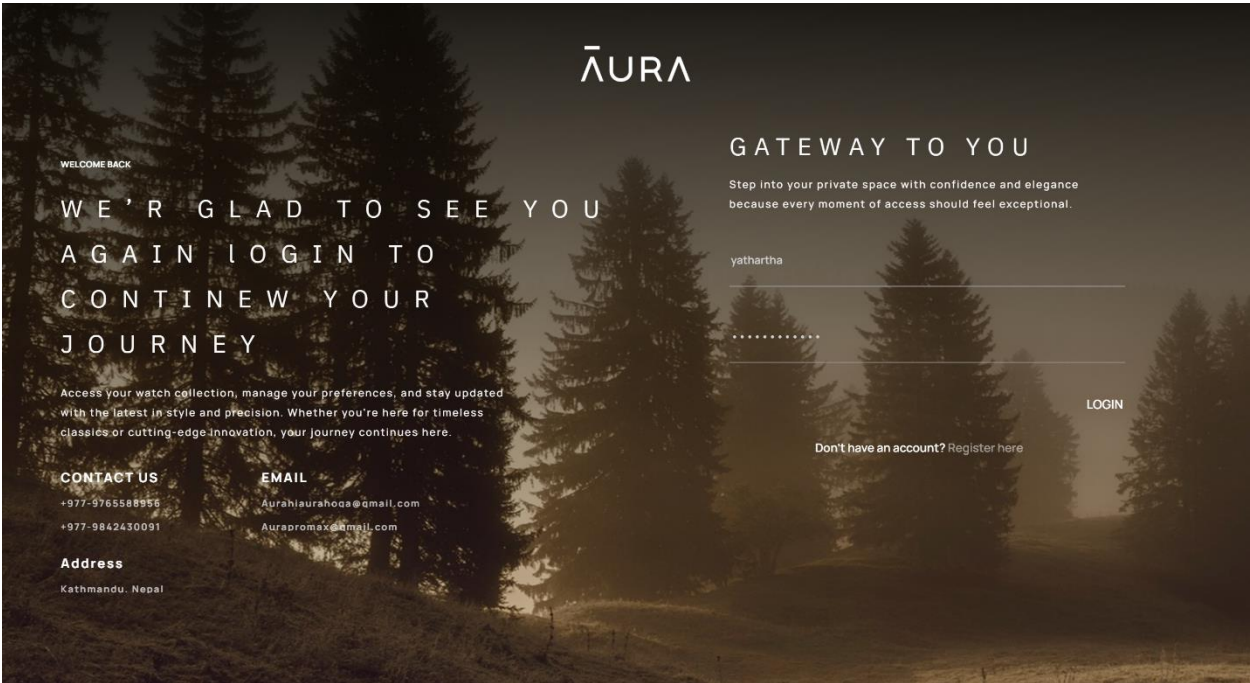


Figure 72 Cookie Validation

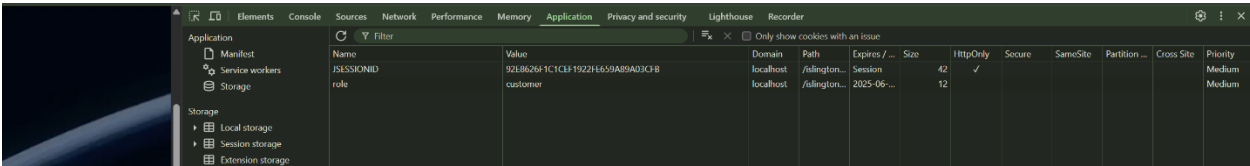


Figure 73 Cookie Validation Evidence

The screenshot shows the phpMyAdmin interface for a database named 'aura'. The 'users' table is selected, and its structure and data are displayed. The table has 8 columns: UserID, FullName, Username, DateOfBirth, Gender, Email, Password, and PhoneNumber. Two rows of data are shown:

UserID	FullName	Username	DateOfBirth	Gender	Email	Password	PhoneNumber
22	yatharthashrestha	yatharthadai	2025-05-09	male	ijfah.insta123@gmail.com	5cGPKZJnvvtk8OCpOwIMMnS1MJ117dyv9PvIZ8il612mY)raF...	9815261782
45	asdfsdf	yathartha	2025-05-14	male	ghvsdafvdfwastf@hghduajsgbd213	nhIZGShzOnkP1xnFjpx0G-lkBD8U1S58k0v3BE4+CFp2ZcJx2FK7...	9817261628

Figure 74 User table

Redirects users to the correct dashboard

Test No	Input	Expected Output	Actual Output	Status
4	Successful login	Redirects to user or admin dashboard based on role	Redirects to home for customer and admin dashboard for admin	Pass

It confirms that users with each assigned role can see the right dashboard when they log in. Upon login, a customer is re-routed to the home page, while an admin is redirected to the Admin Dashboard page. Users were sent to the correct part of the site once the site checked their session or cookie. Role-based routing was tested through the set-up and found to be correct.

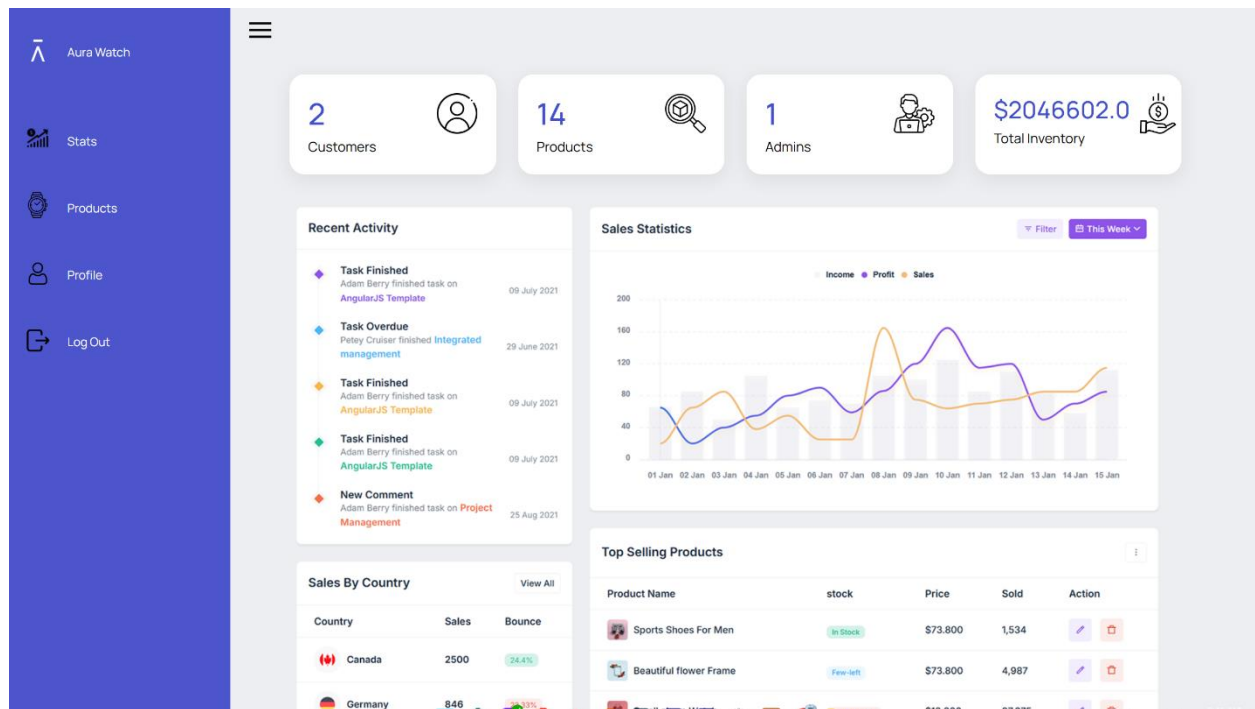


Figure 75 Login success

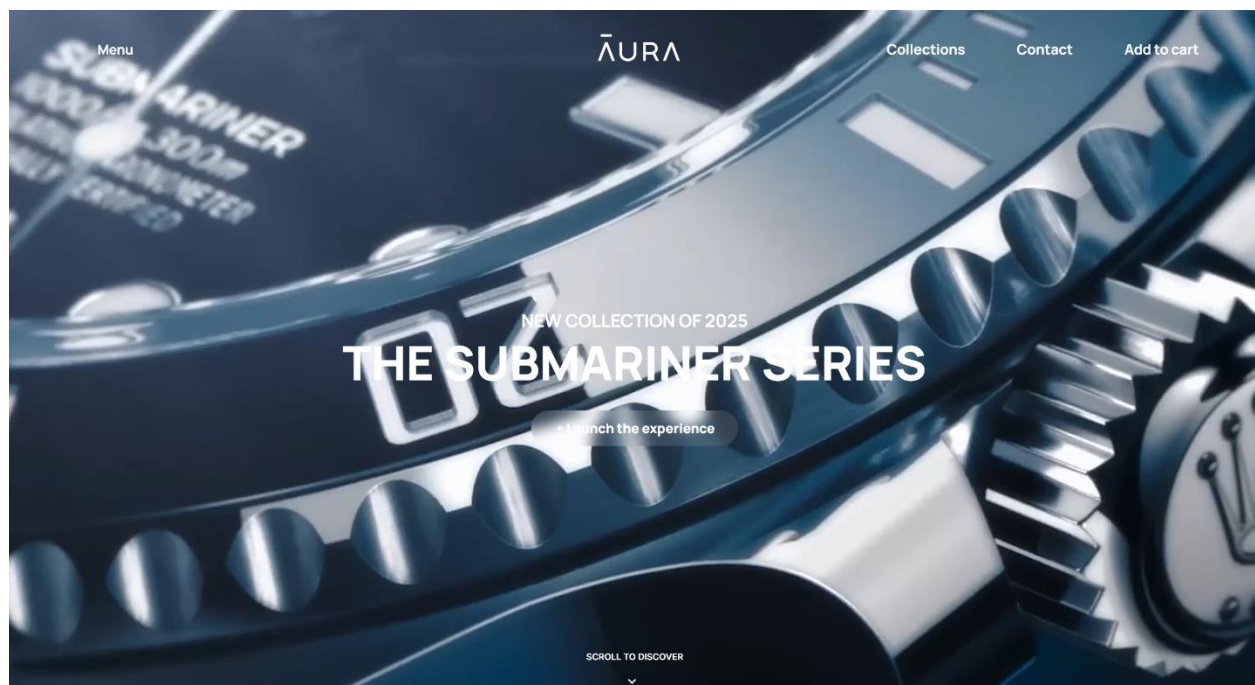


Figure 76 test

Product

Test No	Input	Expected Output	Actual Output	Status
1	Add new product	Product added and displayed on dashboard and Collection	Product appeared on dashboard and Collection page	Pass

The test ensures a new product can be added and edited through the admin panel. The product was made available on the Collection page and admin panel after the correct form details were entered and the form submitted. It means product creation is happening as planned. Everything was fine during the test.

Add Products

Product Name: Product Price: Watch Brand: Previous Bid:

Description:

Product Image:

Manage Products
Manage all products in the system

Product Name	Description	Price	Watch Brand	Previous Bid	Actions
Omega Speedmaster	The Omega Speedmaster, often nicknamed the "Moonwatch", is a legendary chronograph watch first introduced by Omega in 1957. Originally designed for racing and motorsports, it gained worldwide fame for its association with NASA's space missions—most notably becoming the first watch worn on the Moon during the Apollo 11 mission in 1969.	\$7700.0	Omega	\$6500	Delete Edit
Rolex Yacht-Master	The Rolex Yacht-Master is a luxury nautical sports watch introduced in 1992. It blends Rolex's signature rugged performance with a more elegant, refined design, making it a favorite among sailors, skippers, and watch collectors who want both function and style.	\$17000.0	Rolex	\$18700	Delete Edit
Rolex Submariner	The Rolex Submariner is a luxury dive watch first introduced in 1953. It quickly became an icon for both professional divers and watch enthusiasts. Renowned for its robust build, underwater performance, and timeless design, the Submariner has become a symbol of prestige, reliability, and adventure.	\$12000.0	Rolex	\$12600	Delete Edit
Audemars Piguet Royal Oak Double Balance Wheel is an exceptional timepiece that combines cutting-					Delete Edit

Figure 77 - Before adding products

Add Products

Product Name: Enter Product Name Product Price: Enter Price Watch Brand: Enter Watch Brand Previous Bid: Enter Previous Bid

Description: Enter Description Product Image: No file chosen

Manage Products
Manage all products in the system

Product uploaded successfully!

Product Name	Description	Price	Watch Brand	Previous Bid	Actions
Celestial Moon Age	Patek Philippe is a prestigious Swiss watch manufacturer known for producing some of the most luxurious and technically advanced timepieces in the world. These watches are often crafted using 18k gold or platinum, with dials made from materials like enamel, mother-of-pearl, or precious stones.	\$142511.0	Patek Philippe	\$152511	Delete Edit
Omega Speedmaster	The Omega Speedmaster, often nicknamed the "Moonwatch", is a legendary chronograph watch first introduced by Omega in 1957. Originally designed for racing and motorsports, it gained worldwide fame for its association with NASA's space missions—most notably becoming the first watch worn on the Moon during the Apollo 11 mission in 1969.	\$7700.0	Omega	\$6500	Delete Edit
Rolex Yacht-Master	The Rolex Yacht-Master is a luxury nautical sports watch introduced in 1992. It blends Rolex's signature rugged performance with a more elegant, refined design, making it a favorite among sailors, skippers, and watch collectors who want both function and style.	\$17000.0	Rolex	\$18700	Delete Edit

Figure 78 - Product added successfully

Product updated in collection

Test No	Input	Expected Output	Actual Output	Status
2	Update product info	Product updated and changes reflected on all pages	Product updated and visible changes across UI	Pass

You can use this test to find out if existing information about the product can be changed. A product's name and price were changed by a staff member via the update form. All changes made after submission were properly added to the database and instantly seen on both the dashboard and the Collection page. It proves that the app can be fully updated. The result from the test was what was anticipated.

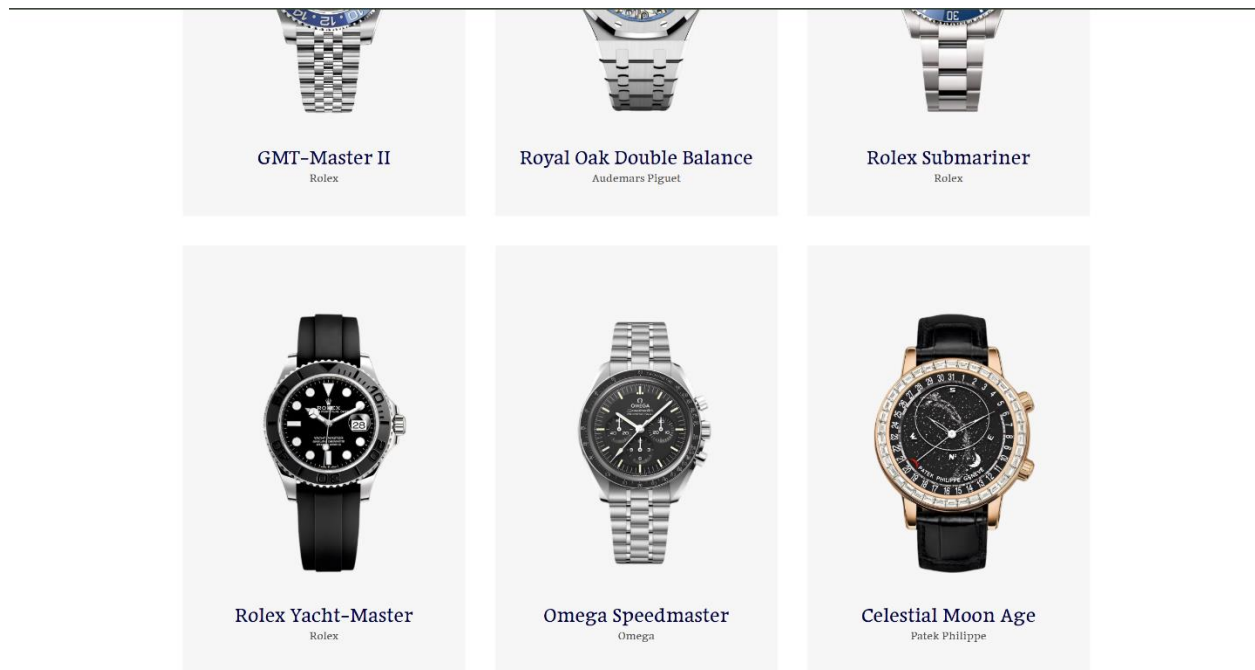


Figure 79 - Product update in collection page

Product updated in product detail page

Test No	Input	Expected Output	Actual Output	Status
2	Update product in product detail page	Product updated and changes product detail page	Product updated and visible changes across UI	Pass

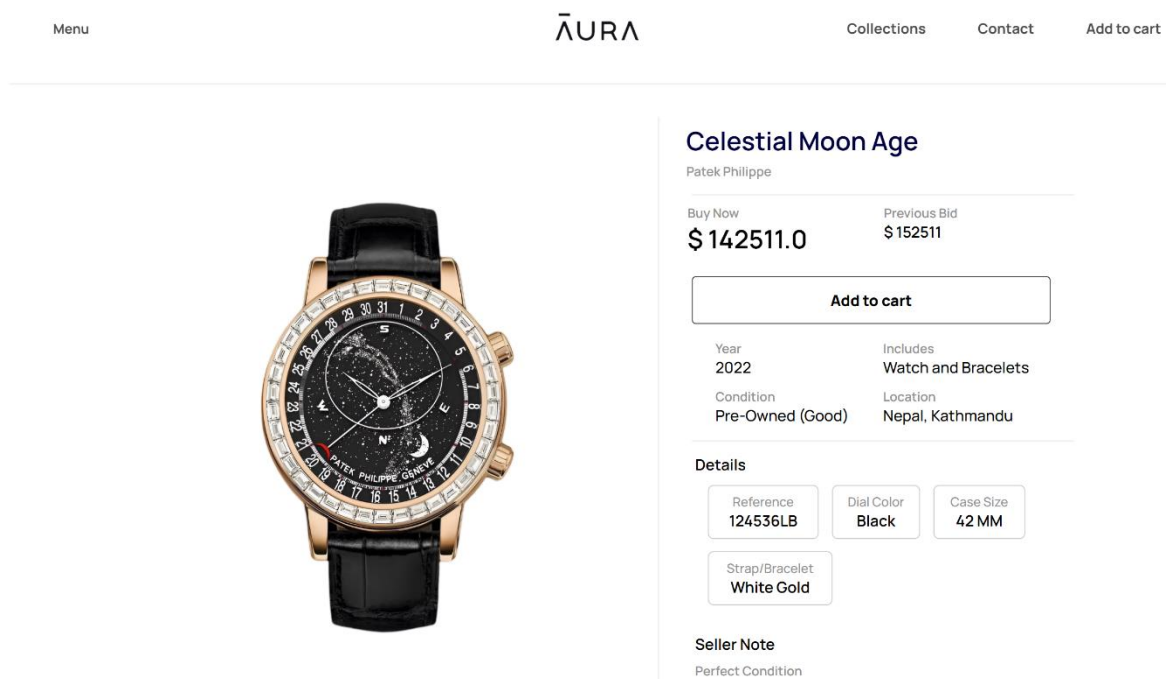


Figure 80 - Product updated in product detail page

Add to Cart

Test No	Input	Expected Output	Actual Output	Status
1	Adding product	Product updated and changes reflected add to cart page	Product updated and visible changes across UI	Pass

Ensuring a new product is inserted into the database and shown on screen, especially in the Add to Cart area, is the key aim of the test. After the product was created using the admin panel, it immediately appeared to users on Collection and cart-related screens. We can see that messages are being transferred correctly between parts of the application. The result of the test was as planned.

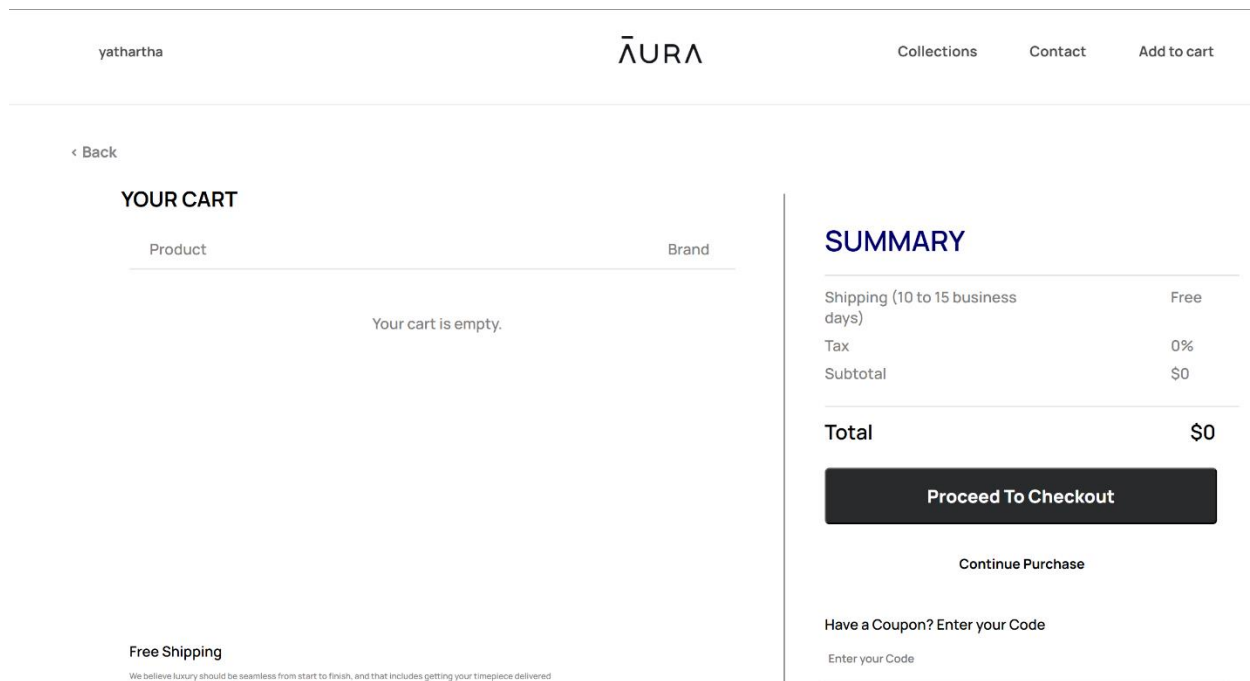


Figure 81- add to cart page before adding products

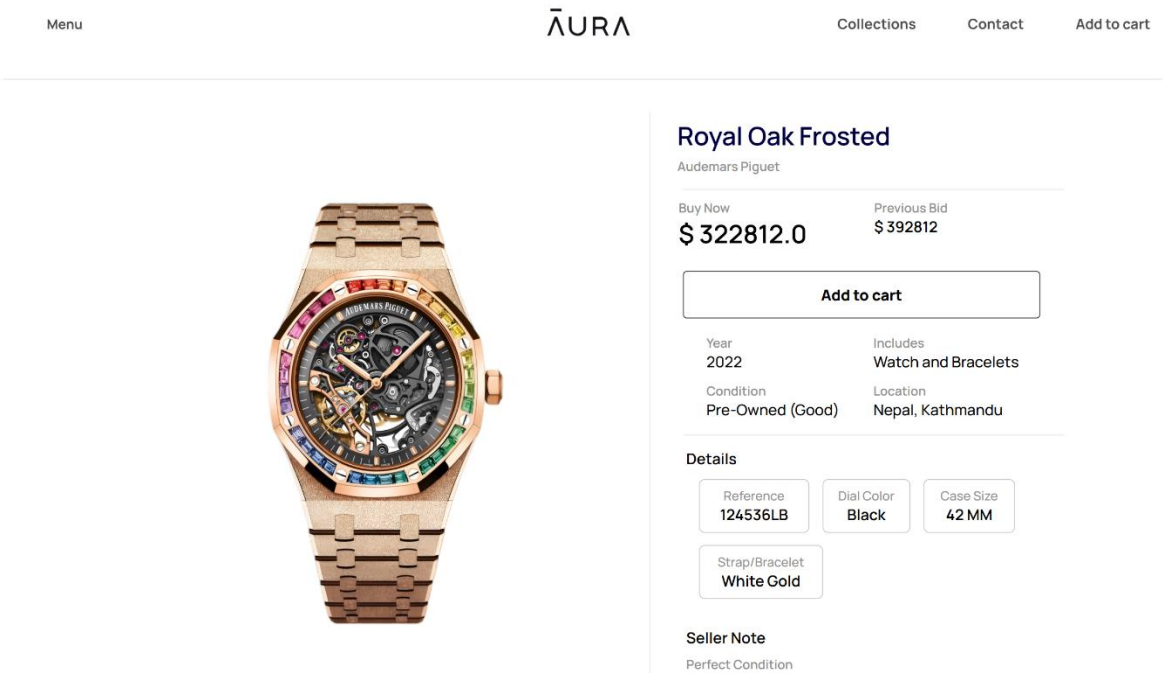


Figure 82 - Adding product on add to cart page

Product Added to add to cart successfully

Test No	Input	Expected Output	Actual Output	Status
2	Added to add to cart	Product updated and changes reflected add to cart page	Product updated and visible changes across UI	Pass

yathartha

ĀURA

CollectionsContactAdd to cart

< Back

YOUR CART

Product	Brand
 Royal Oak Frosted \$ 322812.0 Audemars Piguet	

Free Shipping

We believe luxury should be seamless from start to finish, and that includes getting your timepiece delivered to your door at no extra cost.

SUMMARY

Shipping (10 to 15 business days)	Free
Tax	0%
Subtotal	\$322812.0

Total

\$322812.0

Proceed To Checkout

Continue Purchase

Have a Coupon? Enter your Code

Enter your Code

23050262 Sujal Parajuli

93

6) Development

6.1) UI/UX Development

UI/UX development focused on delivering a neat, professional, and engaging user interface for the luxury watch company. The frontend was developed using JSP and CSS, which enabled dynamic display of content with attractive styling.

All development was done using the Eclipse IDE, which offered a comfortable and efficient workflow.

Key design choices included:

- Looks good on phones, tablets and computer which shows it is a responsive design compatible in all the devices.
- The design stays clean and uncluttered to keep attention on high quality product images and key product information, so that customers can see what they need easily.
- Continuing the same fonts and same color scheme all through the site adds to brand identity and also makes the site appear professional and cohesive.
- The thoughtfulness of the way CSS animations and hover effects add a little layer of interactivity to the site aids by covering up much of the lack of interactivity, but without being distracting.

6.2) Authorization and Authentication

The website relies on a secure Java (Servlets) server-side user authentication mechanism. User credentials are stored and managed through a relational database (e.g., MySQL), form validation, and access control performed both on the front-end as well as back-end.

6.2.1) Register Page

Aura time's e-commerce website registration system was implemented using Java Servlets where RegisterController.java mapped to URL that is /register. It takes care of both GET as well as POST request and annotated with @MultipartConfig if need of file upload arises further in its expansion.

Functional Workflow:

- GET request handling: If a GET request is made to /register, then it forwards the user to register.jsp.

```

104
105
106  @Override
107  protected void doGet(HttpServletRequest request, HttpServletResponse response)
108      throws ServletException, IOException {
109      request.getRequestDispatcher("/WEB-INF/pages/register.jsp").forward(request, response);
110  }
111  }
112

```

- POST Request Handling: In the submission form the servlet takes all the data which are submitted by user from the form which includes fullname, username, DOB, gender, Email, password and phone number. Then it validates input using ValidationUTIL

```

25  }
26  public class RegisterController extends HttpServlet {
27      private static final long serialVersionUID = 1L;
28
29  @Override
30  protected void doPost(HttpServletRequest request, HttpServletResponse response)
31      throws ServletException, IOException {
32
33      String fullName = request.getParameter("FullName");
34      String username = request.getParameter("Username");
35      String dateOfBirth = request.getParameter("Dateofbirth");
36      String gender = request.getParameter("Gender");
37      String email = request.getParameter("Email");
38      String password = request.getParameter("Password");
39      String phoneNumber = request.getParameter("Phonenumber");
40
41      RequestDispatcher dispatcher = request.getRequestDispatcher("/WEB-INF/pages/register.jsp");
42

```

If validation is successful connection is established using `DbConfig.getDbConnection()`. Passwords are securely encrypted using `PasswordUtil.encrypt(password, username)` before storage. Then a success message is shown otherwise failure feedback is provided.

6.2.2) Login Page

Aura times website was developed using the login functionality where it is developed using the Java Servlets and inched to the /login route where the LoginController was mapped. Now follows from Model View Controller (MVC) model and brings with it a responsibility to render login page and at a later point onwards be responsible for user authentication.

Functional Workflow:

- Get Request handling: The controller directs users to login.jsp page through /login endpoint using GET requests using:

```
26
27 ● @Override
28 protected void doGet(HttpServletRequest request, HttpServletResponse response)
29     throws ServletException, IOException {
30     request.getRequestDispatcher("/WEB-INF/pages/login.jsp").forward(request, response);
31 }
32
```

Figure 83 – Code for Get Request Handling

Following page consist of frontend from where users can enter their login details.

- Post Request Handling: The POST method functions as a temporary solution for upcoming implementation. After processing form data and validating credentials and checking the database the POST method will create a session when authentication succeeds.

```
34 ● @Override
35 protected void doPost(HttpServletRequest request, HttpServletResponse response)
36     throws ServletException, IOException {
37
38     String username = request.getParameter("Username");
39     String password = request.getParameter("Password");
40     RequestDispatcher dispatcher;
41
```

Future version of doPost() method is expected to:

- Retrieve from data (email/username and password).
- Checking the details of users against stored encrypted values which are stored in database.
- Set session attributes upon successful login.
- Use the role of the user and redirect them.
- Provide feedback if authentication fails.

6.2.3) Role-based page access Redirection

There are 2 main types of roles for this system: Admin and Users. After logging in with their proper details, they will be sent to another page after according to their roles.

Admin: Admin role after logging in will be directed to a separate admin dashboard interface that would allow them to manage all aspects of the whole system. Here's what the admin panel will include once developed:

- Admins will be able to see, add, edit, or remove user accounts.
- They will have unlimited access to adding new products, editing current ones, or removing products from the store.
- Admins will track customer orders, change order status, and handle cancellations or refunds.
- A dashboard showing sales data, customer trends, and business insights will aid in decision-making.

```
60
61         // Determine role
62         String role = "customer"; // default
63         if ("Sujal".equalsIgnoreCase(rs.getString("Username"))) {
64             role = "admin";
65         }
66         SessionUtil.setAttribute(request, "Role", role);
67
```

User: User role after logging in will be directed to home page which will allow them to see product which they can browse and purchase it by keeping it to add the purchase to cart other roles are described below:

- Users can register, log in, and manage their personal information and always maintain current profile.
- Users browse the product catalog, they can see the details of each watch, and purchase them without any disruptions.
- Customers are able to follow up after making purchases by tracking order, view order history, and get help from customer support regarding order related issues.

```
67
68         // Set role cookie using CookieUtil (valid 30 days)
69         CookieUtil.addCookie(response, "role", role, 30 * 24 * 60 * 60);
70
71         // Redirect based on role
72         if ("admin".equals(role)) {
73             response.sendRedirect("AdminDashBoard"); // Assuming this is a servlet mapping
74             return;
75         } else {
76             dispatcher = request.getRequestDispatcher("/WEB-INF/pages/home.jsp");
77         }
78     }
```

6.3.4) Session and Cookie

6.3.4.1) Cookie Management (CookieUtil)

The essential methods for browser cookies on the client side are provided by the CookieUtil class. Key functionalities include:

- It sets a cookie with a name, value and expiration time like “Remember Me”.
- Retrieves a cookie by name from incoming requests (e.g.: checking authentication tokens).
- Use set cookie with a max age of 0, this will delete cookies and again used during logout.
- Cookies are configured with Path="/" to ensure they can be access across the entire application.

```
23 public static void addCookie(HttpServletResponse response, String name, String value, int maxAge) {  
24     Cookie cookie = new Cookie(name, value);  
25     cookie.setMaxAge(maxAge);  
26     cookie.setPath("/"); // Make cookie available to the entire application  
27     response.addCookie(cookie);  
28 }  
29
```

Figure 84 - Code of add cookie util which defines utility

The Java coding block establishes a utility function known as addCookie. The method requires a web server response object together with the cookie name and value while accepting the maximum age parameter expressed in seconds. The method employs the provided name and value to generate a fresh cookie before configuring the maximum age parameter and ensuring global accessibility across the web application and finally inserting it into the server response to be returned to the browser.

Add Cookie:

```
37 public static Cookie getCookie(HttpServletRequest request, String name) {  
38     if (request.getCookies() != null) {  
39         return Arrays.stream(request.getCookies())  
40             .filter(cookie -> name.equals(cookie.getName()))  
41             .findFirst()  
42             .orElse(null);  
43     }  
44     return null;  
45 }  
46
```

Figure 85 – Code of get cookie

This Java code provides two methods which put data (a value) into the user session through a specified key (name) and retrieve session items using a key while returning the content if present or returning an empty result. These techniques allow storing individual user data through successive web page requests.

Delete Cookie:

```
52  */
53  public static void deleteCookie(HttpServletResponse response, String name) {
54      Cookie cookie = new Cookie(name, null);
55      cookie.setMaxAge(0);
56      cookie.setPath("/"); // Make cookie available to the entire application
57      response.addCookie(cookie);
58  }
59 }
```

Figure 86 - Code for delete cookie

Delete Cookie removes a cookie from the browser by setting up a cookie with an empty value under the same name. It gives the cookie a MaxAge of 0, so the browser deletes it right when it is accessed. Make sure the path uses / so that the deletion will remove all entries for the application. In the end, a cookie is set in the response, telling the browser to perform the deletion.

6.3.4.2) Session Management (SessionUtil)

The SessionUtil class keeps track of user session on server side: when that data is retained while the user is still active.

Key features:

- It is used to store the data, for example, user ID and role in the session.
- This will be used to retrieve Attributes: a way of accessing session data, such as checking if a user is logged in.
- It helps to clear specific session data for example cart items after checkout.
- It kills sessions and user's logout or timeout for security.

1. setAttribute()

The setAttribute method places a value in the user session using a specified key. A session is retrieved or made using request.getSession(), after which the attribute is set with the key and value supplied. Consequently, information about users or their roles is saved during the session, helping manage login, restrictions and personal materials.

```
18  */
19  public static void setAttribute(HttpServletRequest request, String key, Object value) {
20      HttpSession session = request.getSession();
21      session.setAttribute(key, value);
22  }
23  }
```

Figure 87 - code for set Attribute

2. getAttribute()

getAttribute is used to get a value from the session using a specific key. It checks if a session was found (getSession(false)) and then loads the attribute if it exists. Should the session not exist or the attribute not be set, this function returns null. This technique lets you read things like a user's login status or their roles in a session without initiating a new session.

```
30  */
31  public static Object getAttribute(HttpServletRequest request, String key) {
32      HttpSession session = request.getSession(false);
33      if (session != null) {
34          return session.getAttribute(key);
35      }
36      return null;
37  }
38  }
```

Figure 88 - code for get Attribute

3. removeAttribute()

The removeAttribute method removes the specified attribute from the session by using its key. It locates the existing session and removes the named attribute, if it actually exists. It allows you to clear temporary session data such as logout flags and partial form inputs, so the session stays tidy every time the user interacts with the site.

```
45 public static void removeAttribute(HttpServletRequest request, String key) {  
46     HttpSession session = request.getSession(false);  
47     if (session != null) {  
48         session.removeAttribute(key);  
49     }  
50 }  
51
```

Figure 89 - code for remove Attribute

4. invalidateSession()

By using this method, you remove all authorization for the user. It determines if a session is current and, if so, calls invalidate() to delete all stored attributes and flag the session as uncooked. This takes place when someone logs out to make certain no session-related data is available, so someone else cannot access or use the session information.

```
56 /*  
57 public static void invalidateSession(HttpServletRequest request) {  
58     HttpSession session = request.getSession(false);  
59     if (session != null) {  
60         session.invalidate();  
61     }  
62 }  
63 }
```

Figure 90 - code for invalidate session Attribute

7) Coursework Development with Analysis.

7.1) Team Contribution:

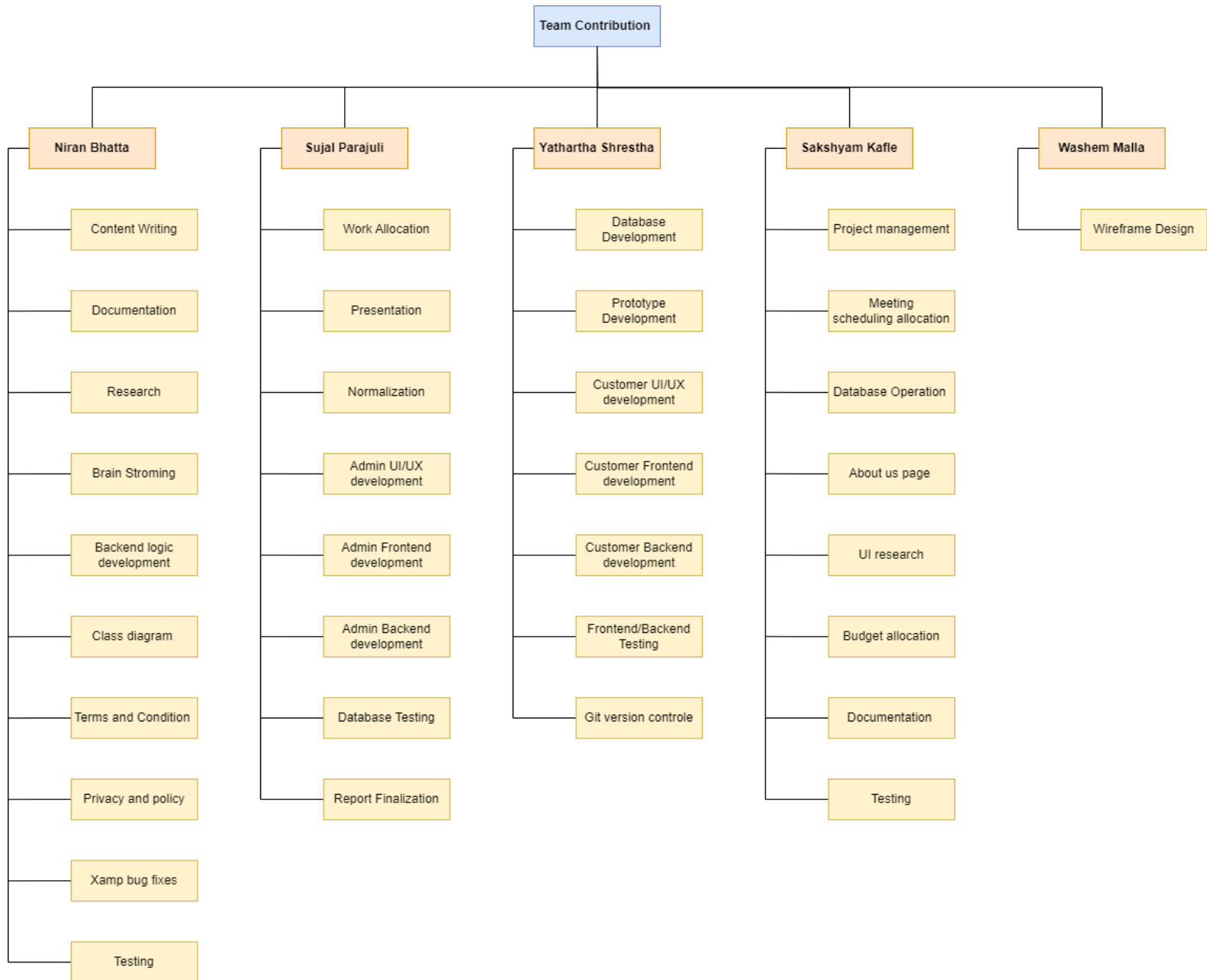


Figure 91 Team Contribution

Sujal Parajuli

Completed Login Feature

- Task Description: Sujal made sure users can enter their credentials to access the parts of the site reserved for them.

Implementation Steps:

- Designed the login form using JSP and CSS then connected it to the backend using Java servlets.
- Implemented form validation.
- Used HttpSession to maintain login state and redirect users based on roles.
- Connected the form to the MySQL database to validate credentials stored in the users table.

Evidence:

- Screenshot of the login form and successful login redirect to the admin dashboard

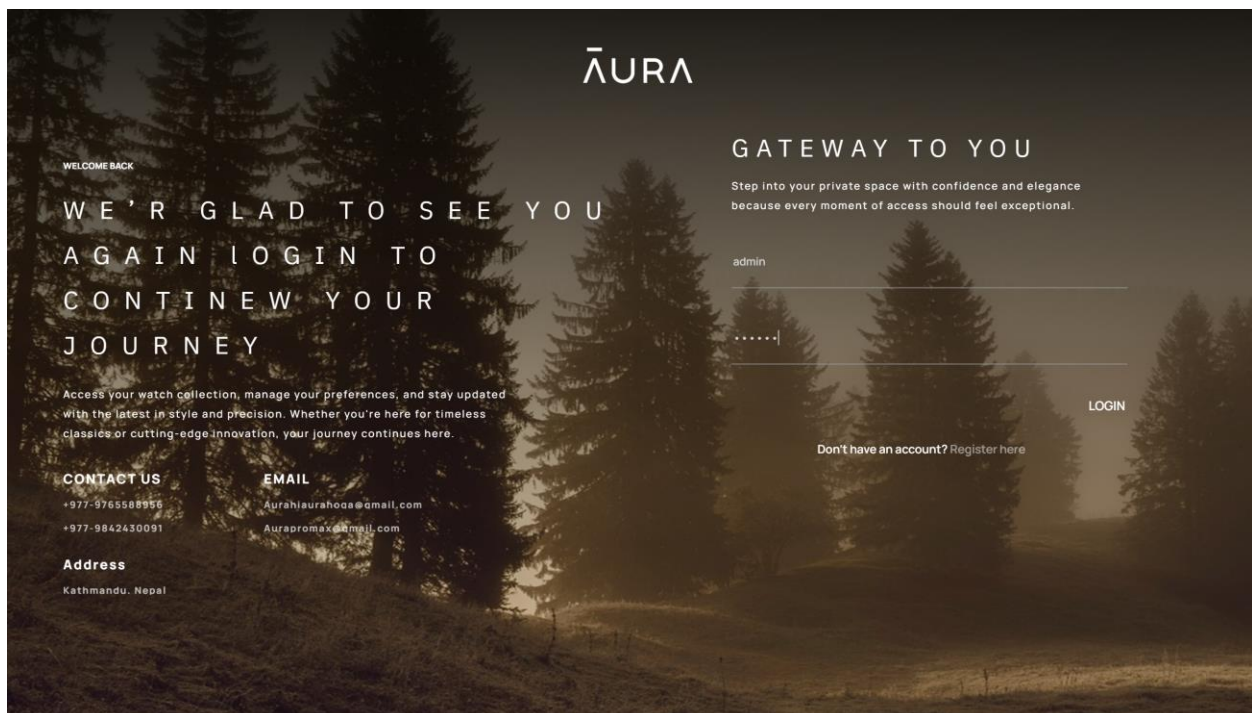


Figure 92 Evidence 1 Login Form

UI/UX Design for Login Feature

- Task Description: Sujal designed the user interface of the login form to be minimalistic and with modern touch, aligning with the branding of the Auratime website.

Implementation Steps:

- Used Figma to create prototype for the login screen.
- Implemented the frontend using JSP and CSS.
- Included form labels, placeholders and other many functions to enhance the usability.
- Added error display for invalid login attempts.

Evidence:

- Screenshot Admin Page redirection

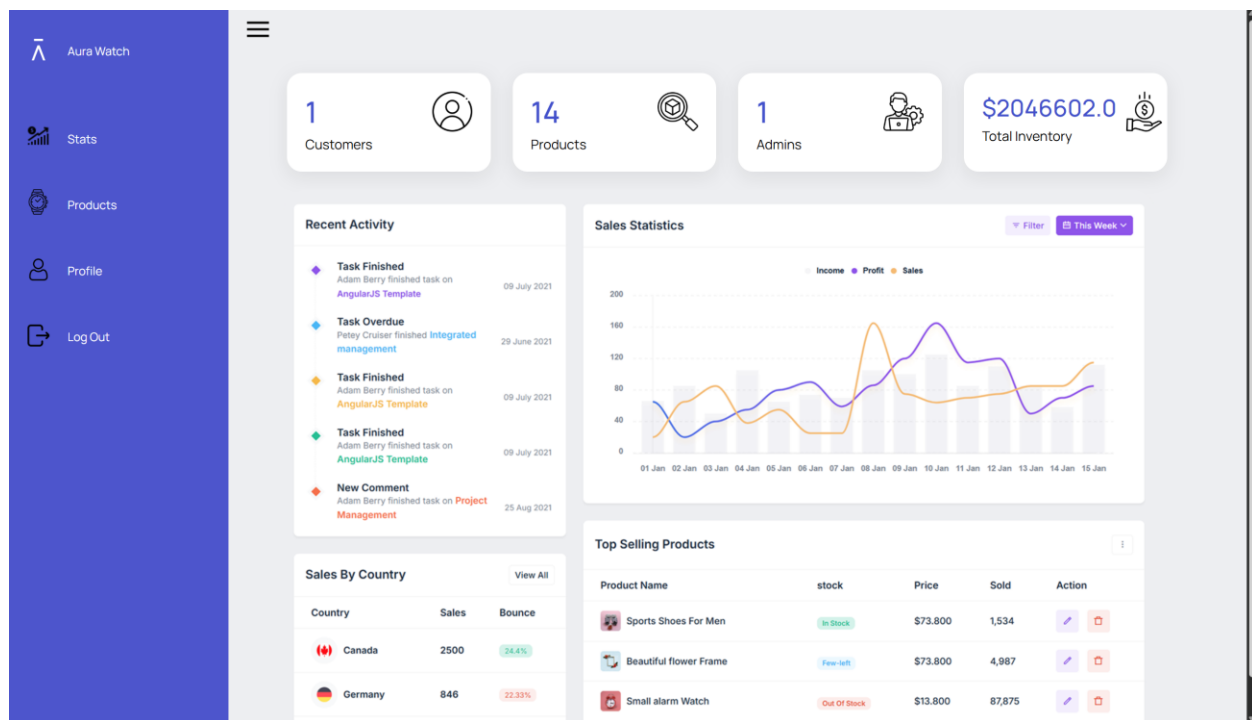


Figure 93 Final Implementation of Login page

Yathartha Shrestha

Completed Home Page Feature

- **Task Description:** Yathartha was tasked with designing and putting together the home page of Auratimes watch e-commerce site. On the home page, customers first see your products, what you do and what sets you apart.

Implementation Steps:

- Developed a design that is easy on the eyes using JSP, HTML and CSS.
- Featured watches are shown in dynamic sections integrated with our backend.
- Attach links to the Shop, About and Contact section of your website.

Evidence:

Screenshot of the final home page interface

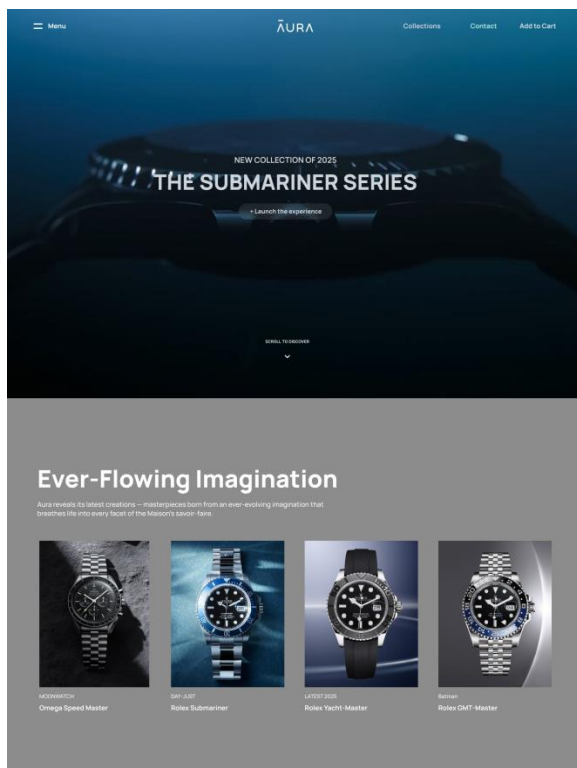


Figure 94 - Final implementation of home page

UI/UX Design for Home Page

- Task Description: The home page developed by Yathartha is easy to navigate, emphasizes Auratimes' unique character and puts its products in focus.

Implementation Steps:

- Used Figma to draft out the plan for the homepage with the section, the products shown and the footer included.
- Selected the same colours, typefaces and layouts for all your images.
- For a smooth UI, I made all the animated effects with JavaScript and CSS for when the user hovers over products.
- Helped ensure members would notice a familiar style similar to other pages.

Evidence:

- Screenshot of the CSS file of home page

Sakshyam Kafle

Completed about us page feature

Task Description: Sakshyam made the design and built the About Us section of the Auratimes watch e-commerce website. The objective of this page is to present the mission, vision, values and team, all to support the formation of user trust and the identity of the brand.

Implementation Steps:

- Created the about us page using JSP and CSS.
- Created a Mission, Vision, Strategy and Team Intro structure for the site layout.
- Made the left side of the template into a full-height picture section to eliminate any white space.
- Made sure the page responded well and looked similar to other site pages.

Evidence:

- Screenshot of the controller.java file of about us page

```
1 package com.islington.controller;
2
3 import jakarta.servlet.ServletException;
4
5
6 /**
7  * Servlet implementation class Home
8  */
9
10 @WebServlet(asyncSupported = true, urlPatterns = { "/aboutus" })
11 public class AboutUsController extends HttpServlet {
12     private static final long serialVersionUID = 1L;
13
14     /**
15      * @see HttpServlet#HttpServlet()
16      */
17     public AboutUsController() {
18         super();
19         // TODO Auto-generated constructor stub
20     }
21
22     @Override
23     protected void doGet(HttpServletRequest request, HttpServletResponse response)
24         throws ServletException, IOException {
25         request.getRequestDispatcher("/WEB-INF/pages/aboutus.jsp").forward(request, response);
26     }
27
28     @Override
29     protected void doPost(HttpServletRequest request, HttpServletResponse response)
30         throws ServletException, IOException {
31         // process form data
32     }
33 }
34
```

Figure 95 - Showing evidence of About us page

UI/US Design for About us page

- Task Description: Sakshyam created the About Us page's design to reflect Auratimes' brand and keep it current, readable and interesting with modern touch.

Implementation Steps:

- Created a wireframe layout in Figma showing the structure.
- Used a simple font with perfect colour combination in text.
- Created a team members introduction showing the details of every individuals.

Evidence: Screenshot showing the about us page:

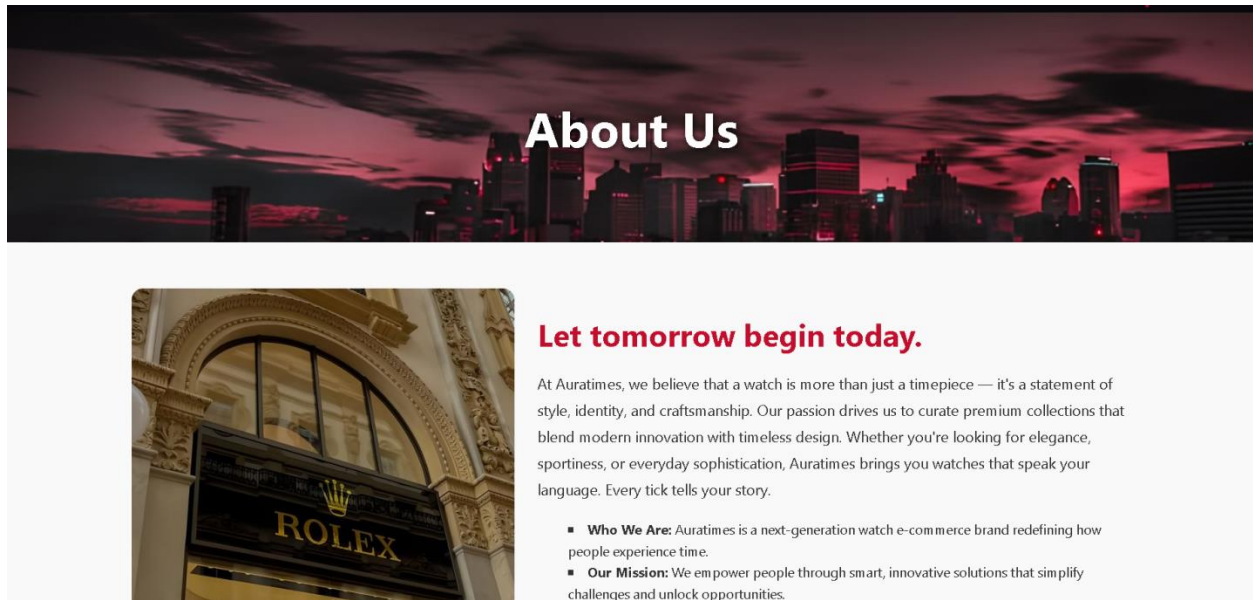


Figure 96 - Final implementation of about us page

Niran Bhatta

Completed Store Location Page Feature

- **Task Description:** Niran led the work on developing the Store Location page of Auratimes' online store. Because of this, users can quickly access directions to stores on an interactive map and get clear contact information.

Implementation Steps:

- I built and developed the Store Location page using JSP, HTML, CSS and JavaScript.
- A Google Maps iframe was placed to show where the store is.
- Store address, contact details, email and opening times are now arranged beside the map for easy reference.

Evidence:

- Screenshot of the JSP file of store location :

UI/UX Design for store Location Page

- Task Description: Niran created a simple and clear interface, making sure you can find both visual and text details alongside any clutter.

Implementation Steps:

- Made a design in Figma with the map and contact information next to each other.
- Used CSS to make the screen split and gave the elements rounded shapes, smooth shadows and hover states.
- Made some pages scroll, added padding and included clear scroll indicators.

Evidence:

- Screenshot of the final webpage

7.2) Critical Analysis.

Challenge 1 Yathartha Shrestha: Having XAMPP installed and accessing the database.

Explanation: It was challenging to get XAMPP (used to host the site and database) working properly. We had a problem with it having the same "tomcat" (port) as all our other applications.

Evidence: We got error messages from XAMPP about port problems.

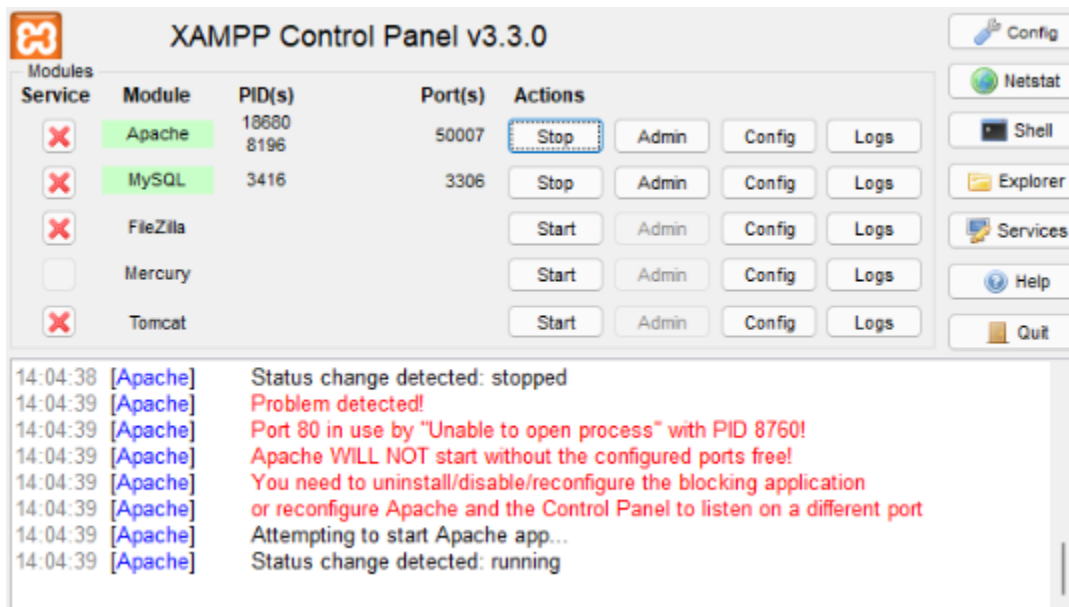


Figure 97 Error for Challenges 1

How we fixed it: We went through XAMPP's messages and found the colliding "port" and changed XAMPP to use different ones.

Challenge 2 Sujal Parajuli: Making the website "think" and shift data with Servlets and Controllers.

Explanation: It was hard to make the website's "brains" (Servlets and Controllers) react to what we were doing (e.g., clicking buttons) and shift information around.

```
1 package com.islington.controller;
2
3 import jakarta.servlet.ServletException;
4
5 /**
6  * Servlet implementation class Home
7  */
8 @WebServlet(asyncSupported = true, urlPatterns = { "/contact" })
9 public class ContactController extends HttpServlet {
10     private static final long serialVersionUID = 1L;
11
12     /**
13      * @see HttpServlet#HttpServlet()
14      */
15     public ContactController() {
16         super();
17         // TODO Auto-generated constructor stub
18     }
19
20     @Override
21     protected void doGet(HttpServletRequest request, HttpServletResponse response)
22         throws ServletException, IOException {
23         request.getRequestDispatcher("/WEB-INF/pages/contact.jsp").forward(request, response);
24     }
25
26     @Override
27     protected void doPost(HttpServletRequest request, HttpServletResponse response)
28         throws ServletException, IOException {
29         // process form data
30     }
31
32 }
```

Figure 98 Error for Challenges 2

Evidence: Sometimes, information got lost in transit between pages, and the website had no clue what to show.).

How we fixed it: We double-checked the way the pages get and pass information, making sure everything was set.

Challenge 3 Sakshyam Kafle: External CSS and JSP was not linked properly

Explanation: Due to the incorrect path CSS files and JSP files was not load properly which was creating the problem in about us page.

Evidence: Browser developer tools showing 404 errors.

How we remedied it: I carefully checked the folder structure and used `${pageContext.request.contextPath}` in JSP to dynamically resolve the correct path and moved all the assets into a structured folder.

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2 <!DOCTYPE html>
3 <html>
4 <head>
5     <meta charset="UTF-8">
6     <title>About Us | Auratimes</title>
7     <link rel="stylesheet" href="${pageContext.request.contextPath}/css/aboutus.css">
8 </head>
9 <body>
10
```

Figure 99 Fixing the Challenges 3

Challenge 4 Niran Bhatta: Contributed to this problem and fixed it.

Problem Name: Store Location JSP is not accessible since the location is outside WEB-INF.

Explanation: The storelocation.jsp page was originally situated directly beneath the webapp directory, below the WEB-INF directory. Therefore, it was not directly accessible through the application's servlet mappings, so that it could not be properly referenced from the home page or elsewhere on the site using servlet links.

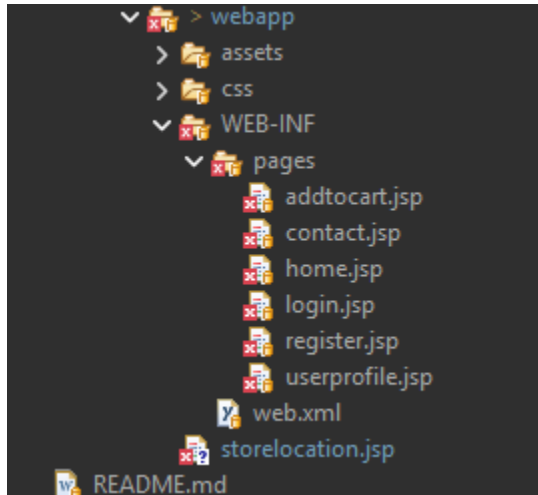


Figure 100 Error for Challenges 4

Evidence: Attempts to access the Store Location section from the home page resulted in errors because the servlet could not locate the JSP file.).

How we fixed it: We accomplished this by placing the storelocation.jsp file in the WEB-INF/pages directory. We then implemented a new servlet controller, named StoreLocationController.java, to handle requests to the store location page. Finally, we established the link in the web site's navigation (on home.jsp or an equivalent file) to point to the URL pattern that is mapped to the StoreLocationController servlet (which would subsequently handle the request to storelocation.jsp).

8) Tools Used:

8.1) XAMPP

Tools Used:

i) XAMPP

XAMPP is a free open-source adaptable solution stack which allows developers to establish basic testing conditions for web applications without needing external servers. Apache Friends created XAMPP as an integrated bundle which contains the Apache HTTP Server alongside MySQL/MariaDB as the database, server-side PHP functionality and phpMyAdmin administration platform for database management.



Figure 101 Tools used 1

Usage: Hosting the web application was done using XAMPP as the server environment on my local machine. MySQL databases were created and managed using phpMyAdmin and running the application locally before putting it online was made possible.

- Evidence: Screenshot of XAMPP Control panel where Apache and MySQL are running.

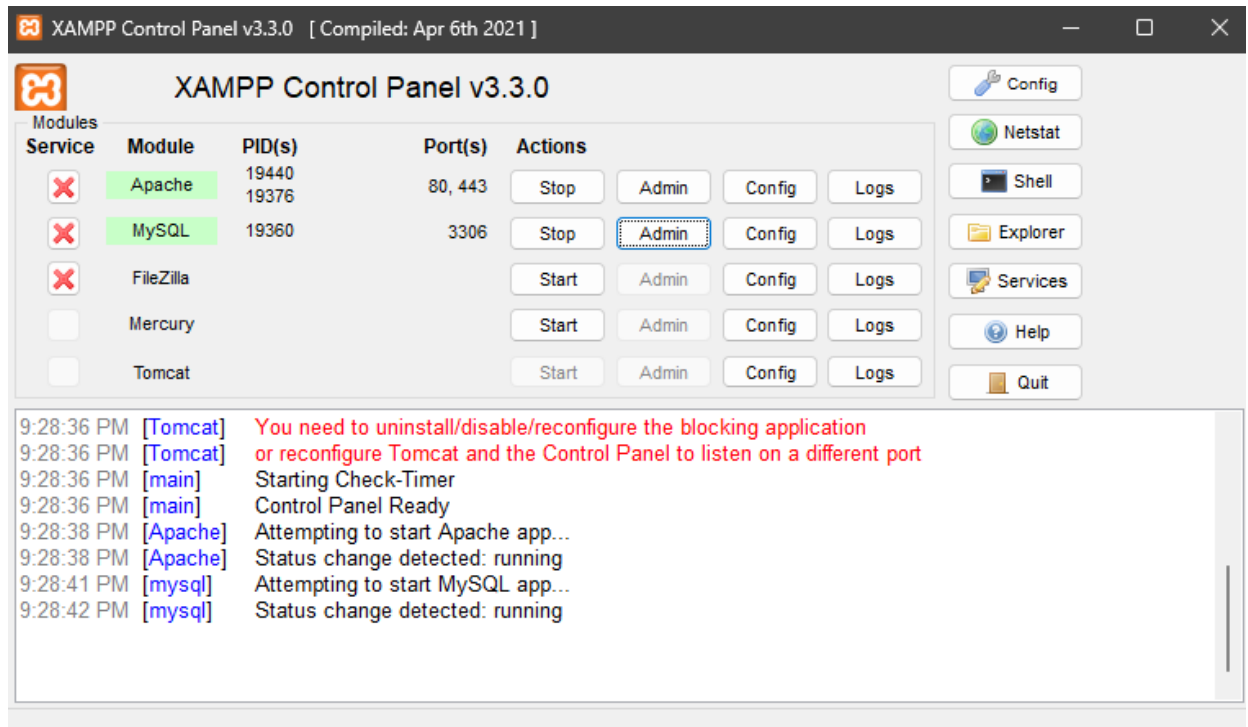


Figure 102 - SS of XAMPP control panel

ii) 8.2 Eclipse IDE

Eclipse is an open-source, cross-platform Integrated Development Environment (IDE) mainly for Java application development that is completely free. Its plug-in architecture is designed to be able to support most other programming languages.

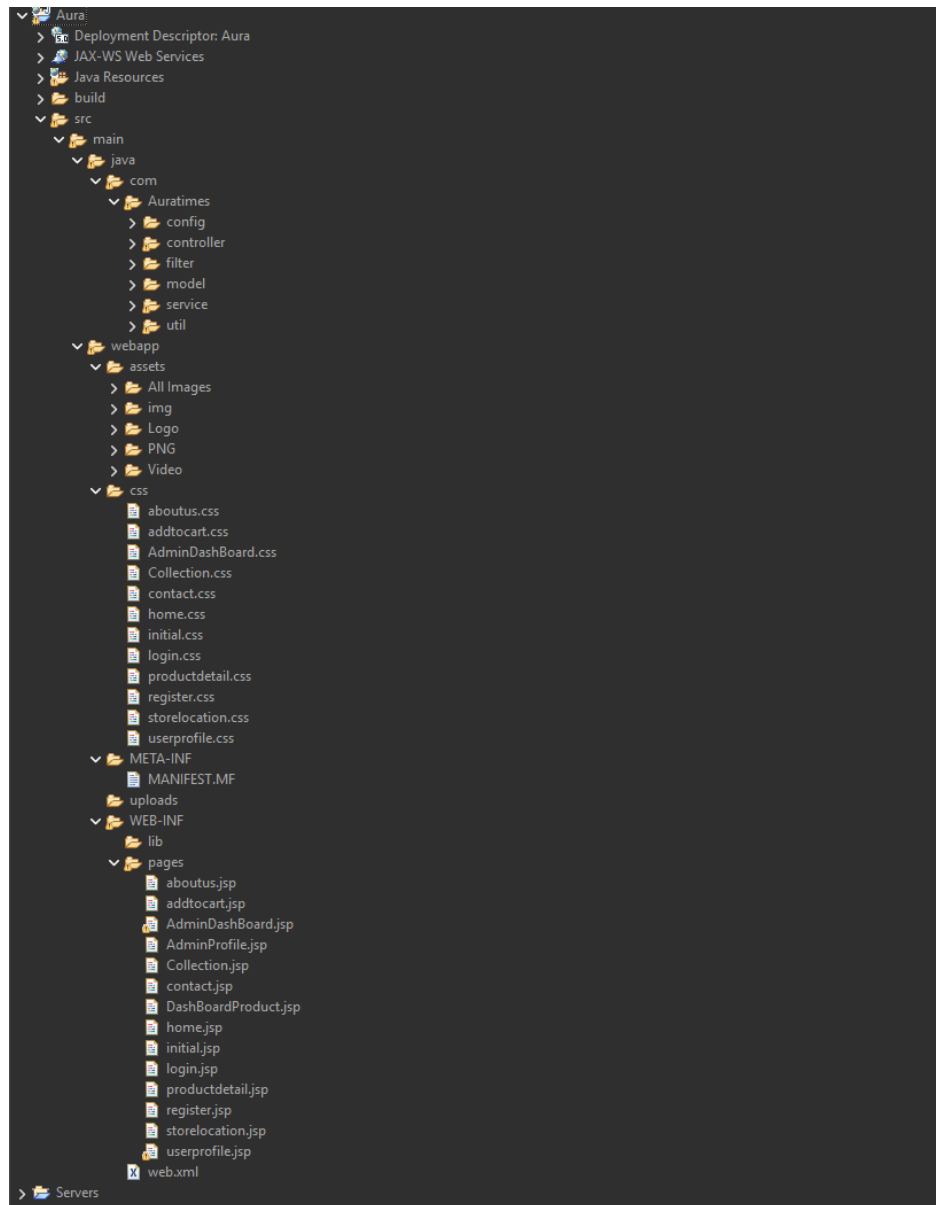


Figure 103 Tools used 2

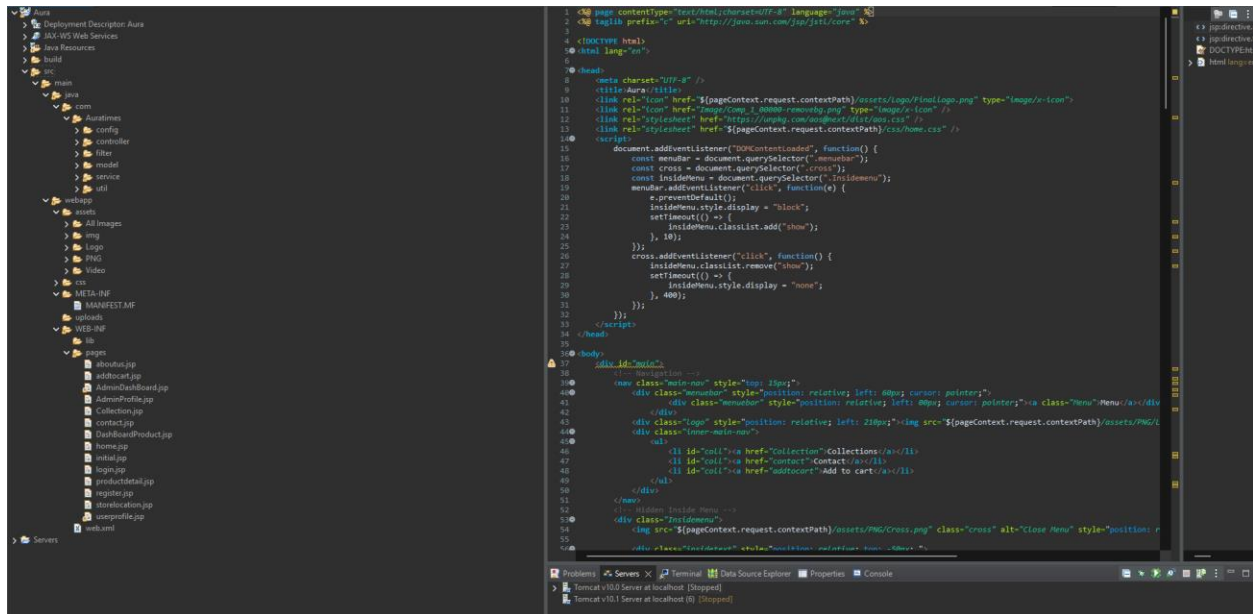
Usage: Java Servlet and JSP code was developed and managed with Eclipse. It allowed me to easily configure the project, solve problems and allow deployment with Apache Tomcat.

Evidence:

- Screenshot of Eclipse workspace showing folder structure



- **Screenshot of code editor with a JSP file open**



8.3)

iii) **Figma:**

Figma enables users to build webpage designs and mobile app interfaces through web browser access directly from their design website at no cost. Figma allows several designers to edit design projects in real-time like working on a shared Google Doc.

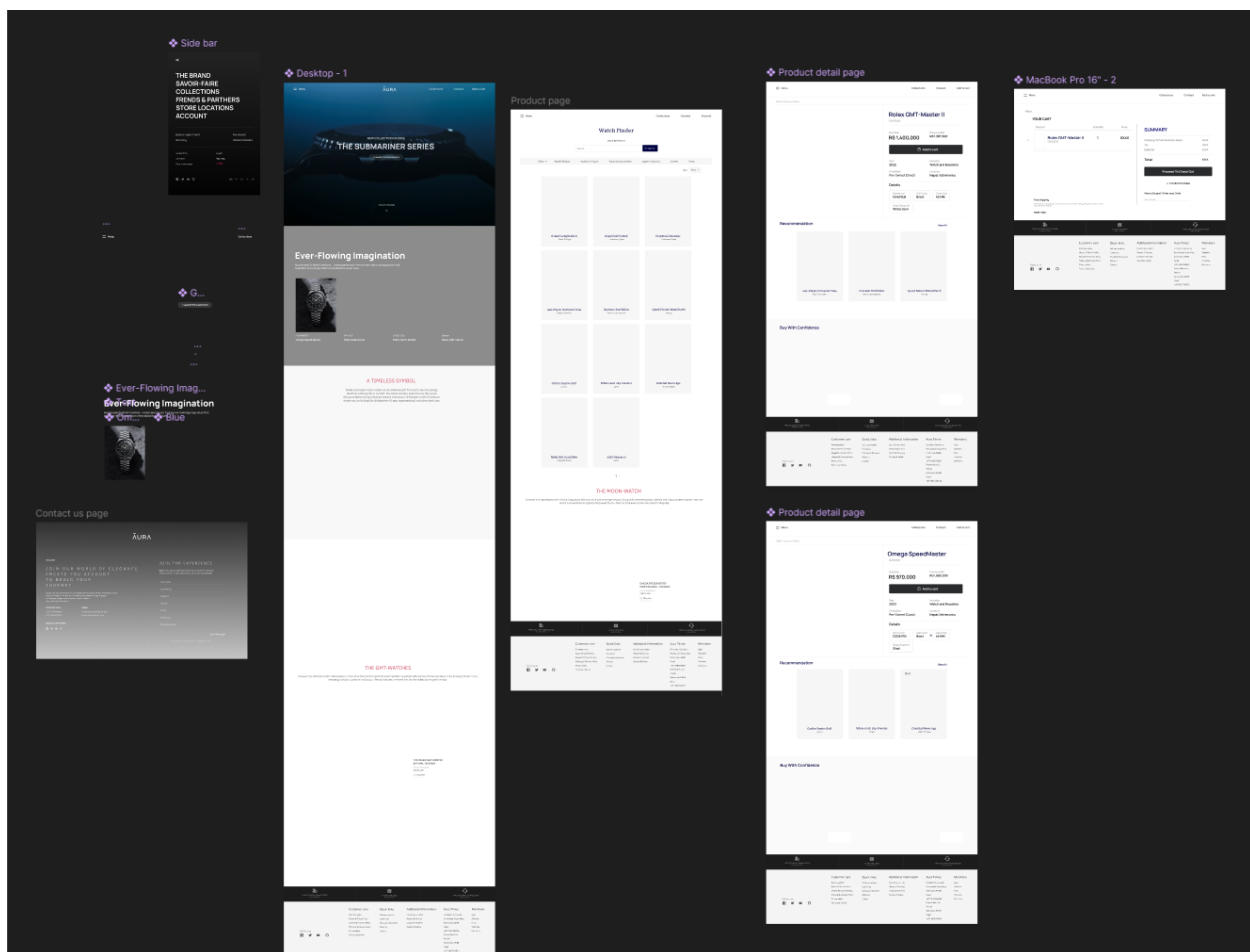


Figure 104 Tools used 3

Usage: Figma helped us design and test how the web application's user interface would work. The login page, the dashboard and product management tools were all designed and wireframed.

Evidence:

- Screenshot of Figma project showing the layout



iv) 8.4) **Apache Tomcat:**

Tomcat Apache serves as a leading open-source Java solution which performs Java web technologies including Servlets, JSP, and WebSocket through its servlet container functionality. The Apache Software Foundation created a light-weight web application execution environment that enables Java web applications to run efficiently through the absence of full Java EE server requirements.

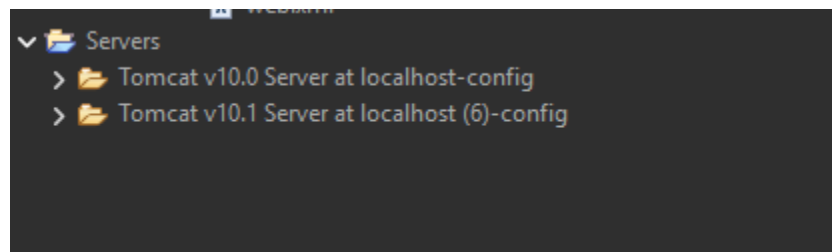


Figure 105 Tools used 4

Usage: The Java-based web application was deployed and run using Tomcat as its servlet container. It made it possible to tie backend logic to the frontend JSP pages.

Evidence:

- **Screenshot of showing tomcat server running inside the eclipse**



i) 8.5) **MySQL Connector:**

Applications use MySQL Connector to establish links between MySQL databases through its database driver functions. The connector supports multiple languages which include Java, Python and .NET and offers SSL security along with connection pooling and transactional procedures capabilities.



Figure 106 Tools used 5

Usage: A connector was included in the project so that servlets could manage the CRUD operations on the MySQL database. It worked to both retrieve and save data related to users, products and orders.

Evidence: Screenshot of database connection code

```
1 package com.Auratimes.config;
2 //Importing
3 import java.sql.Connection;
4
5 public class DbConfig {
6
7     // Database configuration information
8     private static final String DB_NAME = "aura"; // Name of the database
9     private static final String URL = "jdbc:mysql://localhost:3306/" + DB_NAME; // Full JDBC URL including the database name and
10     private static final String USERNAME = "root"; // Username
11     private static final String PASSWORD = ""; // Password
12
13     // This method to create and return a connection to the database
14     public static Connection getDbConnection()
15         throws SQLException, ClassNotFoundException {
16         Class.forName("com.mysql.cj.jdbc.Driver"); // Loading the MySQL
17         return DriverManager.getConnection(URL, USERNAME, PASSWORD);
18     }
19 }
20 }
```

9) Dynamic user profile:

Users should access a secure interface which allows dynamic personal information updates including full name, username, date of birth and gender together with email address password and phone number.

Implementation Details

Servlet Mapping: The UserprofileController servlet responds to both GET and POST HTTP methods which it routes through the /userprofile URL pattern. Future enhancement possibilities including profile picture uploads will be possible because this servlet is annotated with MultipartConfig.

```
23 @WebServlet(asyncSupported = true, urlPatterns = { "/userprofile" })
24 @MultipartConfig(
25     fileSizeThreshold = 1024 * 1024 * 2, // 2MB
26     maxFileSize = 1024 * 1024 * 10, // 10MB
27     maxRequestSize = 1024 * 1024 * 50 // 50MB
28 )
```

Figure 107 - Code showing servlet mapping

Handling GET Requests: The servlet uses GET requests to forward requests toward the userprofile.jsp page for displaying the user profile form with pre-populated data.

```
34● @Override
35 protected void doGet(HttpServletRequest request, HttpServletResponse response)
36     throws ServletException, IOException {
37     request.getRequestDispatcher("/WEB-INF/pages/userprofile.jsp").forward(request, response);
38 }
```

Figure 108 - Code showing servlet uses GET requests

Handling POST Requests: Extracts the data which was submitted from user from request parameters. Then ValidationUtil validates those data and if any validation fails an appropriate status message is set.

```
39  @Override
40
41  protected void doPost(HttpServletRequest request, HttpServletResponse response)
42      throws ServletException, IOException {
43
44      String fullName = request.getParameter("FullName");
45      String username = request.getParameter("Username");
46      String dateOfBirth = request.getParameter("Dateobirth");
47      String gender = request.getParameter("Gender");
48      String email = request.getParameter("Email");
49      String password = request.getParameter("Password");
50      String phoneNumber = request.getParameter("Phonenumber");
51
52      RequestDispatcher dispatcher = request.getRequestDispatcher("/WEB-INF/pages/userprofile.jsp");
53  }
```

Figure 109 - Code showing servlet uses POST requests

10) Meeting Logs**Meeting Log - 1****Day – 1****Date – 21th April, 2025****Start Time – 1:00 PM****End Time – 2:45 PM****Location – Kumari hall, Islington College**

We gathered during the first class of our Advanced Programming course to generate project proposals among team members. The group developed a watch resale platform because it enables users to trade both new and used branded watch products. Our website received the name "Auratimes" due to its uniqueness which matches perfectly with both the watch theme and time concept.

The team prepared and discussed a basic project proposal throughout that session. The proposal explained our website's primary concept together with the functionality we wanted to include which consisted of user authentication and watch listing and payment security features. The proposed application tools consist of Java, JSP and MySQL. The proposed document serves to present our plan to the teacher while leading our project execution.

All went smoothly in our meeting until we settled on making our project basic to meet our deadline. The next team session will finalize the proposal before assigning work divisions to team members. Work will begin on designing the website structure alongside creating a database design.

Meeting Log - 2**Day – 2****Dates – 12th April, 2025****Start Time – 11:30 PM****End Time – 2:00 PM****Location – Boys Hostel, Charkhal Road**

Our team conducted its technical project meeting on Day 2. We established the database structure before starting to normalize it for eliminating redundant data. We generated a basic Entity Relationship Diagram (ERD) which demonstrated the relational pattern between users, products, orders along with other database objects. This allowed us to determine proper methods for handling database storage and data linkages.

Our team examined the visual aspects that the user interface should present to visitors. People from the group proposed website appearance elements such as the homepage and product listings and login/signup pages among others. Most group members chose to design an easy-to-use layout with simple appearance which incorporates contemporary design elements and fluid movement between pages. The group discussed the implementation of JSP alongside CSS alongside possible animation techniques to prettify their website.

All group members received separate assignments to carry out after concluding the meeting session. The assignment split members between frontend design and backend development and database construction responsibilities. Every team member becomes aware of their responsibilities after this way they can begin working independently before the next meeting.

Meeting Log - 3**Day – 3****Dates – 16th April, 2025****Start Time – 10:00 PM****End Time – 12:00 PM****Location – Khaja Time, Kumari Marg**

The team reunited for a new working session on Day 3 to further develop the project. Although one member was absent from the session because of personal issues we delivered a complete report about our team discussions and work progress to him afterward. The team members who attended the meeting waited together to execute all pending tasks.

The team accomplished approximately 60% of project work by this day. Team members dedicated most of their efforts to fixing errors that produced database and backend problems preventing the website from running correctly. All team members joined forces to detect the coding and database problems which we resolved while meeting.

We conducted tests on everything to validate proper system functionality after resolving encountered issues. We finished our work after conducting the meeting and confirming both the progress and fixes satisfied us. The work day proved successful because it accelerated our completion of the project schedule.

Meeting Log - 4**Day – 4****Dates – 21th April, 2025****Start Time – 12:00 PM****End Time – 2:45 PM****Location – Khaja Time, Kumari Marg**

The team assembled for the last meeting prior to the Milestone 1 deadline on Day 4. The team aimed to finish the report tasks which were part of the project. Together we wrote and checked the report to provide a clear understanding of our concept alongside our database structure and interface blueprint as well as our current accomplishments.

Our team performed final verification on all Milestone 1 elements including the proposal design and ERD with normalization and initial design implementation. We ensured each project section met all requirements along with being well arranged. Our team completed the submission of Milestone 1 after performing a last review.

We achieved confidence about our advancement once the submission process was complete. All of us spent a day that felt both demanding and successful. We finished all necessary work before issuing our home departure for the day feeling satisfied about our on-time deadline completion.

Meeting Log - 5**Day – 5****Dates – 26th April, 2025****Start Time – 11:30 PM****End Time – 3:45 PM****Location – Kumari Hall, Islington College**

Our team worked on CRUD operations together with the admin dashboard development during Day 5. We worked on enhancing frontend and backend elements of data creation, reading, and updating and deletion operations. The team moved forward with designing as well as connecting an administrative panel that enables product management and user management while permitting straightforward order administration.

The session required our team to solve numerous problems particularly when we worked on linking frontend components to backend operations and managing the database queries. Together with proper research we successfully became familiar with the problems and solved them. The whole group dedicated their efforts to produce solutions which addressed our encountered problems.

The team managed to build operational base versions of the CRUD functions together with the admin dashboard before concluding the workday. The productive session resulted in a system that was starting to form its shape according to our satisfaction. We finished our work and left for the day after performing all required checks.

Meeting Log - 6**Day – 6****Dates - 5th May, 2025****Start Time – 12:00 PM****End Time – 7:45 PM****Location – Coffee Station, Islington College**

The team assembled on Day 6 right before Milestone 2 deadline to finish all remaining work after class in the college environment. We partnered together to finalize the report while part of the team worked on coding tasks that remained after the report completion.

The creation of a class diagram along with descriptions about all project classes proved to be the most laborious task. We needed to devise a detailed approach for making it suitable with the existing code structure. At the same time others concentrated on ensuring system stability while verifying the backend functionality was complete.

The day stretched into many hours of work until we finished together while offering continuous support to our classmates. Completing all work required extensive checking in addition to reviewing the report and code so we arrived at home after dark. We felt content with finishing Milestone 2 despite the lengthy hours of work.

11) Conclusion:

Our group was able to develop "Aura Times," an online shopping website for watches. The main aim was to create a welcoming website by which customers can easily view, choose, and buy watches. We used technologies including HTML, CSS, JavaScript, JSP, and MySQL to implement the website. Throughout the project, we divided the workload among our team members: Sujal, Yathartha, Niran, and Sakshyam. Each member had assigned tasks, such as designing the website, backend development, designing the store location page, and report development.

We faced some problems during development. One of them was setting up the XAMPP environment and linking it to the database. We also had to carefully craft the controllers and servlets to control user activities and data. Having the website look good and work correctly on devices with different screen sizes using CSS was also a challenge. We also had an issue with the storelocation.jsp file not being linked correctly, which was resolved by Niran Bhatta by moving the file and implementing a controller. We also encountered some technical problems. For example, we got a "ClassNotFoundException" error, which we solved by adding the MySQL driver to our project. We also needed to correct errors in our SQL statements and servlet mapping errors that gave rise to "404 Not Found" errors. Despite the odds, we were able to build a functional e-commerce website with user authentication, product listing, shopping cart, and order management functionality. The website includes an admin panel for managing products and users.

In conclusion, this project allowed us to use our advanced programming skills to develop a real-world program. We gained valuable experiences in teamwork, solving problems, and web development. Not only did the experience fulfil the requirements of our module, but it also equipped us with the practical skills which will be beneficial to us in the future.